



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΕΣΣΑΛΙΑΣ

ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

ΕΝΣΩΜΑΤΩΣΗ ΤΥΠΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ ΣΥΣΚΕΥΩΝ ΣΕ ΕΞΥΠΝΟ ΣΠΙΤΙ ΜΕ ΧΡΗΣΗ RASPBERRY PI

ΠΑΡΑΘΥΡΑΣ ΔΑΥΙΔ-ΠΑΝΑΓΙΩΤΗΣ

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

ΥΠΕΥΘΥΝΟΣ

ΔΑΔΑΛΙΑΡΗΣ ΑΝΤΩΝΙΟΣ

Λαμία 12/05 έτος 2026



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΘΕΣΣΑΛΙΑΣ

ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ

ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ

ΕΝΣΩΜΑΤΩΣΗ ΤΥΠΙΚΩΝ ΗΛΕΚΤΡΟΝΙΚΩΝ ΣΥΣΚΕΥΩΝ ΣΕ ΕΞΥΠΝΟ ΣΠΙΤΙ ΜΕ ΧΡΗΣΗ RASPBERRY PI

ΠΑΡΑΘΥΡΑΣ ΔΑΥΙΔ-ΠΑΝΑΓΙΩΤΗΣ

ΠΤΥΧΙΑΚΗ ΕΡΓΑΣΙΑ

ΥΠΕΥΘΥΝΟΣ

ΔΑΔΑΛΙΑΡΗΣ ΑΝΤΩΝΙΟΣ

Λαμία 12/05 έτος 2026



UNIVERSITY OF
THESSALY

SCHOOL OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE & TELECOMMUNICATIONS

INTEGRATION OF STANDARD
ELECTRONIC DEVICES IN A SMART HOME
USING RASPBERRY PI

PARATHYRAS DAVID-PANAGIOTIS

FINAL THESIS

ADVISOR

DADALIARIS ANTONIOS

Lamia 12/05 year 2026

«Με ατομική μου ευθύνη και γνωρίζοντας τις κυρώσεις ⁽¹⁾, που προβλέπονται από της διατάξεις της παρ. 6 του άρθρου 22 του Ν. 1599/1986, δηλώνω ότι:

1. Δεν παραθέτω κομμάτια βιβλίων ή άρθρων ή εργασιών άλλων αυτολεξεί **χωρίς να τα περικλείω σε εισαγωγικά** και χωρίς να αναφέρω το συγγραφέα, τη χρονολογία, τη σελίδα. Η αυτολεξεί παράθεση χωρίς εισαγωγικά χωρίς αναφορά στην πηγή, είναι λογοκλοπή. Πέραν της αυτολεξεί παράθεσης, λογοκλοπή θεωρείται και η παράφραση εδαφίων από έργα άλλων, συμπεριλαμβανομένων και έργων συμφοιτητών μου, καθώς και η παράθεση στοιχείων που άλλοι συνέλεξαν ή επεξεργάστηκαν, χωρίς αναφορά στην πηγή. Αναφέρω πάντοτε με πληρότητα την πηγή κάτω από τον πίνακα ή σχέδιο, όπως στα παραθέματα.
2. Δέχομαι ότι η αυτολεξεί **παράθεση χωρίς εισαγωγικά**, ακόμα κι αν συνοδεύεται από αναφορά στην πηγή σε κάποιο άλλο σημείο του κειμένου ή στο τέλος του, είναι αντιγραφή. Η αναφορά στην πηγή στο τέλος π.χ. μιας παραγράφου ή μιας σελίδας, δεν δικαιολογεί συρραφή εδαφίων έργου άλλου συγγραφέα, έστω και παραφρασμένων, και παρουσίασή τους ως δική μου εργασία.
3. Δέχομαι ότι υπάρχει επίσης περιορισμός στο μέγεθος και στη συχνότητα των παραθεμάτων που μπορώ να εντάξω στην εργασία μου εντός εισαγωγικών. Κάθε μεγάλο παράθεμα (π.χ. σε πίνακα ή πλαίσιο, κλπ), προϋποθέτει ειδικές ρυθμίσεις, και όταν δημοσιεύεται προϋποθέτει την άδεια του συγγραφέα ή του εκδότη. Το ίδιο και οι πίνακες και τα σχέδια
4. Δέχομαι όλες τις συνέπειες σε περίπτωση λογοκλοπής ή αντιγραφής.

Ημερομηνία: 12/05/2026

Ο Δηλών

ΠΑΡΑΘΥΡΑΣ ΔΑΥΙΔ-ΠΑΝΑΓΙΩΤΗΣ

(1) «Όποιος εν γνώσει του δηλώνει ψευδή γεγονότα ή αρνείται ή αποκρύπτει τα αληθινά με έγγραφη υπεύθυνη δήλωση του άρθρου 8 παρ. 4 Ν. 1599/1986 τιμωρείται με φυλάκιση τουλάχιστον τριών μηνών. Εάν ο υπαίτιος αυτών των πράξεων σκόπευε να προσπορίσει στον εαυτόν του ή σε άλλον περιουσιακό όφελος βλάπτοντας τρίτον ή σκόπευε να βλάψει άλλον, τιμωρείται με κάθειρξη μέχρι 10 ετών.»

ΠΕΡΙΛΗΨΗ

Τα περισσότερα σπίτια είναι γεμάτα τυπικές ηλεκτρονικές συσκευές — τηλεοράσεις, κλιματιστικά, ηχοσυστήματα — που εξακολουθούν να λειτουργούν με υπέρυθρα τηλεχειριστήρια και παραμένουν εκτός του σύγχρονου έξυπνου σπιτιού, το οποίο θεωρεί δεδομένο ότι κάθε συσκευή επικοινωνεί μέσω δικτύου. Η παρούσα εργασία περιγράφει ένα σύστημα που ενσωματώνει αυτές τις καθημερινές συσκευές στο Home Assistant με χρήση Raspberry Pi, χωρίς να στέλνει δεδομένα στο cloud και χωρίς να απαιτεί αντικατάσταση αυτών.

Το σύστημα χωρίζεται σε δύο μέρη. Ένα Raspberry Pi εξοπλισμένο με ένα ANAVI Infrared pHAT τοποθετείται κοντά στη συσκευή-στόχο και λειτουργεί ως «γέφυρα»: μπορεί να μάθει υπέρυθρα σήματα από οποιοδήποτε υπάρχον φυσικό τηλεχειριστήριο και να τα αναπαράγει κατ' επιλογή. Από την άλλη πλευρά, μια προσαρμοσμένη ενσωμάτωση (integration) για το Home Assistant επικοινωνεί με τη «γέφυρα» μέσω MQTT, δημιουργώντας κουμπιά και αισθητήρες για κάθε πλήκτρο που έχει μάθει η «γέφυρα». Όλες οι ενέργειες — η καταγραφή ενός νέου πλήκτρου, η μετονομασία ή η διαγραφή του, ακόμη και ο συνδυασμός πολλαπλών πατημάτων σε μια μακροεντολή «εικονικού πλήκτρου» — εκτελούνται από το περιβάλλον του Home Assistant, χωρίς να χρειάζεται καμία αλληλεπίδραση με το Pi μετά την αρχική εγκατάσταση. Πολλαπλές «γέφυρες» μπορούν να λειτουργούν παράλληλα, χρησιμοποιώντας διαφορετικά προθέματα MQTT topics, οπότε η ίδια διάταξη κλιμακώνεται φυσικά από ένα δωμάτιο σε ολόκληρο σπίτι.

Οι εκδόσεις διαχειρίζονται από ένα pipeline συνεχούς ενσωμάτωσης (CI) που δημοσιεύει νέα έκδοση μόνο όταν περάσουν επιτυχώς οι έλεγχοι HACS και Hassfest, γεγονός που σημαίνει ότι η ενσωμάτωση μπορεί να εγκατασταθεί μέσω του Home Assistant Community Store όπως οποιοδήποτε άλλη επέκταση της κοινότητας. Το αποτέλεσμα είναι ένας μικρός, πλήρως τοπικός και ανοιχτού κώδικα τρόπος ώστε τυπικές ηλεκτρονικές συσκευές να ενταχθούν σε ένα σύγχρονο οικοσύστημα έξυπνου σπιτιού.

Λέξεις-κλειδιά: Έξυπνο σπίτι, Διαδίκτυο των Πραγμάτων (IoT), Home Assistant, Raspberry Pi, υπέρυθρο τηλεχειριστήριο, MQTT, ενσωμάτωση συσκευών, λογισμικό ανοιχτού κώδικα

ABSTRACT

Most homes are full of standard electronic devices — TVs, Air Conditioners, Stereo Amplifiers — that are still operated by infrared remotes and sit outside the modern smart-home, which expects everything to speak over the network. This thesis describes a system that integrates these everyday devices into Home Assistant using a Raspberry Pi, without sending any data to the cloud and without replacing the original hardware.

The system is split into two parts. A Raspberry Pi fitted with an ANAVI Infrared pHAT sits near the target device and acts as a bridge: it can learn IR signals from any existing physical remote and play them back on command. On the other side, a custom Home Assistant integration talks to the bridge over MQTT, creating buttons and sensors for every key the bridge has learned. Everything — recording a new key, renaming it, deleting it, or stitching several presses into a single "virtual key" macro — is done from the Home Assistant interface, with no need to touch the Pi after setup. Multiple bridges can run in parallel by using different MQTT topic prefixes, so the same setup scales naturally from one room to a whole house.

Releases are handled by a Continuous Integration (CI) pipeline that only ships a new version once HACS and Hassfest validation checks pass, which means the integration can be installed through the Home Assistant Community Store like any other community add-on. The result is a small, fully local, open-source way to bring standard electronic devices into a modern smart-home ecosystem.

Keywords: Smart home; Internet of Things (IoT); Home Assistant; Raspberry Pi; infrared remote control; MQTT; device integration; open-source software

Table of Contents

ΠΕΡΙΛΗΨΗ	I
ABSTRACT	III
ΚΕΦΑΛΑΙΟ 1 – ΕΙΣΑΓΩΓΗ.....	2
ΑΥΤΟΜΑΤΙΣΜΟΣ ΟΙΚΙΑΣ ΚΑΙ ΥΠΕΡΥΘΡΕΣ ΣΥΣΚΕΥΕΣ	1.1..... 2
ΣΚΟΠΟΣ ΚΑΙ ΔΟΜΗ ΤΗΣ ΕΡΓΑΣΙΑΣ	1.2 3
ΚΕΦΑΛΑΙΟ 2 – ΒΙΒΛΙΟΓΡΑΦΙΚΗ ΕΠΙΣΚΟΠΗΣΗ	4
ΕΙΣΑΓΩΓΗ	2.1..... 4
ΈΞΥΠΝΟ ΣΠΙΤΙ ΚΑΙ ΔΙΑΔΙΚΤΥΟ ΤΩΝ ΠΡΑΓΜΑΤΩΝ	2.2 5
ΕΙΣΑΓΩΓΗ	2.2.A..... 5
ΟΡΙΣΜΟΙ ΚΑΙ ΤΥΠΙΚΕΣ ΑΡΧΙΤΕΚΤΟΝΙΚΕΣ	2.2.B..... 5
ΠΛΑΤΦΟΡΜΕΣ ΟΙΚΙΑΚΟΥ ΑΥΤΟΜΑΤΙΣΜΟΥ	2.2.Γ 6
ΣΥΣΧΕΤΙΣΗ ΜΕ ΤΗΝ ΠΑΡΟΥΣΑ ΕΡΓΑΣΙΑ	2.2.Δ..... 7
ΥΠΕΡΥΘΡΗ ΕΠΙΚΟΙΝΩΝΙΑ ΚΑΙ ΤΗΛΕΧΕΙΡΙΣΜΟΣ	2.3..... 8
ΕΙΣΑΓΩΓΗ	2.3.A..... 8
ΠΡΩΤΟΚΟΛΛΑ IR (NEC, RC5, SONY SIRC)	2.3.B..... 8
ΚΑΤΑΓΡΑΦΗ ΚΑΙ ΑΝΑΠΑΡΑΓΩΓΗ ΣΗΜΑΤΩΝ	2.3.Γ 9
ΣΥΣΧΕΤΙΣΗ ΜΕ ΤΗΝ ΠΑΡΟΥΣΑ ΕΡΓΑΣΙΑ	2.3.Δ..... 10
ΠΡΩΤΟΚΟΛΛΟ MQTT	2.4..... 11
ΕΙΣΑΓΩΓΗ	2.4.A..... 11
ΜΟΝΤΕΛΟ PUBLISH/SUBSCRIBE	2.4.B..... 11
QOS, RETAINED MESSAGES ΚΑΙ PERSISTENCE	2.4.Γ..... 12
ΣΥΣΧΕΤΙΣΗ ΜΕ ΤΗΝ ΠΑΡΟΥΣΑ ΕΡΓΑΣΙΑ	2.4.Δ..... 13
ΠΛΑΤΦΟΡΜΑ RASPBERRY PI ΚΑΙ ΑΝΑΒΙ INFRARED PHAT	2.5..... 14
ΕΙΣΑΓΩΓΗ	2.5.A..... 14
RASPBERRY PI	2.5.B 14
ΑΝΑΒΙ INFRARED PHAT	2.5.Γ 15
ΣΥΣΧΕΤΙΣΗ ΜΕ ΤΗΝ ΠΑΡΟΥΣΑ ΕΡΓΑΣΙΑ	2.5.Δ..... 16
HOME ASSISTANT ΚΑΙ HACS	2.6..... 17
ΕΙΣΑΓΩΓΗ	2.6.A..... 17
ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΤΟΥ HOME ASSISTANT	2.6.B..... 17
CUSTOM INTEGRATIONS ΚΑΙ ΔΙΑΝΟΜΗ ΜΕΣΩ HACS	2.6.Γ 18
ΣΥΣΧΕΤΙΣΗ ΜΕ ΤΗΝ ΠΑΡΟΥΣΑ ΕΡΓΑΣΙΑ	2.6.Δ..... 19
ΚΕΦΑΛΑΙΟ 3 – ΥΛΟΠΟΙΗΣΗ	20
ΕΙΣΑΓΩΓΗ	3.1 20
ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΤΟΥ ΣΥΣΤΗΜΑΤΟΣ	3.2 21
ΥΛΟΠΟΙΗΣΗ ΤΗΣ «ΓΕΦΥΡΑΣ» ΣΤΟ RASPBERRY PI	3.3 22
ΕΓΚΑΤΑΣΤΑΣΗ ΥΛΙΚΟΥ ΚΑΙ ΛΕΙΤΟΥΡΓΙΚΟΥ ΣΥΣΤΗΜΑΤΟΣ	3.3.A 22
ΛΟΓΙΣΜΙΚΟ «ΓΕΦΥΡΑΣ»	3.3.B..... 24

ΚΑΤΑΓΡΑΦΗ ΚΑΙ ΑΝΑΠΑΡΑΓΩΓΗ ΥΠΕΡΥΘΡΩΝ ΣΗΜΑΤΩΝ	3.3.Γ	25
ΔΟΜΗ ΑΠΟΘΗΚΕΥΣΗΣ ΣΥΣΚΕΥΩΝ ΚΑΙ ΠΛΗΚΤΡΩΝ	3.3.Δ	26
ΕΙΚΟΝΙΚΑ ΠΛΗΚΤΡΑ ΚΑΙ ΜΑΚΡΟΕΝΤΟΛΕΣ	3.3.Ε	27
ΥΠΗΡΕΣΙΑ SYSTEMD	3.3.ΣΤ	28
ΥΛΟΠΟΙΗΣΗ ΤΟΥ HOME ASSISTANT INTEGRATION	3.4	30
ΔΟΜΗ ΤΟΥ CUSTOM COMPONENT	3.4.Α	30
ΡΟΗ ΔΙΑΜΟΡΦΩΣΗΣ (CONFIG FLOW)	3.4.Β	31
ΟΝΤΟΤΗΤΕΣ BUTTON, SENSOR, NUMBER, TEXT	3.4.Γ	33
ΔΥΝΑΜΙΚΗ ΔΗΜΙΟΥΡΓΙΑ ΚΑΙ ΕΝΗΜΕΡΩΣΗ ΟΝΤΟΤΗΤΩΝ	3.4.Δ	35
ΠΡΩΤΟΚΟΛΛΟ ΕΠΙΚΟΙΝΩΝΙΑΣ MQTT	3.5	37
ΔΟΜΗ TOPIC ΚΑΙ ΠΡΟΘΕΜΑ ΓΙΑ ΠΟΛΛΑΠΛΕΣ «ΓΕΦΥΡΕΣ»	3.5.Α	37
ΜΗΝΥΜΑΤΑ ΕΝΤΟΛΗΣ ΚΑΙ ΕΝΗΜΕΡΩΣΗΣ ΚΑΤΑΣΤΑΣΗΣ	3.5.Β	38
ΑΥΤΟΜΑΤΟΠΟΙΗΜΕΝΗ ΔΗΜΟΣΙΕΥΣΗ ΕΚΔΟΣΕΩΝ	3.6	39
CI/CD PIPELINE ΣΤΑ GITHUB ACTIONS	3.6.Α	39
ΈΛΕΓΧΟΙ HACS ΚΑΙ HASSFEST	3.6.Β	41
ΔΗΜΟΣΙΕΥΣΗ ΕΚΔΟΣΕΩΝ ΣΤΟ HOME ASSISTANT COMMUNITY STORE	3.6.Γ	42
ΟΔΗΓΙΕΣ ΕΓΚΑΤΑΣΤΑΣΗΣ ΚΑΙ ΧΡΗΣΗΣ	3.7	45
<u>ΚΕΦΑΛΑΙΟ 4 – ΠΕΙΡΑΜΑΤΑ.....</u>		47
ΕΙΣΑΓΩΓΗ	4.1	47
ΕΦΑΡΜΟΓΗ ΣΕ ΠΡΑΓΜΑΤΙΚΕΣ ΣΥΣΚΕΥΕΣ	4.2	48
ΚΑΘΥΣΤΕΡΗΣΗ ΚΑΙ ΑΞΙΟΠΙΣΤΙΑ ΕΠΙΚΟΙΝΩΝΙΑΣ	4.3	51
ΑΝΤΟΧΗ ΣΕ ΣΦΑΛΜΑΤΑ (ΔΙΑΚΟΠΕΣ ΔΙΚΤΥΟΥ, ΕΠΑΝΕΚΚΙΝΗΣΕΙΣ)	4.4	52
ΠΕΡΙΟΡΙΣΜΟΙ ΚΑΙ ΠΡΟΒΛΗΜΑΤΑ	4.5	53
<u>ΚΕΦΑΛΑΙΟ 5 – ΣΥΜΠΕΡΑΣΜΑΤΑ.....</u>		55
ΑΝΑΚΕΦΑΛΑΙΩΣΗ ΚΑΙ ΑΞΙΟΛΟΓΗΣΗ ΑΠΟΤΕΛΕΣΜΑΤΩΝ	5.1	55
ΜΕΛΛΟΝΤΙΚΕΣ ΒΕΛΤΙΩΣΕΙΣ	5.2	56
<u>ΒΙΒΛΙΟΓΡΑΦΙΑ.....</u>		58

ΚΕΦΑΛΑΙΟ 1 – Εισαγωγή

Στο κεφάλαιο αυτό παρουσιάζεται το γενικό πλαίσιο και οι στόχοι της εργασίας. Παρόλο που οι τεχνολογίες οικιακού αυτοματισμού έχουν εξελιχθεί σημαντικά τα τελευταία χρόνια, πολλές οικιακές συσκευές εξακολουθούν να λειτουργούν μέσω υπέρυθρων τηλεχειριστηρίων, γεγονός που δυσκολεύει την ενσωμάτωσή τους σε σύγχρονα δικτυωμένα συστήματα. Στην **ενότητα 1.1** εξετάζεται το συγκεκριμένο ζήτημα και οι υπάρχουσες λύσεις που έχουν προταθεί για την αντιμετώπισή του. Στην **ενότητα 1.2** περιγράφεται ο σκοπός της εργασίας, τα βασικά κριτήρια σχεδιασμού που ακολουθήθηκαν, καθώς και η δομή των κεφαλαίων που ακολουθούν.

Αυτοματισμός οικίας και υπέρυθρες συσκευές

1.1

Ο οικιακός αυτοματισμός δεν είναι νέα ιδέα. Όπως καταγράφει ο **Rye (1999)** [1], οι πρώτες προσπάθειες χρονολογούνται από τη δεκαετία του 1970, με το πρωτόκολλο X10 να μεταφέρει εντολές μέσω των καλωδίων του ρεύματος του οικιακού ηλεκτρικού δικτύου. Στη συνέχεια, η εξέλιξη οδήγησε στα πλήρως δικτυωμένα οικοσυστήματα του Internet of Things, για τα οποία οι **Atzori, Iera και Morabito (2010)** [2] παρουσιάζουν αναλυτικά μέσα από τις αρχιτεκτονικές και τις εφαρμογές τους. Η μετάβαση στη σύγχρονη εποχή χαρακτηρίζεται από δύο βασικές αλλαγές: η συσκευή δεν είναι πλέον απλώς προγραμματιζόμενη, αλλά δικτυωμένη και η επικοινωνία μεταξύ συσκευών γίνεται μέσω κοινών δικτυακών πρωτοκόλλων. Πλατφόρμες όπως το Home Assistant [3], το openHAB [4] και το SmartThings [5] ενοποιούν, σε μόνο ένα περιβάλλον διαχείρισης, χιλιάδες συσκευές διαφορετικών κατασκευαστών και ο χρήστης τις χειρίζεται είτε μέσω εφαρμογής, είτε με φωνητικό βοηθό, είτε με αυτοματισμούς.

Ο υπέρυθρος έλεγχος παραμένει δημοφιλής, επειδή πρόκειται για μια τεχνολογία αξιόπιστη και οικονομική στην υλοποίησή της. Για τον λόγο αυτό, καθιερώθηκε ως πρότυπο για δεκαετίες και συνεχίζει να βρίσκεται σε ευρεία χρήση ακόμη και σήμερα. Ωστόσο, ενώ οι νεότερες συσκευές ενσωματώνονται εύκολα σε δίκτυα και πλατφόρμες αυτοματισμού, οι συσκευές που βασίζονται αποκλειστικά σε υπέρυθρες εντολές, παραμένουν σε μεγάλο βαθμό απομονωμένες από αυτά τα περιβάλλοντα.

Το πρόβλημα είναι ότι αυτές οι συσκευές δεν μπορούν να συνδεθούν στο διαδίκτυο. Δεν έχουν διεύθυνση IP ή API. Δε συνομιλούν με το router του σπιτιού και από τη σκοπιά ενός σύγχρονου συστήματος έξυπνου σπιτιού, ουσιαστικά, δεν υπάρχουν. Όποιος θέλει να εντάξει την τηλεόραση ή το κλιματιστικό του στους αυτοματισμούς του σπιτιού, έχει συνήθως δύο επιλογές: είτε να αντικαταστήσει τη συσκευή με μια καινούρια, smart έκδοση — κάτι ακριβό και οικολογικά αμφισβητήσιμο για συσκευές που δουλεύουν μια χαρά — είτε να παρεμβάλει μια «γέφυρα» ανάμεσα στο δίκτυό του και το υπέρυθρο τηλεχειριστήριο, που μεταφράζει τις δικτυακές εντολές σε υπέρυθρα σήματα.

Η δεύτερη επιλογή έχει εφαρμοστεί στην πράξη, τόσο σε εμπορικά προϊόντα, όσο και σε έργα ανοιχτού κώδικα. Χαρακτηριστικά παραδείγματα αποτελούν το Logitech Harmony Hub, το οποίο, σύμφωνα με τη Logitech (2021) **Error! Reference source not found.**, έχει πλέον αποσυρθεί από την αγορά και το Broadlink RM Pro [7], τα οποία προσφέρουν τη δυνατότητα ελέγχου υπέρυθρων συσκευών μέσω δικτύου. Παράλληλα, έργα ανοιχτού κώδικα, όπως το ESPHome IR [8] και το Tasmota IR [9], αξιοποιούν οικονομικούς μικροελεγκτές ESP8266 και ESP32 για την υλοποίηση αντίστοιχης λειτουργικότητας. Ωστόσο, οι περισσότερες από αυτές τις εμπορικές λύσεις φέρουν συμβιβασμούς: εξάρτηση από υπηρεσίες cloud, vendor lock-in και περιορισμένη ενσωμάτωση σε σύγχρονες πλατφόρμες αυτοματισμού. Οι αντίστοιχες open-source εναλλακτικές αποφεύγουν τα παραπάνω, αλλά απαιτούν υψηλό βαθμό τεχνικής εξοικείωσης για τη διαμόρφωσή τους. Στο **Κεφάλαιο 2** θα εξετάσουμε αυτές τις λύσεις πιο αναλυτικά. Για την παρούσα εργασία, το ζητούμενο είναι να καλυφθεί το συγκεκριμένο κενό με μια λύση που να είναι ταυτόχρονα τοπική, ανοιχτού κώδικα, οικονομική και υποστηρίζεται από την επικρατέστερη ανοιχτή πλατφόρμα έξυπνου σπιτιού: το Home Assistant.

Σκοπός και δομή της εργασίας

1.2

Σκοπός της παρούσας εργασίας είναι ο σχεδιασμός και η υλοποίηση ενός ολοκληρωμένου συστήματος, που εντάσσει τις παραδοσιακές υπέρυθρες οικιακές συσκευές, σε ένα σύγχρονο οικοσύστημα έξυπνου σπιτιού. Το σύστημα αποτελείται από δύο μέρη που λειτουργούν συντονισμένα. Το πρώτο είναι μια «γέφυρα» βασισμένη σε Raspberry Pi με υπέρυθρο δέκτη και πομπό, η οποία τοποθετείται κοντά στις συσκευές-στόχους και αναλαμβάνει την καταγραφή και αναπαραγωγή των υπέρυθρων σημάτων. Το δεύτερο μέρος είναι μια προσαρμοσμένη ενσωμάτωση (integration) για το Home Assistant [3], που κάνει διαθέσιμη τη λειτουργία στον χρήστη, μέσω του γραφικού περιβάλλοντος της πλατφόρμας.

Η συγκεκριμένη επιλογή σχεδιασμού δεν είναι τυχαία. Όπως αναλύθηκε στην προηγούμενη ενότητα, καμία από τις υπάρχουσες λύσεις δεν συνδυάζει ταυτόχρονα τέσσερα κριτήρια που θεωρούνται κρίσιμα: πλήρως τοπική λειτουργία χωρίς εξάρτηση από υπηρεσίες cloud, ανοιχτός κώδικας, native ενσωμάτωση με το Home Assistant (αντί για μια γενικής χρήσης MQTT «γέφυρα») και εύκολη εγκατάσταση μέσω του Home Assistant Community Store (HACS) [10]. Η εργασία στοχεύει στον συνδυασμό αυτών των τεσσάρων σε ένα ενιαίο και πρακτικό σύστημα.

Στο επίπεδο του τελικού χρήστη, το σύστημα προσφέρει τις εξής λειτουργίες, προσβάσιμες αποκλειστικά μέσω της διεπαφής του Home Assistant, χωρίς ανάγκη σύνδεσης μέσω SSH στο Raspberry Pi ή επεξεργασίας αρχείων. Πιο συγκεκριμένα, το σύστημα επιτρέπει την καταγραφή νέων υπέρυθρων εντολών από υπάρχοντα τηλεχειριστήρια και την αποθήκευσή τους για μελλοντική χρήση. Οι αποθηκευμένες εντολές μπορούν στη συνέχεια να εκτελούνται μέσω αυτοματισμών ή άλλων μηχανισμών του Home Assistant. Παράλληλα, υποστηρίζεται η διαχείριση πολλαπλών συσκευών, όπως τηλεοράσεων, κλιματιστικών και ηχοσυστημάτων, ακόμη και όταν αυτές εξυπηρετούνται από περισσότερες από μία υπέρυθρες «γέφυρες». Επιπλέον, παρέχεται η δυνατότητα δημιουργίας εικονικών πλήκτρων,

τα οποία αντιστοιχούν σε ακολουθίες πολλαπλών υπέρυθρων εντολών. Με αυτόν τον τρόπο, μπορούν να υλοποιηθούν μακροεντολές που εκτελούν διαδοχικές ενέργειες, με δυνατότητα καθορισμού χρονικής καθυστέρησης μεταξύ των επιμέρους πλήκτρων.

Η συνέχεια της εργασίας οργανώνεται ως εξής: Στο **Κεφάλαιο 2** παρουσιάζεται η βιβλιογραφική επισκόπηση των θεμελιωδών τεχνολογιών στις οποίες βασίζεται το σύστημα — υπέρυθρη επικοινωνία, πρωτόκολλο MQTT, πλατφόρμα Home Assistant — καθώς και των υφιστάμενων, ανταγωνιστικών λύσεων. Το **Κεφάλαιο 3** περιγράφει αναλυτικά την υλοποίηση, ξεκινώντας από τη συνολική αρχιτεκτονική και προχωρώντας στις λεπτομέρειες της «γέφυρας», του Home Assistant integration, του πρωτοκόλλου επικοινωνίας μέσω MQTT και της αυτοματοποιημένης δημοσίευσης εκδόσεων. Στο **Κεφάλαιο 4** αξιολογείται το σύστημα σε πραγματικά σενάρια χρήσης, με μετρήσεις καθυστέρησης, αξιοπιστίας και ασφαλείας. Τέλος, στο **Κεφάλαιο 5** συνοψίζονται τα συμπεράσματα της εργασίας και προτείνονται πιθανές μελλοντικές βελτιώσεις και επεκτάσεις.

ΚΕΦΑΛΑΙΟ 2 – Βιβλιογραφική επισκόπηση

Εισαγωγή

2.1

Στο κεφάλαιο αυτό παρουσιάζονται οι βασικές τεχνολογίες, στις οποίες βασίζεται το προτεινόμενο σύστημα της παρούσας εργασίας, ώστε ο αναγνώστης να αποκτήσει το απαραίτητο υπόβαθρο, πριν από τη λεπτομερή περιγραφή της υλοποίησης στο **Κεφάλαιο 3**.

Η δομή του κεφαλαίου ακολουθεί τη φυσική ροή των τεχνολογικών επιπέδων, ξεκινώντας από τις αφηρημένες έννοιες και καταλήγοντας στο φυσικό υλικό και το λογισμικό που χρησιμοποιείται στο σύστημα. Στην **ενότητα 2.2** εξετάζεται η έννοια του έξυπνου σπιτιού ως υποσύνολο του Διαδικτύου των Πραγμάτων, καθώς και οι βασικές αρχιτεκτονικές των σχετικών συστημάτων, ενώ παράλληλα, παρουσιάζονται οι σημαντικότερες πλατφόρμες οικιακού αυτοματισμού που είναι διαθέσιμες σήμερα. Στην **ενότητα 2.3** αναλύεται η τεχνολογία υπέρυθρου τηλεχειρισμού, με έμφαση στα πιο διαδεδομένα πρωτόκολλα (NEC, RC5 και Sony SIRC), καθώς και στις βασικές αρχές καταγραφής και αναπαραγωγής υπέρυθρων σημάτων. Στην **ενότητα 2.4** παρουσιάζεται το πρωτόκολλο MQTT, το μέσο επικοινωνίας των επιμέρους τμημάτων του συστήματος, μαζί με τα χαρακτηριστικά του, που σχετίζονται με την παρούσα υλοποίηση. Στην **ενότητα 2.5** περιγράφεται το υλικό της «γέφυρας», δηλαδή το Raspberry Pi ως υπολογιστική πλατφόρμα και το ANAVI Infrared pHAT ως επέκτασή της, για υπέρυθρη επικοινωνία.

Τέλος, στην **ενότητα 2.6** παρουσιάζεται η πλατφόρμα Home Assistant και το οικοσύστημα HACS, μέσω του οποίου διανέμεται και εγκαθίσταται το integration που αναπτύσσεται στο πλαίσιο της εργασίας. Σε κάθε υποκεφάλαιο υπάρχει μια τελική ενότητα συσχέτισης, όπου τα ευρήματα της βιβλιογραφικής ανάλυσης

συνδέονται με τις σχεδιαστικές επιλογές της εργασίας. Με αυτόν τον τρόπο διαμορφώνεται ένα συνεκτικό πλαίσιο που υποστηρίζει τη συνέχεια της μελέτης.

Έξυπνο σπίτι και Διαδίκτυο των Πραγμάτων

2.2

Εισαγωγή

2.2.A

Το παρόν υποκεφάλαιο εισάγει το θεωρητικό πλαίσιο της εργασίας, με επίκεντρο το έξυπνο σπίτι, ως ειδική περίπτωση του Διαδικτύου των Πραγμάτων. Στην **ενότητα 2.2.B** αποσαφηνίζονται βασικοί όροι, παρουσιάζονται οι κυριότερες αρχιτεκτονικές που συναντώνται σε αντίστοιχα συστήματα και αναδεικνύεται η σημασία της αρχιτεκτονικής για τη συμπεριφορά της τελικής λύσης. Στην **ενότητα 2.2.Γ** παρουσιάζονται οι σημαντικότερες πλατφόρμες οικιακού αυτοματισμού που είναι διαθέσιμες σήμερα, τόσο «ανοιχτές» όσο και εμπορικές και συγκρίνονται μεταξύ τους. Τέλος, στην **ενότητα 2.2.Δ** συνοψίζονται τα ευρήματα των προηγούμενων ενοτήτων και συνδέονται με τους σχεδιαστικούς στόχους που τέθηκαν στο **Κεφάλαιο 1**. Με βάση αυτή τη συσχέτιση αιτιολογείται η επιλογή του Home Assistant για την υλοποίηση της συγκεκριμένης πτυχιακής εργασίας.

Ορισμοί και τυπικές αρχιτεκτονικές

2.2.B

Ο όρος «έξυπνο σπίτι» (smart home) αναφέρεται σε μια κατοικία όπου οι ηλεκτρικές και ηλεκτρονικές συσκευές είναι διασυνδεδεμένες και μπορούν να ελέγχονται ή να λειτουργούν συντονισμένα μέσω ενός κοινού δικτύου. Οι δυνατότητες που προσφέρονται, περιλαμβάνουν τον απομακρυσμένο έλεγχο συσκευών, την αυτοματοποίηση λειτουργιών προσαρμόζοντας το οικιακό περιβάλλον στις ανάγκες και τις συνήθειες των χρηστών. Οι **Alaa et al. (2017)** [11], τα περισσότερα συστήματα έξυπνου σπιτιού βασίζονται σε ορισμένα κοινά χαρακτηριστικά, όπως η επικοινωνία μέσω δικτυακών πρωτοκόλλων, η συλλογή δεδομένων από αισθητήρες, η κεντρική διαχείριση των συσκευών και η παροχή εφαρμογών που επιτρέπουν την αλληλεπίδραση του χρήστη, είτε μέσω γραφικού περιβάλλοντος, είτε μέσω φωνητικών εντολών.

Το έξυπνο σπίτι αποτελεί μία από τις πιο διαδεδομένες εφαρμογές του Διαδικτύου των Πραγμάτων (Internet of Things - IoT) σε ένα οικιακό περιβάλλον. Σύμφωνα με τους **Atzori, Iera και Morabito (2010)** [2], το IoT περιγράφει ένα μοντέλο στο οποίο αντικείμενα της καθημερινής ζωής διαθέτουν δυνατότητες αναγνώρισης, συλλογής δεδομένων και επικοινωνίας, ώστε να μπορούν να αλληλεπιδρούν μέσω δικτύων και να συμμετέχουν σε αυτοματοποιημένες διαδικασίες. Ενώ το IoT έχει εφαρμογή σε πληθώρα πεδίων όπως η βιομηχανία, οι μεταφορές, η υγεία και η αγροτική παραγωγή, στο οικιακό περιβάλλον υλοποιείται κυρίως μέσω των πλατφορμών έξυπνου σπιτιού.

Από αρχιτεκτονικής άποψης, τα συστήματα IoT και τα συστήματα έξυπνου σπιτιού, μπορούν να οργανωθούν με διαφορετικούς τρόπους. Η πιο συνηθισμένη προσέγγιση είναι η αρχιτεκτονική τριών επιπέδων (three-tier architecture). Σε αυτήν, οι συσκευές βρίσκονται στο φυσικό επίπεδο, συνδέονται σε μια τοπική «πύλη» (gateway) και μέσω αυτής επικοινωνούν με υπηρεσίες cloud, όπου πραγματοποιούνται λειτουργίες όπως η αποθήκευση δεδομένων, η επεξεργασία πληροφοριών και η κεντρική διαχείριση του συστήματος. Μια εναλλακτική προσέγγιση είναι η αρχιτεκτονική edge ή fog computing, η οποία παρουσιάστηκε από τους **Bonomi et al. (2012)** [12]. Στην περίπτωση αυτή, μέρος της επεξεργασίας μεταφέρεται από το cloud σε συστήματα που βρίσκονται πιο κοντά στις ίδιες τις συσκευές, όπως η «πύλη» ή ένας τοπικός κόμβος. Με αυτόν τον τρόπο, μειώνεται η καθυστέρηση στην απόκριση, περιορίζεται η μεταφορά δεδομένων προς το διαδίκτυο και ενισχύεται η προστασία της ιδιωτικότητας. Τέλος, υπάρχει και η τοπικά προσανατολισμένη αρχιτεκτονική (local-first), όπου το μεγαλύτερο μέρος ή ακόμη και εξ ολοκλήρου, εκτελείται τοπικά. Σε τέτοια συστήματα η εξάρτηση από υπηρεσίες cloud είναι περιορισμένη ή δεν υπάρχει καθόλου, αφού ο κεντρικός κόμβος αναλαμβάνει τη διαχείριση των συσκευών και των αυτοματισμών εντός του τοπικού δικτύου. Χαρακτηριστικό παράδειγμα αυτής της προσέγγισης αποτελεί το Home Assistant [3].

Η παρούσα εργασία βασίζεται στην τρίτη αρχιτεκτονική. Η επιλογή αυτή συνδέεται άμεσα με τους στόχους που τέθηκαν στην **ενότητα 1.2**, καθώς δίνει έμφαση στην πλήρως τοπική λειτουργία του συστήματος, στη χρήση λογισμικού ανοιχτού κώδικα και στον περιορισμό της εξάρτησης από υπηρεσίες στο cloud. Οι λόγοι για τους οποίους επιλέχθηκε η συγκεκριμένη προσέγγιση, καθώς και τα πλεονεκτήματα που προσφέρει στο πλαίσιο της παρούσας εργασίας, θα παρουσιαστούν αναλυτικότερα στις επόμενες ενότητες.

Πλατφόρμες οικιακού αυτοματισμού

2.2.Γ

Σήμερα, υπάρχουν πολλές πλατφόρμες οικιακού αυτοματισμού, οι οποίες διαφέρουν τόσο ως προς τα τεχνικά χαρακτηριστικά τους όσο και ως προς τη φιλοσοφία ανάπτυξης και τη διάθεσή τους. Μια βασική διάκριση αφορά τις πλατφόρμες ανοιχτού κώδικα, οι οποίες αναπτύσσονται και υποστηρίζονται κυρίως από κοινότητες χρηστών και προγραμματιστών και τις εμπορικές λύσεις που προσφέρονται από μεγάλες εταιρείες του χώρου. Οι **Risteska Stojkoska και Trivodaliev (2017)** [16] παρέχουν μια συγκριτική μελέτη των πιο διαδεδομένων λύσεων, εξετάζοντας αρχιτεκτονικές, πρωτόκολλα και συχνά σενάρια χρήσης.

Στην κατηγορία του ανοιχτού κώδικα, οι τρεις πιο γνωστές πλατφόρμες είναι το Home Assistant [3], το openHAB [4] και το Domoticz [15]. Το Home Assistant, γραμμένο σε Python, χαρακτηρίζεται από εκτενή υποστήριξη integrations για χιλιάδες συσκευές, ενεργή κοινότητα και πλούσιο οικοσύστημα επεκτάσεων, μέσω του HACS [10]. Το openHAB, βασισμένο στο οικοσύστημα Eclipse και γραμμένο σε Java, ακολουθεί παρόμοιους στόχους με αυστηρότερη αρθρωτή αρχιτεκτονική και ισχυρή υποστήριξη βιομηχανικών πρωτοκόλλων. Το Domoticz, υλοποιημένο σε C++, είναι μία από τις παλαιότερες λύσεις της κατηγορίας, με έμφαση στην ελαφριά και τη χαμηλή κατανάλωση πόρων, αλλά με μικρότερη κοινότητα και πιο αργό

ρυθμό βελτίωσης από τα προηγούμενα. Λεπτομερής ανάλυση του Home Assistant — που αποτελεί την πλατφόρμα ασχολίας για την παρούσα εργασία — δίνεται στην **ενότητα 2.6**.

Στην κατηγορία των εμπορικών λύσεων, κυρίαρχο ρόλο έχουν πλατφόρμες που αναπτύσσονται από μεγάλα ονόματα εταιρειών. Το Samsung SmartThings βασίζεται σε συνδυασμό φυσικού hub και cloud υποδομής για τον συντονισμό και την ενοποίηση συσκευών από διαφορετικούς κατασκευαστές. Το Apple HomeKit [13] περιορίζεται κατά κανόνα σε συσκευές πιστοποιημένες από την Apple και απαιτεί συσκευή του ίδιου οικοσυστήματος (iPhone, iPad ή Apple TV / HomePod) ως κέντρο διαχείρισης. Το Google Home [14] και το Amazon Alexa [17] βασίζονται στους αντίστοιχους φωνητικούς βοηθούς, με ενσωμάτωση τρίτων κατασκευαστών να εκτελούνται μέσω συνδέσεων cloud-to-cloud.

Για το σκεπτικό της παρούσας εργασίας, οι σημαντικότεροι άξονες σύγκρισης μεταξύ των παραπάνω πλατφορμών είναι: (α) η δυνατότητα πλήρους τοπικής λειτουργίας χωρίς εξάρτηση από εξωτερικές υπηρεσίες cloud, (β) η διαθεσιμότητα του πηγαίου κώδικα και της αρχιτεκτονικής, (γ) η απουσία vendor lock-in που θα περιόριζε τις επιλογές συσκευών και (δ) η επεκτασιμότητα μέσω third-party integrations, η οποία επιτρέπει σε προγραμματιστές, ερευνητές και την ευρύτερη κοινότητα να προσθέτουν νέες υποστηριζόμενες συσκευές, χωρίς να απαιτείται συμμετοχή του βασικού κατασκευαστή της πλατφόρμας. Οι εμπορικές λύσεις αποτυγχάνουν τυπικά σε τουλάχιστον δύο από τα τέσσερα αυτά κριτήρια, ενώ μεταξύ των ανοιχτών λύσεων, το Home Assistant ικανοποιεί και τα τέσσερα.

Με βάση τα παραπάνω, η παρούσα εργασία επιλέγει το Home Assistant ως πλατφόρμα. Η αναλυτική τεκμηρίωση της επιλογής, καθώς και η περιγραφή της αρχιτεκτονικής του, παρουσιάζεται στην **ενότητα 2.6**, μαζί με το HACS.

Συσχέτιση με την παρούσα εργασία

2.2.Δ

Από όσα αναπτύχθηκαν στις προηγούμενες ενότητες του **2.2**, προκύπτουν δύο βασικές παρατηρήσεις που καθορίζουν τη σχεδιαστική προσέγγιση της παρούσας εργασίας.

Πρώτον, μεταξύ των τριών τυπικών αρχιτεκτονικών IoT που παρουσιάστηκαν, η τοπικά εστιασμένη (local-first) προσέγγιση είναι αυτή που ευθυγραμμίζεται καλύτερα με τα κριτήρια του **Κεφαλαίου 1** — δηλαδή την αποφυγή εξάρτησης από υπηρεσίες cloud, τη διασφάλιση της ιδιωτικότητας των δεδομένων του χρήστη και τη μείωση καθυστέρησης απόκρισης των συσκευών. Η αρχιτεκτονική τριών επιπέδων, παρότι κυρίαρχη στις εμπορικές λύσεις, δημιουργεί σημεία αποτυχίας εκτός του τοπικού δικτύου και επιφέρει εξάρτηση από τη μακροπρόθεσμη βιωσιμότητα τρίτων υπηρεσιών. Η edge/fog προσέγγιση αποτελεί μια ενδιάμεση λύση, αλλά εξακολουθεί τυπικά να εξαρτάται από το cloud. Στην παρούσα εργασία επιλέγεται η πλήρως τοπική εκτέλεση.

Δεύτερον, μεταξύ των πλατφορμών οικιακού αυτοματισμού που εξετάστηκαν, μόνο μια μικρή υποομάδα ανοιχτών λύσεων ικανοποιεί ταυτόχρονα και τα τέσσερα κριτήρια που τέθηκαν: τοπική εκτέλεση, διαθεσιμότητα του πηγαίου κώδικα,

απουσία vendor lock-in και υψηλό βαθμό επεκτασιμότητας. Από αυτήν την υποομάδα, το Home Assistant ξεχωρίζει για το μέγεθος της κοινότητας, τον ρυθμό εξέλιξης και τη ποικιλία επεκτάσεων του οικοσυστήματος μέσω του HACS. Η επιλογή του για την παρούσα εργασία είναι, επομένως, τεκμηριωμένη.

Με αυτό το πλαίσιο εδραιωμένο, οι επόμενες ενότητες κεφαλαίου αναλύουν τις επιμέρους τεχνολογίες πάνω στις οποίες χτίζεται το σύστημα: την υπέρυθρη επικοινωνία και τα σχετικά πρωτόκολλα (2.3), το πρωτόκολλο MQTT ως μέσο επικοινωνίας (2.4), την πλατφόρμα Raspberry Pi μαζί με το ANAVI Infrared pHAT (2.5) και τέλος την αρχιτεκτονική του ίδιου του Home Assistant και του οικοσυστήματος HACS (2.6).

Υπέρυθρη επικοινωνία και τηλεχειρισμός

2.3

Εισαγωγή

2.3.A

Η ενότητα αυτή εστιάζει στην τεχνολογία που βρίσκεται στον πυρήνα του προβλήματος που εξετάζει η εργασία: την υπέρυθρη επικοινωνία ως μέσο τηλεχειρισμού οικιακών συσκευών. Στην **ενότητα 2.3.B** παρουσιάζονται τα τρία πιο διαδεδομένα πρωτόκολλα IR — NEC, Philips RC5 και Sony SIRC — με έμφαση στη μέθοδο κωδικοποίησης, στη δομή του πλαισίου και στις βασικές διαφορές μεταξύ τους. Η **ενότητα 2.3.Γ** περιγράφει τις τεχνικές αρχές της καταγραφής και της αναπαραγωγής σημάτων IR, συμπεριλαμβανομένων των τυπικών μορφών αποθήκευσης και των απαιτήσεων ακρίβειας του χρονισμού. Τέλος, η **ενότητα 2.3.Δ** συνοψίζει τα ευρήματα της ανάλυσης και τα συνδέει με τις σχεδιαστικές επιλογές της εργασίας — ιδίως την υιοθέτηση της προσέγγισης raw timing και την επιλογή του υλικού (ANAVI Infrared pHAT) που θα παρουσιαστεί στην **ενότητα 2.5**.

Πρωτόκολλα IR (NEC, RC5, Sony SIRC)

2.3.B

Η επικοινωνία μέσω υπέρυθρων δεν ακολουθεί ένα ενιαίο διεθνές πρότυπο. Κάθε μεγάλος κατασκευαστής έχει αναπτύξει το δικό του πρωτόκολλο για την κωδικοποίηση των εντολών, με αποτέλεσμα να συνυπάρχουν στην αγορά δεκάδες ασύμβατα, μεταξύ τους, σχήματα. Παρά την ποικιλομορφία, η συντριπτική πλειοψηφία των οικιακών τηλεχειριστηρίων ακολουθεί ένα από τρία θεμελιώδη πρωτόκολλα: το NEC, το Philips RC5 και το Sony SIRC. Όλα χρησιμοποιούν διαμόρφωση σε συχνότητα 36–40 kHz, αλλά διαφέρουν στη μέθοδο κωδικοποίησης των bits, στο μήκος του πλαισίου και στη δομή των πεδίων διεύθυνσης και εντολής [18], [19].

Το πρωτόκολλο NEC, που αναπτύχθηκε από την NEC Corporation, είναι ευρέως διαδεδομένο σε τηλεοράσεις, ηχοσυστήματα και κλιματιστικά. Χρησιμοποιεί διαμόρφωση Pulse-Distance Encoding, όπου κάθε bit αναπαρίσταται από έναν παλμό σταθερής διάρκειας, ακολουθούμενο από ένα διάστημα παύσης διαφορετικής

διάρκειας — η διάρκεια της παύσης καθορίζει αν το bit είναι 0 ή 1. Ένα πλήρες πλαίσιο NEC αποτελείται από έναν παλμό εκκίνησης 9 ms, μια παύση 4.5 ms, και τη μετάδοση 32 bits ωφέλιμου φορτίου, που περιλαμβάνουν 8 bits διεύθυνσης συσκευής, 8 bits συμπληρωματικής (αντεστραμμένης) διεύθυνσης, 8 bits εντολής και 8 bits συμπληρωματικής εντολής [18]. Η αντεστραμμένη μορφή χρησιμοποιείται για έλεγχο σφαλμάτων στον δέκτη.

Το πρωτόκολλο RC5, σχεδιασμένο από την Philips, χρησιμοποιείται κυρίως σε ευρωπαϊκές μάρκες οικιακών συσκευών ψυχαγωγίας. Σε αντίθεση με το NEC, εφαρμόζει διαμόρφωση Manchester, όπου κάθε bit αναπαρίσταται από μια μετάβαση στο μέσο της χρονικής θυρίδας (time slot) — ανερχόμενη ακμή για το 1 και κατερχόμενη για το 0. Το πλαίσιο RC5 είναι αρκετά μικρότερο από αυτό του NEC: συνολικά 14 bits που περιλαμβάνουν δύο bits εκκίνησης, ένα toggle bit (το οποίο αλλάζει τιμή σε διαδοχικά πατήματα του ίδιου κουμπιού, ώστε ο δέκτης να διακρίνει επαναλαμβανόμενα πατήματα από παρατεταμένα), 5 bits διεύθυνσης συσκευής και 6 bits εντολής [18].

Το πρωτόκολλο Sony SIRC, που χρησιμοποιείται σε τηλεοράσεις, ηχοσυστήματα και βιντεοκάμερες της Sony, διατίθεται σε τρεις παραλλαγές μήκους: 12, 15 ή 20 bits ανά πλαίσιο. Χρησιμοποιεί Pulse-Width Encoding, όπου το μήκος του ίδιου του παλμού (και όχι του διαστήματος που ακολουθεί) καθορίζει την τιμή του bit. Επιπλέον, μια εντολή SIRC μεταδίδεται συνήθως τρεις φορές διαδοχικά για την ενίσχυση της αξιοπιστίας της λήψης σε συνθήκες θορύβου [18].

Πέρα από τα παραπάνω, στην πράξη συναντώνται και άλλα πρωτόκολλα, όπως το Panasonic Kaseikyo, το Samsung 32, το RC6 (διάδοχος του RC5), και άλλα ειδικών χρήσεων. Μια ολοκληρωμένη συλλογή τεχνικών προδιαγραφών δεκάδων πρωτοκόλλων, παρουσιάζεται στο SB-Projects [18], η οποία αποτελεί de facto πρακτική αναφορά για την κοινότητα του υπέρυθρου τηλεχειρισμού. Όπως θα αναλυθεί στη συνέχεια, η ικανότητα του προτεινόμενου συστήματος να καταγράφει και να αναπαράγει ένα σήμα IR δεν εξαρτάται από τη γνώση του εκάστοτε πρωτοκόλλου: η «γέφυρα» καταγράφει το raw χρονικό προφίλ των παλμών και το αναπαράγει αυτούσιο, καθιστώντας το σύστημα συμβατό με όλα τα παραπάνω πρωτόκολλα και τις παραλλαγές τους.

Καταγραφή και αναπαραγωγή σημάτων

2.3.Γ

Η εκπομπή ενός υπέρυθρου σήματος από μια «γέφυρα» απαιτεί δύο στάδια: την καταγραφή του σήματος από ένα φυσικό τηλεχειριστήριο και την πιστή αναπαραγωγή του προς τη συσκευή-στόχο. Το πλεονέκτημα μιας τέτοιας προσέγγισης, έναντι της προεγκατεστημένης γνώσης συγκεκριμένων πρωτοκόλλων, είναι ότι το σύστημα μπορεί να υποστηρίξει οποιαδήποτε συσκευή, ανεξαρτήτως κατασκευαστή ή πρωτοκόλλου, αρκεί ο χρήστης να διαθέτει το αρχικό τηλεχειριστήριο.

Καταγραφή. Στο στάδιο της καταγραφής του σήματος από τη «γέφυρα», ένας υπέρυθρος δέκτης (όπως ο Vishay TSOP [19]) λαμβάνει τον υπέρυθρο παλμό που εκπέμπει το τηλεχειριστήριο. Ο δέκτης ενσωματώνει αυτόματα την αποδιαμόρφωση της φέρουσας συχνότητας και παράγει στην ψηφιακή του έξοδο έναν παλμό LOW

όταν ανιχνεύει φέρον σήμα και HIGH όταν δεν ανιχνεύει. Συνεπώς, αυτό που λαμβάνει η μονάδα ελέγχου είναι η χρονική σειρά των ON και OFF διαστημάτων του σήματος, χωρίς να χρειάζεται δειγματοληψία στην αρχική συχνότητα της φέρουσας συχνότητας. Το ζητούμενο της καταγραφής περιορίζεται στη μέτρηση των διαρκειών αυτών των διαστημάτων με ακρίβεια τάξης μικροδευτερολέπτων (μs).

Μορφές αποθήκευσης. Το πιο γενικό σχήμα είναι ο πίνακας ακατέργαστων χρονισμών (raw timing array): μια ακολουθία ακεραίων που αναπαριστούν διαδοχικά τα ON και OFF διαστήματα σε μs , μαζί με την τιμή της φέρουσας συχνότητας. Αυτή η μορφή είναι αδιάφορη από πλευράς πρωτοκόλλου και υποστηρίζει οποιοδήποτε σήμα. Εναλλακτικά, εφόσον το πρωτόκολλο είναι γνωστό, η εντολή μπορεί να αποθηκευτεί σε πιο συμπαγή μορφή — π.χ. ως 32-bit κωδικός για το NEC ή ως 14-bit για το RC5 — εξοικονομώντας χώρο και επιτρέποντας επεξεργασία αργότερα. Στην πράξη, ευρέως διαδεδομένες είναι οι μορφές Pronto HEX (από τα Philips Pronto και Logitech Harmony) και LIRC [20], το οποίο αναπτύχθηκε στο πλαίσιο του ομώνυμου project σε Linux. Στην παρούσα εργασία επιλέγεται η μορφή raw timing, για λόγους συμβατότητας με μη τυποποιημένες συσκευές, όπως ορισμένα κλιματιστικά που μεταδίδουν εκτενή πλαίσια εκτός των βασικών πρωτοκόλλων.

Αναπαραγωγή. Στο στάδιο της αναπαραγωγής, η μονάδα ελέγχου διαβάζει τον αποθηκευμένο πίνακα χρονισμών και μέσω ενός υπέρυθρου πομπού (συνήθως IR LED συμπεριλαμβανομένου τρανζίστορ ενίσχυσης) σύμφωνα με τις διάρκειες. Στις περιόδους ON, το LED τροφοδοτείται με τετραγωνικό κύμα στη φέρουσα συχνότητα του πρωτοκόλλου (38 kHz για το NEC και το RC5, 40 kHz για το Sony SIRC). Στις περιόδους OFF το LED παραμένει σβηστό. Η ακρίβεια του χρονισμού είναι κρίσιμη: αποκλίσεις πέραν του 10% από τις αναμενόμενες διάρκειες οδηγούν συνήθως σε απόρριψη του σήματος από τον δέκτη της συσκευής. Για τον λόγο αυτό, η αναπαραγωγή εκτελείται είτε από εξειδικευμένο μικροελεγκτή είτε από διαδικασία υψηλής προτεραιότητας στο λειτουργικό σύστημα του υπολογιστή που ελέγχει τον πομπό.

Συσχέτιση με την παρούσα εργασία

2.3.Δ

Από όσα παρουσιάστηκαν στις προηγούμενες ενότητες του **2.3**, προκύπτουν δύο σχεδιαστικές αποφάσεις που υιοθετούνται στην παρούσα εργασία.

Πρώτον, παρότι αναπτύχθηκαν αναλυτικά τα τρία πιο διαδεδομένα πρωτόκολλα IR (NEC, RC5, Sony SIRC), το προτεινόμενο σύστημα δεν ενσωματώνει ειδική γνώση κανενός από αυτά. Όπως εξηγήθηκε στην **ενότητα 2.3.Γ**, η καταγραφή σε μορφή raw timing και η αναπαραγωγή με βάση τον αρχικό χρονισμό είναι μια προσέγγιση αδιάφορη από πλευράς πρωτοκόλλου, που υποστηρίζει ταυτόχρονα όλα τα γνωστά πρωτόκολλα, αλλά και οποιαδήποτε παραλλαγή ή ειδική υλοποίηση από κατασκευαστές κλιματιστικών και λοιπών συσκευών. Η προσέγγιση αυτή απλοποιεί την υλοποίηση του συστήματος και εξασφαλίζει υψηλό βαθμό συμβατότητας με συσκευές διαφορετικών κατασκευαστών και κατηγοριών. Το μοναδικό της μειονέκτημα είναι η ελάχιστα αυξημένη απαίτηση σε χώρο αποθήκευσης για κάθε εντολή, κόστος μικρό σε σχέση με τα οφέλη που προσφέρει.

Δεύτερον, η ακρίβεια του χρονισμού που απαιτείται κατά την αναπαραγωγή (της τάξης των μικροδευτερολέπτων) θέτει συγκεκριμένες απαιτήσεις στην επιλογή του υλικού. Στην παρούσα εργασία επιλέγεται το ANAVI Infrared pHAT, μια εξειδικευμένη πλακέτα επέκτασης για Raspberry Pi, που ενσωματώνει υπέρυθρο δέκτη και πομπό, με υλικό ικανό να ανταποκριθεί στις παραπάνω χρονικές απαιτήσεις. Η αναλυτική παρουσίαση του υλικού γίνεται στην **ενότητα 2.5**.

Με αυτές τις δύο επιλογές εδραιωμένες, μένει να εξεταστεί ο τρόπος με τον οποίο η «γέφυρα» — που στο φυσικό επίπεδο «μιλάει» IR — μπορεί να ενσωματωθεί σε ένα δίκτυο. Τον ρόλο αυτό αναλαμβάνει το πρωτόκολλο MQTT, το οποίο εξετάζεται στην επόμενη ενότητα.

Πρωτόκολλο MQTT

2.4

Εισαγωγή

2.4.A

Η ενότητα αυτή παρουσιάζει το πρωτόκολλο MQTT, το οποίο αποτελεί μέσο επικοινωνίας μεταξύ της «γέφυρας» και του Home Assistant στο προτεινόμενο σύστημα. Στην **ενότητα 2.4.B** αναλύεται το θεμελιώδες μοντέλο publish/subscribe, στο οποίο βασίζεται το MQTT και η δομή των topics και τα πλεονεκτήματα του πρωτοκόλλου, σε σχέση με κλασικά μοντέλα client/server. Η **ενότητα 2.4.Γ** εξετάζει τρία χαρακτηριστικά που έχουν άμεση σχέση με την αξιοπιστία του συστήματος: τα τρία επίπεδα ποιότητας υπηρεσίας (QoS), τα διατηρούμενα μηνύματα (retained messages) και τη διατήρηση καταστάσεως (persistence). Τέλος, η **ενότητα 2.4.Δ** συνοψίζει τα παραπάνω και τα συνδέει με τις σχεδιαστικές επιλογές της εργασίας, αιτιολογώντας τόσο τη χρήση του MQTT ως πρωτόκολλο μεταφοράς όσο και τις συγκεκριμένες παραμέτρους που υιοθετούνται για κάθε κατηγορία μηνύματος.

Μοντέλο publish/subscribe

2.4.B

Το πρότυπο publish/subscribe (pub/sub) είναι μια ευρέως διαδεδομένη αρχιτεκτονική επικοινωνίας λογισμικού, στην οποία ο αποστολέας ενός μηνύματος (publisher) δε στέλνει το μήνυμα απευθείας σε συγκεκριμένους παραλήπτες, αλλά το «δημοσιεύει» σε μια κατηγορία που ονομάζεται topic. Οι αποδέκτες, με τη σειρά τους, «εγγράφονται» (subscribe) σε ένα ή περισσότερα topics και λαμβάνουν αυτόματα κάθε νέο μήνυμα που δημοσιεύεται σε αυτά. Το βασικό χαρακτηριστικό του μοντέλου είναι ότι publisher και subscriber δεν γνωρίζουν την ύπαρξη ο ένας του άλλου: η μεταξύ τους επικοινωνία γίνεται μέσω ενός ενδιάμεσου εξυπηρετητή που ονομάζεται broker και αναλαμβάνει τη δρομολόγηση των μηνυμάτων από τους publishers προς τους κατάλληλους subscribers.

Το MQTT [21] (Message Queuing Telemetry Transport) είναι ένα από τα πιο διαδεδομένα πρωτόκολλα που υλοποιούν αυτό το μοντέλο. Σχεδιάστηκε αρχικά από

την IBM στα τέλη της δεκαετίας του 1990 για την παρακολούθηση πετρελαϊκών αγωγών, μέσω δορυφορικών συνδέσεων χαμηλού εύρους ζώνης και αργότερα τυποποιήθηκε από τον οργανισμό OASIS (Organization for the Advancement of Structured Information Standards). Σήμερα αποτελεί de facto standard για την επικοινωνία συσκευών IoT, με διαθέσιμες υλοποιήσεις σε όλες τις βασικές πλατφόρμες και γλώσσες προγραμματισμού.

Στο MQTT, τα topics οργανώνονται σε ιεραρχική δομή κατά το πρότυπο των διαδρομών συστήματος αρχείων (filesystem paths), με τη χρήση της καθέτου (/) ως διαχωριστικό επιπέδων. Για παράδειγμα, ένα topic όπως το `home/livingroom/tv/power`, μπορεί να εκφράζει την κατάσταση τροφοδοσίας της τηλεόρασης του σαλονιού. Η ιεραρχική δομή επιτρέπει στους subscribers να εγγράφονται σε ολόκληρες ομάδες topics μέσω χαρακτήρων μπαλαντέρ (wildcards): ο χαρακτήρας '+' αντικαθιστά ένα μεμονωμένο επίπεδο (π.χ. το `home/+/tv/power` ταιριάζει με κάθε δωμάτιο), ενώ ο χαρακτήρας '#' αντικαθιστά ένα ολόκληρο το υπόλοιπο της διαδρομής (π.χ. το `home/livingroom/#` ταιριάζει με κάθε topic κάτω από αυτό το πρόθεμα).

Το μοντέλο pub/sub προσφέρει αρκετά πλεονεκτήματα έναντι των κλασικών μοντέλων επικοινωνίας client/server. Πρώτον, οι παραγωγοί και οι καταναλωτές (producers/consumers) δεδομένων είναι αυτόνομοι: μπορούν να προστίθενται, να αφαιρούνται ή να αναβαθμίζονται ανεξάρτητα ο ένας από τον άλλον, χωρίς να απαιτείται επαναδιαμόρφωση του συστήματος. Δεύτερον, ο ίδιος publisher μπορεί να εξυπηρετήσει πολλούς subscribers με μία μόνο δημοσίευση, αφού ο broker αναλαμβάνει την αποστολή του μηνύματος προς όλους τους ενδιαφερόμενους. Τρίτον, το πρωτόκολλο είναι ελαφρύ — τα πλαίσιά του μπορούν να είναι έως και 2 bytes σε μέγεθος, για μηνύματα ελάχιστου ωφέλιμου φορτίου — κατάλληλο για συσκευές με περιορισμένους πόρους και συνδέσεις χαμηλού εύρους ζώνης. Όλα τα παραπάνω εξηγούν γιατί το MQTT έχει επικρατήσει στην επικοινωνία μεταξύ συσκευών IoT, έναντι βαρύτερων πρωτοκόλλων, όπως το HTTP.

QoS, retained messages και persistence

2.4.Γ

Το MQTT προσφέρει τρία επίπεδα ποιότητας υπηρεσίας (Quality of Service, QoS) που εγγυώνται την παράδοση κάθε μηνύματος, τόσο μεταξύ publisher και broker, όσο και μεταξύ broker και subscriber [21]. Στο QoS 0 (at most once), το μήνυμα αποστέλλεται μία φορά χωρίς επιβεβαίωση· είναι το πιο γρήγορο και ελαφρύ επίπεδο, αλλά δεν εγγυάται την παράδοση: αν η σύνδεση διακοπεί τη στιγμή της αποστολής, το μήνυμα χάνεται. Στο QoS 1 (at least once), ο αποστολέας περιμένει επιβεβαίωση (ACK) από τον broker και αν δεν την λάβει εντός χρόνου, αναμεταδίδει το μήνυμα· εξασφαλίζει ότι το μήνυμα θα παραδοθεί τουλάχιστον μία φορά, αλλά με πιθανή διπλή λήψη. Στο QoS 2 (exactly once), η πλήρης χειραψία τεσσάρων βημάτων (PUBLISH, PUBREC, PUBREL, PUBCOMP) εξασφαλίζει ότι το μήνυμα παραδίδεται ακριβώς μία φορά, χωρίς απώλειες ή διπλοεγγραφές· είναι όμως το πιο βαρύ επίπεδο σε επιβάρυνση δικτύου. Η επιλογή του κατάλληλου επιπέδου είναι σχεδιαστική απόφαση που εξαρτάται από την κρισιμότητα του μηνύματος: μία μέτρηση θερμοκρασίας που μεταδίδεται ανά δευτερόλεπτο, μπορεί κάλλιστα να

χρησιμοποιήσει QoS 0, ενώ μια εντολή απενεργοποίησης συσκευής που θεωρείται σημαντική, δικαιολογεί QoS 2.

Retained messages. Ένα ιδιαίτερο χαρακτηριστικό του MQTT είναι η δυνατότητα να χαρακτηριστεί ένα μήνυμα ως «διατηρούμενο» (retained message): ο broker κρατά την τελευταία τέτοια δημοσίευση ανά topic και κάθε νέος subscriber που εγγράφεται σε εκείνο το topic τη λαμβάνει αμέσως, χωρίς να περιμένει την επόμενη δημοσίευση. Το χαρακτηριστικό αυτό είναι χρήσιμο για topics που αναπαριστούν κατάσταση (state) και όχι γεγονότα (events) — για παράδειγμα, η τρέχουσα κατάσταση μιας συσκευής (on/off). Όταν μια νέα οντότητα ξεκινά και θέλει να μάθει την κατάσταση, ο broker της απαντά αμέσως με την τελευταία γνωστή τιμή χωρίς να απαιτείται περιοδικός έλεγχος (polling).

Persistence και session state. Το MQTT υποστηρίζει δύο διαφορετικές μορφές διατήρησης κατάστασης. Πρώτον, σε επίπεδο σύνδεσης, η παράμετρος (flag) `clean_session` (ή Clean Start στην 5^η έκδοση) ορίζει αν ο broker θα διατηρεί τις εγγραφές και τα μηνύματα που δεν παραδόθηκαν στον client, όταν η σύνδεση διακοπεί και αργότερα αποκατασταθεί. Δεύτερον, σε επίπεδο μηνυμάτων, οι σύγχρονοι brokers (όπως οι Mosquitto, EMQX, HiveMQ) αποθηκεύουν μόνιμα τα retained messages και τις ουρές QoS≥1, ώστε να διατηρούνται ακόμα και μετά από επανεκκίνηση του broker. Σημαντικό για την αξιοπιστία του συστήματος είναι επίσης το Last Will and Testament (LWT) — μια ειδική δημοσίευση, όπου ο broker αναλαμβάνει να εκπέμψει αυτομάτως, όταν ένας client αποσυνδεθεί ξαφνικά, ώστε οι υπόλοιπες οντότητες να ενημερωθούν ότι η συσκευή είναι offline.

Συσχέτιση με την παρούσα εργασία

2.4.Δ

Από όσα παρουσιάστηκαν στις προηγούμενες ενότητες, προκύπτει ότι το MQTT προσφέρει ένα ιδανικό μέσο επικοινωνίας για την αρχιτεκτονική που υιοθετείται. Πέρα από την καταλληλότητά του ως πρωτόκολλο IoT, τρία είναι τα χαρακτηριστικά του αξιοποιούνται στο σύστημα.

Πρώτον, η ανεξαρτησία μεταξύ publisher και subscriber μέσω broker επιτρέπει στο Home Assistant και στη «γέφυρα» να εξελίσσονται ανεξάρτητα. Η «γέφυρα» δεν χρειάζεται να γνωρίζει τίποτα για την ύπαρξη ή τη διεύθυνση του Home Assistant — το ίδιο ισχύει και αντίστροφα. Και τα δύο μέρη γνωρίζουν μόνο τη διεύθυνση του broker και τη δομή των topics, η οποία αναλύεται στην **ενότητα 3.5**. Έτσι, η αντικατάσταση ή η αναβάθμιση οποιουδήποτε, δεν επηρεάζει το άλλο.

Δεύτερον, τα διατηρούμενα μηνύματα (retained messages) αξιοποιούνται για την αναπαράσταση της κατάστασης της «γέφυρας» και των συσκευών που διαχειρίζεται. Όταν το Home Assistant κάνει επανεκκίνηση — π.χ. μετά από ενημέρωση λογισμικού ή διακοπή ρεύματος — μπορεί να ανακτήσει αμέσως την τρέχουσα κατάσταση όλων των συσκευών, μέσω των retained δημοσιεύσεων, χωρίς να χρειάζεται να «ρωτήσει» ενεργά τη «γέφυρα».

Τρίτον, η επιλογή του κατάλληλου επιπέδου QoS εξαρτάται από τη φύση του κάθε μηνύματος. Στις εντολές αναπαραγωγής σημάτων IR, η επιλογή ενός QoS μεγαλύτερου από 0 είναι περιορισμένη, καθώς το συγκεκριμένο επίπεδο IR δεν

παρέχει επιβεβαίωση από τη συσκευή: το MQTT QoS εξασφαλίζει μόνο την παράδοση της εντολής στη «γέφυρα», όχι την επιτυχή εκπομπή και λήψη του IR σήματος από τη συσκευή. Στα μηνύματα κατάστασης, αντιθέτως, η επιλογή μεγαλύτερου QoS με `retained=true` εξασφαλίζει αξιόπιστη και συγχρονισμένη παρακολούθηση από όλους τους ενδιαφερόμενους subscribers.

Έχοντας ολοκληρώσει την παρουσίαση την επικοινωνίας, στη συνέχεια, θα στραφούμε στο υλικό της «γέφυρας»: στο Raspberry Pi και στην επέκταση ANAVI Infrared pHAT.

Πλατφόρμα Raspberry Pi και ANAVI Infrared pHAT 2.5

Εισαγωγή

2.5.A

Η ενότητα αυτή παρουσιάζει το υλικό πάνω στο οποίο υλοποιείται η «γέφυρα» του συστήματος. Στην **ενότητα 2.5.B** εξετάζεται το Raspberry Pi ως πλατφόρμα — η ιστορία και η αρχιτεκτονική του, τα κύρια τεχνικά χαρακτηριστικά και οι λόγοι για τους οποίους αποτελεί δημοφιλή επιλογή για εφαρμογές edge computing και IoT. Η **ενότητα 2.5.Γ** εστιάζει στο ANAVI Infrared pHAT, μία εξειδικευμένη επέκταση για Raspberry Pi, με ενσωματωμένο υπέρυθρο πομπό και δέκτη, καθώς και στους λόγους καταλληλότητας στη συγκεκριμένη εφαρμογή. Τέλος, η **ενότητα 2.5.Δ** συνδέει την επιλογή υλικού με τα κριτήρια που τέθηκαν στο **Κεφάλαιο 1**, αιτιολογώντας τη συμβατότητά της με τη φιλοσοφία της εργασίας.

Raspberry Pi

2.5.B

Το Raspberry Pi είναι ένας μικρός υπολογιστής χαμηλού κόστους, μονής πλακέτας (Single Board Computer, SBC) που σχεδιάστηκε αρχικά από το βρετανικό μη κερδοσκοπικό Raspberry Pi Foundation [22], με στόχο την προώθηση της εκπαίδευσης στην πληροφορική και την ηλεκτρονική. Η πρώτη γενιά παρουσιάστηκε το 2012 και από τότε έχουν κυκλοφορήσει αρκετές διαδοχικές γενιές με ολοένα μεγαλύτερες δυνατότητες, καθώς και μικρότερες παραλλαγές όπως η σειρά Raspberry Pi Zero. Η ευρεία διάδοση και το προσιτή τιμή (από περίπου 15 ευρώ για το Zero μέχρι περίπου 80 ευρώ για το Pi 5) έχουν κάνει την πλατφόρμα δημοφιλή επιλογή για πειραματισμό, εκπαιδευτικά projects και — όλο και περισσότερο — για εφαρμογές edge computing και IoT.

Από αρχιτεκτονική σκοπιά, το Raspberry Pi βασίζεται σε ολοκληρωμένο σύστημα (System-on-Chip, SoC) της Broadcom, με επεξεργαστή ARM (Cortex-A στις νεότερες γενιές), ενσωματωμένη GPU και μνήμη RAM που κυμαίνεται από 256 MB στις παλαιότερες γενιές, έως 16 GB στις σύγχρονες. Διαθέτει πλήθος διεπαφών για την επικοινωνία με περιφερειακό υλικό: θύρες USB, HDMI, Ethernet, Wi-Fi και Bluetooth (σε νεότερες γενιές), καθώς και θύρες για κάμερα και οθόνη. Το πιο κρίσιμο χαρακτηριστικό για εφαρμογές embedded και IoT είναι όμως οι 40 ακροδέκτες γενικού σκοπού (General Purpose Input/Output, GPIO), οι οποίοι

επιτρέπουν την απευθείας σύνδεση με αισθητήρες, διακόπτες, LED και άλλα ηλεκτρονικά εξαρτήματα, μέσω πρωτοκόλλων όπως I2C, SPI, UART, αλλά και μέσω απλών ψηφιακών σημάτων ON/OFF.

Σε επίπεδο λογισμικού, η πλατφόρμα υποστηρίζει επίσημα το Raspberry Pi OS (πρώην Raspbian), μια διανομή Linux βασισμένη στο Debian, αλλά μπορεί να εκτελέσει και άλλα λειτουργικά συστήματα όπως Ubuntu, DietPi, καθώς και εξειδικευμένα συστήματα όπως το Home Assistant OS. Η ωριμότητα του Λειτουργικού Linux καθιστά διαθέσιμες χιλιάδες βιβλιοθήκες και εργαλεία ανάπτυξης λογισμικού, ιδιαίτερα σε γλώσσες όπως η Python, η C/C++ και άλλες.

Για τις απαιτήσεις της παρούσας εργασίας, το Raspberry Pi προσφέρει μια ιδανική ισορροπία: επαρκή υπολογιστική ισχύ για να εκτελέσει τη «γέφυρα» και έναν MQTT client, GPIO ακροδέκτες για τη σύνδεση με την επέκταση και συνεπώς με τον υπέρυθρο δέκτη και πομπό, ποικιλία βιβλιοθηκών Python για τον προγραμματισμό του υλικού και πολύ μικρή κατανάλωση ενέργειας (συνήθως κάτω από 5 W), που επιτρέπει 24ωρη λειτουργία στο σπίτι, χωρίς επιβάρυνση στον λογαριασμό ρεύματος.

ANAVI Infrared pHAT

2.5.Γ

Το ANAVI Infrared pHAT [23] είναι μία εξειδικευμένη επέκταση (add-on board) για Raspberry Pi, σχεδιασμένη ειδικά για εφαρμογές υπέρυθρης επικοινωνίας. Αναπτύχθηκε από την ANAVI Technology, μια μικρή βουλγαρική εταιρεία ανοιχτού υλικού (open-source hardware) και αποτελεί έναν από τους πιο ολοκληρωμένους συνδυασμούς υπέρυθρου πομπού και δέκτη για την πλατφόρμα Raspberry Pi στο εμπόριο.

Φυσικά χαρακτηριστικά. Όπως δηλώνει και το όνομά του, το προϊόν ακολουθεί το πρότυπο pHAT (Pi Hardware Attached on Top), που είναι μια πιο συμπαγής παραλλαγή του HAT προτύπου της Raspberry Pi Foundation, με διαστάσεις προσαρμοσμένες στις μικρότερες πλακέτες της σειράς Pi Zero. Παρόλα αυτά, το pHAT παραμένει συμβατό με όλα τα μοντέλα Raspberry Pi που διαθέτουν 40 ακροδέκτες GPIO, συμπεριλαμβανομένων των Pi 3, Pi 4 και Pi 5 και η σύνδεσή του γίνεται χωρίς συγκολλήσεις ή καλωδιώσεις.

Ηλεκτρονικά εξαρτήματα. Στην πλακέτα του ANAVI Infrared pHAT ενσωματώνονται τρία βασικά υπέρυθρα στοιχεία: (α) ένας υπέρυθρος δέκτης τύπου TSOP της Vishay [19], ο οποίος λαμβάνει και αποκωδικοποιεί σήματα IR από οποιοδήποτε τηλεχειριστήριο, σε συχνότητα γύρω των 38 kHz, (β) δύο υπέρυθρα LED μεγάλης γωνίας εκπομπής για την αναπαραγωγή σημάτων και (γ) ένα ηλεκτρονικό κύκλωμα ενίσχυσης μέσω τρανζίστορ, για τη σωστή λειτουργία των LED, με επαρκές ρεύμα. Συμπληρωματικά, η πλακέτα περιλαμβάνει υποδοχές I2C για τη σύνδεση εξωτερικών αισθητήρων (θερμοκρασίας, υγρασίας, φωτεινότητας, ατμοσφαιρικής πίεσης), οι οποίοι ωστόσο δεν είναι απαραίτητοι στην παρούσα φάση της εφαρμογής.

Ανοιχτός σχεδιασμός και τεκμηρίωση. Ένα χαρακτηριστικό που διακρίνει το ANAVI Infrared pHAT από αντίστοιχα εμπορικά προϊόντα είναι ο πλήρως ανοιχτός

χαρακτήρας του σχεδιασμού. Τα σχηματικά διαγράμματα της πλακέτας, τα αρχεία PCB σε μορφή KiCad και οι οδηγίες εγκατάστασης και χρήσης διατίθενται δημόσια από τον κατασκευαστή. Αυτό επιτρέπει σε ερευνητές, εκπαιδευτικούς και ερασιτέχνες, όχι μόνο να το χρησιμοποιούν, αλλά να μπορούν να επεκτείνουν τον σχεδιασμό του.

Καταλληλότητα για την παρούσα εργασία. Το ANAVI Infrared pHAT επιλέχθηκε ως υλικό της «γέφυρας» για τέσσερις λόγους. Πρώτον, παρέχει σε ενιαία πλακέτα τις λειτουργίες λήψης και εκπομπής σημάτων, χωρίς ανάγκη πρόσθετης συναρμολόγησης από διάφορα άλλα εξαρτήματα. Δεύτερον, είναι plug-and-play με το Raspberry Pi μέσω του GPIO header, χωρίς απαίτηση συγκόλλησης ή ειδικών καλωδιώσεων. Τρίτον, ο ανοιχτός χαρακτήρας του ευθυγραμμίζεται με τη γενικότερη φιλοσοφία της εργασίας. Τέταρτον, το χαμηλό κόστος (περίπου 20 ευρώ) διατηρεί τη συνολική επένδυση σε υλικό σε πολύ προστά επίπεδα — κάτω από 100 ευρώ μαζί με το Raspberry Pi.

Συσχέτιση με την παρούσα εργασία

2.5.Δ

Ο συνδυασμός Raspberry Pi και ANAVI Infrared pHAT που παρουσιάστηκε στις προηγούμενες ενότητες, αποτελεί τη βασική προϋπόθεση για την υλοποίηση της «γέφυρας» στην παρούσα εργασία. Η επιλογή αυτή, ικανοποιεί τέσσερα κριτήρια που τέθηκαν εξ αρχής στο **Κεφάλαιο 1**.

Πρώτον, η αρχιτεκτονική είναι πλήρως τοπική. Όλη η επεξεργασία — από τη λήψη υπέρυθρων σημάτων, μέχρι την επικοινωνία με το Home Assistant — εκτελείται μέσα στο σπίτι του χρήστη, χωρίς εξάρτηση από οποιαδήποτε εξωτερική υπηρεσία cloud. Αυτό ικανοποιεί τόσο την ανάγκη ιδιωτικότητας όσο και την ανάγκη ανθεκτικότητας απέναντι σε προβλήματα διαδικτυακής συνδεσιμότητας ή στην απόσυρση τρίτων υπηρεσιών.

Δεύτερον, τόσο η πλατφόρμα Raspberry Pi όσο και το ANAVI Infrared pHAT είναι ανοιχτής φύσης: το λογισμικό του Raspberry Pi OS βασίζεται σε Linux και διανέμεται δωρεάν, ενώ το hardware της ANAVI κυκλοφορεί υπό άδεια ανοιχτού υλικού με δημόσια KiCad schematics. Ο ανοιχτού υλικού και λογισμικού αυτός χαρακτήρας, είναι πρακτικά αδύνατος σε κλειστές εμπορικές πλατφόρμες.

Τρίτον, το συνολικό κόστος υλικού παραμένει σημαντικά χαμηλό — κάτω από 100 ευρώ για ένα ολοκληρωμένο σετ. Αυτό καθιστά το σύστημα προστά, όχι μόνο σε ερευνητές, αλλά και σε όσους θέλουν να το αναπαράγουν για το δικό τους σπίτι.

Τέταρτον, η πλατφόρμα είναι επεκτάσιμη: το Raspberry Pi παραμένει ένας πλήρης υπολογιστής γενικού σκοπού, οπότε η ίδια συσκευή μπορεί στο μέλλον να αναλάβει και άλλες λειτουργίες (επιπλέον αισθητήρες), πέρα από τον υπέρυθρο τηλεχειρισμό.

Με τις επιλογές υλικού και πλατφόρμας πλέον δομημένες, μένει μια τελευταία τεχνολογική συνιστώσα προς εξέταση: το ίδιο το Home Assistant και το οικοσύστημα HACS, μέσω του οποίου διανέμεται η ενσωμάτωση της εργασίας, δηλαδή το αντικείμενο της επόμενης ενότητας.

Η ενότητα αυτή ολοκληρώνει τη βιβλιογραφική επισκόπηση, εστιάζοντας στο Home Assistant — την πλατφόρμα-στόχο της παρούσας εργασίας — και στο οικοσύστημα επεκτάσεων HACS, μέσω του οποίου διανέμεται το integration. Στην **ενότητα 2.6.B** αναλύεται η αρχιτεκτονική του Home Assistant, με έμφαση στις θεμελιώδεις έννοιες των οντοτήτων, των καταστάσεων και του event bus, καθώς και οι τρόποι εγκατάστασης που υποστηρίζονται. Η **ενότητα 2.6.Γ** παρουσιάζει τον μηχανισμό των custom integrations του Home Assistant και τη ροή διανομής τους μέσω του HACS, συμπεριλαμβανομένων των αυτοματοποιημένων ελέγχων ποιότητας που απαιτούνται για την ένταξη στο default store. Τέλος, η **ενότητα 2.6.Δ** συσχετίζει τα παραπάνω με τις σχεδιαστικές επιλογές της παρούσας εργασίας, εξηγώντας πώς το συγκεκριμένο οικοσύστημα αξιοποιείται από το integration “IR Remote (ANAVI IR pHAT)”.

Το Home Assistant [3] είναι μια πλατφόρμα οικιακού αυτοματισμού ανοιχτού κώδικα, γραμμένη σε Python, η οποία λειτουργεί ως κεντρικός κόμβος ενοποίησης και ελέγχου για δεκάδες κατηγορίες συσκευών έξυπνου σπιτιού. Σχεδιάστηκε το 2013 από τον Paulus Schoutsen και έχει εξελιχθεί σε μία από τις πιο ενεργές πλατφόρμες ανοιχτού κώδικα στην κατηγορία της, με χιλιάδες συντελεστές παγκοσμίως και κυκλοφορία νέας έκδοσης σχεδόν κάθε μήνα.

Πυρήνας. Στην καρδιά του Home Assistant βρίσκεται το Home Assistant Core, μια ασύγχρονη εφαρμογή Python που εκτελεί τη λογική του συστήματος. Ο πυρήνας οργανώνεται γύρω από δύο θεμελιώδεις έννοιες: την οντότητα (entity) και την κατάσταση (state). Κάθε συσκευή του σπιτιού αναπαρίσταται από μία ή περισσότερες οντότητες — για παράδειγμα, ένας θερμοστάτης μπορεί να έχει χωριστές οντότητες για τη μέτρηση θερμοκρασίας, για τη ρύθμιση επιθυμητού στόχου και για την κατάσταση τροφοδοσίας. Κάθε οντότητα έχει, σε κάθε χρονική στιγμή, μία κατάσταση και κάθε αλλαγή κατάστασης δημιουργεί ένα γεγονός (event) στο εσωτερικό event bus του συστήματος.

Integrations. Η σύνδεση του Home Assistant με τις εκάστοτε συσκευές γίνεται μέσω ενσωματώσεων (integrations) — υποσυστημάτων σε Python που αναλαμβάνουν την επικοινωνία με συγκεκριμένο πρωτόκολλο ή υπηρεσία και δημιουργούν τις αντίστοιχες οντότητες. Το επίσημο αποθετήριο διαθέτει πάνω από 2.500 ενσωματώσεις, καλύπτοντας τα κυριότερα IoT πρωτόκολλα (Zigbee, Z-Wave, MQTT, HTTP REST, Modbus), αλλά και τρίτες υπηρεσίες (Spotify, Plex, Google Calendar κ.ά.). Κάθε integration μπορεί να ορίζει διαφορετικούς τύπους οντοτήτων: light, switch, sensor, climate, button, number, text κ.λπ.

Front-end και αλληλεπίδραση. Το frontend του Home Assistant, γνωστό ως Lovelace, βασίζεται σε σύγχρονες τεχνολογίες Web και προσφέρει πλήρως προσαρμόσιμα dashboards. Οι χρήστες μπορούν να ρυθμίσουν την οθόνη ελέγχου τους ελεύθερα, να οργανώσουν αυτοματισμούς και να επεκτείνουν τη λειτουργικότητα με addons και custom components.

Αρχιτεκτονικά μοντέλα εγκατάστασης. Το Home Assistant διατίθεται σε τέσσερις εκδόσεις: (α) Home Assistant OS — ολοκληρωμένο λειτουργικό σύστημα appliance-style που εγκαθίσταται σε Raspberry Pi ή σε εικονική μηχανή (VM), (β) Home Assistant Container — Docker container για ευέλικτη εγκατάσταση σε οποιοδήποτε Linux host, (γ) Home Assistant Supervised — υβριδική λύση με Docker και Supervisor εγκατεστημένα σε host του χρήστη και (δ) Home Assistant Core — μόνο ο πυρήνας, χωρίς addons, για ανάπτυξη σε Python virtualenv. Η επιλογή του τρόπου εγκατάστασης δεν επηρεάζει την αρχιτεκτονική των integrations.

Επικοινωνία μέσω MQTT. Από τις δεκάδες υποστηριζόμενες ενσωματώσεις, ξεχωρίζει για την παρούσα εργασία η ενσωμάτωση MQTT του Home Assistant, η οποία παρέχει τη βασική σύνδεση με έναν εξωτερικό MQTT broker. Πέρα από αυτήν την τυποποιημένη ενσωμάτωση, οποιοδήποτε custom integration — όπως αυτό που αναπτύσσεται στην παρούσα εργασία — μπορεί να επεκτείνει τη λειτουργικότητα του Home Assistant, δημιουργώντας ίδιες οντότητες και αξιοποιώντας τη βασική MQTT επικοινωνία για ανταλλαγή μηνυμάτων με εξωτερικές συσκευές. Οι λεπτομέρειες αυτής της επέκτασης παρουσιάζονται στην επόμενη ενότητα.

Custom integrations και διανομή μέσω HACS

2.6.Γ

Παράλληλα με τις επίσημες ενσωματώσεις (official integrations) που συμπεριλαμβάνονται στον πυρήνα του Home Assistant, η πλατφόρμα υποστηρίζει εξίσου τις λεγόμενες custom integrations: ανεξάρτητα Python modules που εγκαθίστανται από τον χρήστη στον κατάλογο /config/custom_components και επεκτείνουν τη λειτουργικότητα του συστήματος με νέους τύπους συσκευών, νέες οντότητες ή υπηρεσίες. Η αρχιτεκτονική των custom integrations είναι ταυτοσημή με αυτήν των επίσημων: ένα αρχείο manifest.json δηλώνει τα μεταδεδομένα (όνομα, εκδόσεις HA που υποστηρίζονται, εξαρτήσεις) και ένα ή περισσότερα αρχεία Python υλοποιούν τη λογική.

Η ύπαρξη του μηχανισμού των custom integrations, είναι κρίσιμη για το οικοσύστημα του Home Assistant. Επιτρέπει σε προγραμματιστές της κοινότητας να καλύπτουν εξειδικευμένες ανάγκες — όπως συσκευές εξειδικευμένων κατασκευαστών ή εξατομικευμένες εφαρμογές παρόμοιες με την παρούσα — χωρίς να αναμένουν την έγκριση και ενσωμάτωσή τους στο επίσημο αποθετήριο, διαδικασία η οποία είναι σχετικά αυστηρή και χρονοβόρα. Παράλληλα, διασφαλίζει ότι ο πυρήνας παραμένει σταθερός και υπό επιτήρηση, αφού τα custom integrations βρίσκονται εκτός αυτού.

Το HACS (Home Assistant Community Store) [10] είναι ένα ξεχωριστό custom integration που λειτουργεί ως διαχειριστής πακέτων για άλλα custom integrations και custom frontend cards. Παρέχει στον χρήστη μια ενιαία διεπαφή μέσω της

οποίας μπορεί να αναζητήσει, να εγκαταστήσει, να ενημερώσει ή να αφαιρέσει επεκτάσεις της κοινότητας, χωρίς να χρειάζεται να κατεβάσει χειροκίνητα αρχεία από το GitHub ή να επεξεργάζεται απευθείας το `/config` μονοπάτι. Η τεχνική αρχιτεκτονική του HACS βασίζεται στο GitHub: κάθε custom integration διανέμεται ως δημόσιο GitHub repository και το HACS αναγνωρίζει τη δομή των αρχείων και τις διαθέσιμες εκδόσεις μέσω git tags ή GitHub releases.

Η εγγραφή στο HACS γίνεται σε δύο επίπεδα. Αρχικά, ο χρήστης μπορεί να προσθέσει οποιοδήποτε GitHub repo ως custom repository στο HACS instance του και να το χρησιμοποιήσει αμέσως. Η ομάδα του HACS διατηρεί ένα default store — ένα επιμελημένο αποθετήριο με integrations που έχουν περάσει βασικούς ελέγχους ποιότητας και είναι ήδη διαθέσιμα προς αναζήτηση σε όλες τις εγκαταστάσεις HACS. Η ένταξη στο default store απαιτεί pull request στο hacs/default repository και επιτυχή έλεγχο της δομής και της λειτουργικότητας του integration μέσω αυτοματοποιημένων εργαλείων: του Hassfest [24] (επίσημο εργαλείο επικύρωσης από τον οργανισμό Home Assistant) και του HACS Action [25].

Η παρούσα εργασία αξιοποιεί άμεσα αυτό το οικοσύστημα. Κατά τη συγγραφή της εργασίας, το integration “IR Remote (ANAVI IR rHAT)” βρισκόταν σε διαδικασία ένταξης στο HACS default store, μέσω εκκρεμούς pull request [26], το οποίο, στη συνέχεια, εγκρίθηκε και έγινε merge στις 2 Ιουλίου 2026 και είναι πλέον διαθέσιμο προς άμεση αναζήτηση και εγκατάσταση μέσα από το HACS. Ένα CI/CD pipeline επιβεβαιώνει σε κάθε νέα έκδοση τη συμμόρφωσή του με τους ελέγχους Hassfest και HACS πριν τη δημοσιοποίηση. Οι λεπτομέρειες αυτής της ροής παρουσιάζονται αναλυτικά στο **Κεφάλαιο 3**.

Συσχέτιση με την παρούσα εργασία

2.6.Δ

Όσα παρουσιάστηκαν στις προηγούμενες ενότητες του **2.6** περιγράφουν την αρχιτεκτονική του Home Assistant και του οικοσυστήματος επεκτάσεων HACS και ταυτόχρονα αιτιολογούν την επιλογή τους για την υλοποίηση της παρούσας εργασίας.

Πρώτον, η αρχιτεκτονική του Home Assistant, βασισμένη σε οντότητες και ένα ασύγχρονο event bus, ευνοεί την επεκτασιμότητα μέσω custom integrations. Η εργασία αξιοποιεί ακριβώς αυτό το μοντέλο: το integration “IR Remote (ANAVI IR rHAT)” δημιουργεί δυναμικά οντότητες button, sensor, number και text — ανά σήμα που έχει μάθει η «γέφυρα» — και αντιδρά σε αλλαγές κατάστασης μέσω του ίδιου του event bus, χωρίς να απαιτεί τροποποίηση στον πυρήνα του Home Assistant.

Δεύτερον, η ενσωμάτωση MQTT του Home Assistant παρέχει την έτοιμη σύνδεση με το δίκτυο επικοινωνίας του συστήματος. Το integration της εργασίας, εγγράφεται στα κατάλληλα topics και επεξεργάζεται τα μηνύματα που λαμβάνει.

Τρίτον, το HACS προσφέρει έναν τυποποιημένο και φιλικό προς τον χρήστη μηχανισμό διανομής. Με μία απλή εγκατάσταση του HACS και προσθήκη του αντιστοιχικού repository, οποιοσδήποτε χρήστης του Home Assistant μπορεί να εγκαταστήσει και να ενημερώνει αυτόματα το integration χωρίς να χρειάζεται

τεχνικές γνώσεις git ή Python. Σε συνδυασμό με το αυτοματοποιημένο CI/CD pipeline που εξασφαλίζει την επιτυχία των ελέγχων Hassfest και HACS Action σε κάθε νέα έκδοση, η εμπειρία εγκατάστασης για τον τελικό χρήστη ισοδυναμεί με εκείνη των επίσημων ενσωματώσεων του Home Assistant.

Με την παρουσίαση του Home Assistant και του HACS ολοκληρώνεται το θεωρητικό πλαίσιο της εργασίας. Στο **Κεφάλαιο 3** που ακολουθεί, περιγράφεται αναλυτικά η υλοποίηση του προτεινόμενου συστήματος, αξιοποιώντας όλα όσα παρουσιάστηκαν στο παρόν κεφάλαιο.

ΚΕΦΑΛΑΙΟ 3 – Υλοποίηση

Εισαγωγή

3.1

Στο κεφάλαιο αυτό περιγράφεται η υλοποίηση του συστήματος που αναπτύχθηκε στο πλαίσιο της παρούσας εργασίας — ένα ολοκληρωμένο πλαίσιο ενσωμάτωσης τυπικών ηλεκτρονικών συσκευών με τηλεχειρισμό υπέρυθρων στο οικοσύστημα του Home Assistant, αξιοποιώντας Raspberry Pi και το ANAVI Infrared pHAT, ως φυσική «γέφυρα» μεταξύ του ψηφιακού και του υπέρυθρου κόσμου. Το βιβλιογραφικό υπόβαθρο που αναπτύχθηκε στο **Κεφάλαιο 2** (έξυπνο σπίτι, υπέρυθρη επικοινωνία, MQTT, Raspberry Pi, Home Assistant, HACS) αποτελεί τη βάση πάνω στην οποία συγκροτείται η παρακάτω σχεδίαση και υλοποίηση.

Η δομή του κεφαλαίου ακολουθεί τη ροή από τη συνολική εικόνα προς τις επιμέρους λεπτομέρειες και κατόπιν προς τη χρήση. Στην **ενότητα 3.2** παρουσιάζεται η συνολική αρχιτεκτονική του συστήματος, με τα δύο κύρια μέρη του («γέφυρα» στο Raspberry Pi και custom integration στο Home Assistant) και το πρωτόκολλο MQTT ως μέσο επικοινωνίας μεταξύ τους. Στην **ενότητα 3.3** αναλύεται η υλοποίηση της «γέφυρας»: το υλικό και το λειτουργικό σύστημα, το λογισμικό σε Python, ο μηχανισμός καταγραφής και αναπαραγωγής υπέρυθρων σημάτων, μέσω της βιβλιοθήκης pigpio, η δομή αποθήκευσης των συσκευών και των πλήκτρων σε JSON, ο μηχανισμός εικονικών πλήκτρων και μακροεντολών και η ενσωμάτωση ως υπηρεσία systemd. Στην **ενότητα 3.4**, περιγράφεται η υλοποίηση του Home Assistant integration "IR Remote (ANAVI IR pHAT)": η δομή του custom component, η ροή διαμόρφωσης (config flow), οι τέσσερις τύποι οντοτήτων (button, sensor, number, text) και ο μηχανισμός δυναμικής δημιουργίας και ενημέρωσής τους, κατά τη διάρκεια εκτέλεσης. Στην **ενότητα 3.5** τυποποιείται το πρωτόκολλο επικοινωνίας με χρήση του MQTT, με αναλυτική παρουσίαση της δομής των topic, του προθέματος, που επιτρέπει πολλαπλές «γέφυρες» και των μηνυμάτων εντολής και ενημέρωσης κατάστασης, που ανταλλάσσουν τα δύο μέρη. Στην **ενότητα 3.6**, αναπτύσσεται η αυτοματοποιημένη δημοσίευση εκδόσεων, μέσω των GitHub Actions, με τα τέσσερα workflows που συνθέτουν το CI/CD pipeline, τους ελέγχους εγκυρότητας HACS και Hassfest, και τη διαδικασία δημοσίευσης του integration στο Home Assistant Community Store. Τέλος, στην **ενότητα 3.7** παρατίθεται ένας

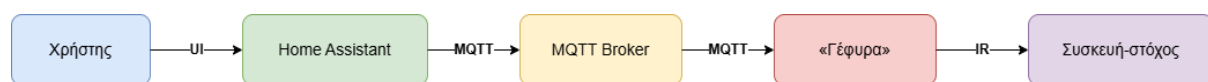
συνοπτικός οδηγός εγκατάστασης και χρήσης, που επιτρέπει στον αναγνώστη να αναπαραγάγει το σύστημα σε δικό του περιβάλλον.

Αρχιτεκτονική του συστήματος

3.2

Η αρχιτεκτονική του συστήματος της παρούσας εργασίας ακολουθεί την τοπικά εστιασμένη (local-first) προσέγγιση που τεκμηριώθηκε στο **Κεφάλαιο 2** και αποτελείται από τρία λογικά μέρη που λειτουργούν εξ ολοκλήρου εντός του οικιακού δικτύου του χρήστη, χωρίς εξάρτηση από οποιαδήποτε εξωτερική υπηρεσία cloud.

Συνολική επισκόπηση. Η ροή πληροφορίας στο σύστημα παρουσιάζεται σχηματικά παρακάτω:



Στη μία άκρη βρίσκεται ο τελικός χρήστης, ο οποίος αλληλεπιδρά με το σύστημα μέσα από το γραφικό περιβάλλον του Home Assistant. Στην άλλη άκρη βρίσκεται μια κλασική υπέρυθρη συσκευή (τηλεόραση, κλιματιστικό, ηχοσύστημα κ.ά.). Μεταξύ τους παρεμβάλλονται τρία τμήματα λογισμικού-υλικού: το Home Assistant με το προσαρμοσμένο integration της παρούσας εργασίας, ένας MQTT broker (συνήθως ο Mosquitto, εγκατεστημένος ως add-on του Home Assistant) και η «γέφυρα» που βασίζεται στο Raspberry Pi με το ANAVI Infrared pHAT.

Ρόλος κάθε συνιστώσας. Το integration του Home Assistant αναλαμβάνει τρεις λειτουργίες: (α) δημιουργεί δυναμικά οντότητες (button, sensor, number, text) που αναπαριστούν τα διαθέσιμα πλήκτρα του τηλεχειριστηρίου και τις κρίσιμες παραμέτρους του συστήματος, (β) μεταφράζει τα events οντοτήτων που προκαλεί ο χρήστης σε MQTT μηνύματα εντολής και (γ) ενημερώνει τις οντότητες όταν λαμβάνει μηνύματα κατάστασης από τη «γέφυρα». Ο MQTT broker αναλαμβάνει αποκλειστικά τη δρομολόγηση μηνυμάτων μεταξύ Home Assistant και «γέφυρας» — δεν εκτελεί καμία λογική του συστήματος, αλλά παρέχει την υποδομή pub/sub. Τέλος, η «γέφυρα» είναι ο εκτελεστής στο φυσικό επίπεδο: λαμβάνει IR σήματα από φυσικά τηλεχειριστήρια κατά το στάδιο εκμάθησης, αποθηκεύει τα χρονικά τους προφίλ και αναπαράγει αυτά τα σήματα κατά την εντολή που λαμβάνει από το Home Assistant.

Ροή εντολής. Όταν ο χρήστης πατήσει ένα κουμπί στο Home Assistant — π.χ. το "Power" μιας τηλεόρασης — η ακόλουθη αλληλουχία εκτελείται: (1) το integration δημοσιεύει στο κατάλληλο MQTT topic ένα μήνυμα εντολής που περιέχει το όνομα της συσκευής και του πλήκτρου, (2) ο broker προωθεί το μήνυμα στη «γέφυρα» που έχει εγγραφεί στο topic αυτό, (3) η «γέφυρα» αναζητά το αποθηκευμένο χρονικό προφίλ του πλήκτρου και (4) οδηγεί τα IR LED του ANAVI pHAT, ώστε να εκπέμπουν το σήμα προς την τηλεόραση. Η συνολική καθυστέρηση

από το πάτημα του κουμπιού μέχρι την εκπομπή τυπικά παραμένει κάτω από 200 ms.

Ροή κατάστασης και ανακάλυψης. Σε αντίθετη κατεύθυνση, η «γέφυρα» δημοσιεύει σε δεδομένα topics — με τη σημαία `retained=true` — τη λίστα των διαχειριζόμενων συσκευών, τα διαθέσιμα πλήκτρα ανά συσκευή και την τρέχουσα κατάσταση καταγραφής (`idle`, `recording`, `done`, `error`). Το integration του Home Assistant είναι εγγεγραμμένο σε αυτά τα topics και ανταποκρίνεται άμεσα: όταν αναγνωρίσει νέα συσκευή ή νέο πλήκτρο, δημιουργεί αυτόματα τις αντίστοιχες οντότητες, χωρίς να απαιτείται επανεκκίνηση του Home Assistant. Αυτή η δυναμική ανακάλυψη επιτρέπει στον χρήστη να διαχειρίζεται όλες τις συσκευές αποκλειστικά μέσω της διεπαφής του Home Assistant.

Πολλαπλές «γέφυρες». Η αρχιτεκτονική επεκτείνεται φυσικά σε σενάρια με πολλαπλές «γέφυρες», σε διαφορετικά δωμάτια του σπιτιού. Κάθε «γέφυρα» διαμορφώνεται με δικό της MQTT topic prefix (π.χ. `ir_remote`, `ir_remote_bedroom`) και το integration προστίθεται μία φορά ανά «γέφυρα» στο Home Assistant. Όλες οι «γέφυρες» επικοινωνούν μέσω του ίδιου broker, αλλά είναι λογικά απομονωμένες μεταξύ τους, χάρη στα διαφορετικά topic prefixes.

Υλοποίηση της «γέφυρας» στο Raspberry Pi

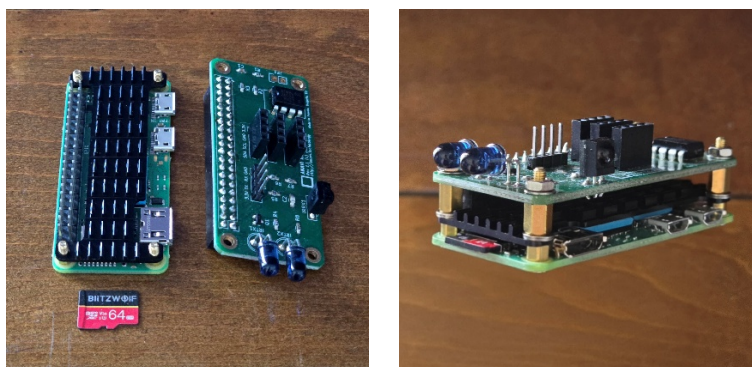
3.3

Εγκατάσταση υλικού και Λειτουργικού Συστήματος

3.3.A

Η υλοποίηση της «γέφυρας» ξεκινά με τη συναρμολόγηση του υλικού και την εγκατάσταση του λειτουργικού συστήματος στο Raspberry Pi. Στην ενότητα αυτή περιγράφονται οι σχετικές διαδικασίες, καθώς και κάποιες αποφάσεις που επηρέασαν τη συνέχεια της υλοποίησης.

Συναρμολόγηση υλικού. Το ANAVI Infrared pHAT τοποθετείται κατευθείαν πάνω στο 40-pin GPIO header του Raspberry Pi, χωρίς να απαιτούνται συγκολλήσεις ή πρόσθετα καλώδια. Η σύνδεση είναι μηχανικά σταθερή και ηλεκτρικά πλήρης: τόσο η τροφοδοσία όσο και τα σήματα δεδομένων μεταφέρονται μέσω του ίδιου header. Η πλακέτα παρέχει εξωτερική πρόσβαση στους ίδιους ακροδέκτες GPIO, ώστε ο χρήστης να μπορεί να συνδέσει επιπλέον αισθητήρες I2C αν το επιθυμεί — λειτουργία που ωστόσο δεν αξιοποιείται στην παρούσα εφαρμογή.



Εικόνα 3.1: Αριστερά, τα επιμέρους εξαρτήματα της «γέφυρας» (Raspberry Pi Zero W με heatsink, ANAVI Infrared pHAT, microSD κάρτα). Δεξιά, η ολοκληρωμένη «γέφυρα» μετά τη συναρμολόγηση.

Εγκατάσταση λειτουργικού συστήματος. Για την εγκατάσταση του Raspberry Pi OS χρησιμοποιήθηκε το επίσημο εργαλείο Raspberry Pi Imager [22], που διατίθεται από το Raspberry Pi Foundation. Από τη λίστα διαθέσιμων εκδόσεων επιλέχθηκε το Raspberry Pi OS Lite (64-bit) — η ελαφριά headless έκδοση χωρίς γραφικό περιβάλλον, η οποία είναι κατάλληλη για εφαρμογές όπως η παρούσα, που λειτουργούν ως διεργασίες στο παρασκήνιο. Το Raspberry Pi Imager παρέχει, στη συνέχεια, τη δυνατότητα διαμόρφωσης κρίσιμων παραμέτρων, πριν την εγγραφή στη κάρτα microSD: το όνομα συσκευής (hostname), τον λογαριασμό χρήστη (username), στοιχεία ασύρματου δικτύου (Wi-Fi) και ενεργοποίηση της υπηρεσίας SSH. Όλες αυτές οι παράμετροι ορίστηκαν σε αυτή τη φάση, ώστε το Pi να είναι έτοιμο για απομακρυσμένη πρόσβαση από την πρώτη εκκίνηση, χωρίς να απαιτείται οθόνη και πληκτρολόγιο.

Αρχική διαμόρφωση. Μετά την πρώτη εκκίνηση, εκτελέστηκε ενημέρωση του συστήματος μέσω των εντολών `sudo apt update` και `sudo apt upgrade`, ώστε όλα τα πακέτα να βρίσκονται στην πιο πρόσφατη διαθέσιμη έκδοση. Στη συνέχεια επιβεβαιώθηκε η σωστή αναγνώριση του pHAT, καθώς η διεπαφή GPIO ήταν ήδη ενεργή εξ' ορισμού, στη χρησιμοποιούμενη έκδοση του Raspberry Pi OS, χωρίς να απαιτείται χειροκίνητη ενεργοποίηση, όπως σε παλαιότερες εκδόσεις.

Σημείωση για την έκδοση του λειτουργικού. Κατά τη διάρκεια της υλοποίησης παρατηρήθηκε ότι οι επίσημες οδηγίες της ANAVI Technology για το Infrared pHAT [23], απευθύνονταν σε παλαιότερη έκδοση του Raspberry Pi OS (Bookworm) και βασίζονταν στη χρήση του LIRC ως kernel-level driver. Στην έκδοση Trixie, η οποία χρησιμοποιήθηκε στην παρούσα εργασία, το LIRC είχε αφαιρεθεί από το επίσημο apt repository και δεν λειτουργούσε αξιόπιστα. Η αλλαγή αυτή οδήγησε στην επιλογή μιας εναλλακτικής προσέγγισης για τον προγραμματισμό του υλικού σε user-space, η οποία περιγράφεται αναλυτικά στις επόμενες ενότητες.

Αλληλεξάρτηση των δύο τμημάτων. Σημειώνεται ότι τα δύο τμήματα της εργασίας — η «γέφυρα» και το integration του Home Assistant — είναι αλληλένδετα: το integration, χωρίς ενεργή «γέφυρα», δεν έχει συσκευές να ελέγξει, ενώ η «γέφυρα», χωρίς το integration, μπορεί μεν να λειτουργήσει αυτόνομα μέσω του CLI εργαλείου της (remote.py), αλλά δεν εκθέτει τη λειτουργικότητά της σε αυτοματισμούς του χρήστη ή σε γραφική διεπαφή. Τα δύο σχεδιάστηκαν εξ αρχής για να συνεργάζονται και διανέμονται σε ξεχωριστά repositories ώστε καθένα να μπορεί να εξελίσσεται ανεξάρτητα.

Το λογισμικό της «γέφυρας» αποτελεί μια ανεξάρτητη υπηρεσία υλοποιημένη σε Python 3, η οποία εκτελείται στο Raspberry Pi και διαμεσολαβεί μεταξύ του MQTT broker και του ANAVI Infrared pHAT. Ο πλήρης κώδικας είναι διαθέσιμος στο δημόσιο GitHub repository “pi-dumb2smart-ir-bridge” [27].

Επιλογή γλώσσας. Η επιλογή της Python δεν είναι τυχαία: αποτελεί την default γλώσσα του οικοσυστήματος Raspberry Pi, υποστηρίζεται εξαιρετικά καλά στο Raspberry Pi OS, διαθέτει πλούσιο σύνολο βιβλιοθηκών για ηλεκτρονικά και IoT και επιτρέπει ταχύ προγραμματισμό και δοκιμή. Η σχετικά χαμηλότερη απόδοση σε σύγκριση με γλώσσες όπως η C, δεν αποτελεί περιορισμό στην παρούσα εφαρμογή, καθώς οι χρονικά κρίσιμες λειτουργίες (παραγωγή υπέρυθρων παλμών σε ακρίβεια ms) διαχειρίζονται από εξειδικευμένες βιβλιοθήκες που δίνουν εντολές σε kernel-level daemon (διεργασία που εκτελείται στο παρασκήνιο).

Εξαρτήσεις λογισμικού. Τρεις είναι οι κύριες εξωτερικές εξαρτήσεις της «γέφυρας». Η pigpio [28] είναι μια βιβλιοθήκη υψηλής ακρίβειας για τον προγραμματισμό των GPIO του Raspberry Pi. Αντί να βασίζεται σε επανειλημμένες κλήσεις από user-space, χρησιμοποιεί έναν background daemon, το pigpiod, που τρέχει με root προνόμια και έχει πρόσβαση σε hardware timers, εξασφαλίζοντας ακρίβεια της τάξης του μικροδευτερολέπτου — γεγονός που την καθιστά κατάλληλη για την παραγωγή χρονικά κρίσιμων IR σημάτων. Η PiIR [29] είναι μια εξειδικευμένη βιβλιοθήκη που χτίζεται πάνω στο pigpio και προσφέρει υψηλότερου επιπέδου API για την εκπομπή και λήψη υπέρυθρων σημάτων. Παρέχει έτοιμη υλοποίηση για τα βασικά πρωτόκολλα IR (NEC, RC5, Sony SIRC κ.ά.), αλλά υποστηρίζει και την καταγραφή και αναπαραγωγή σε raw timing format, που είναι η προσέγγιση που υιοθετείται στην παρούσα εργασία (όπως τεκμηριώθηκε στο **2.3.Γ**). Τέλος, η paho-mqtt [30] είναι η de facto βιβλιοθήκη Python για MQTT επικοινωνία· παρέχει ασύγχρονο API για connect, subscribe, publish, καθώς και callbacks για εισερχόμενα μηνύματα, υποστηρίζοντας πλήρως τα QoS levels και τα retained messages που αναλύθηκαν στην **ενότητα 2.4**.

Αρχιτεκτονική κώδικα. Ο κώδικας οργανώνεται σε δύο κύρια Python αρχεία. Το mqtt_bridge.py αποτελεί το κύριο service που εκτελείται διαρκώς· αναλαμβάνει τη σύνδεση με τον broker, την εγγραφή στα command topics, τη φόρτωση των αποθηκευμένων device JSON files (που περιγράφονται στο **3.3.Δ**), και τη δρομολόγηση κάθε εισερχόμενου μηνύματος στην αντίστοιχη λειτουργία (αναπαραγωγή IR, καταγραφή νέου πλήκτρου, διαχείριση συσκευών). Το remote.py είναι ένα προαιρετικό CLI εργαλείο για άμεση τοπική χρήση από SSH· επιτρέπει στον προχωρημένο χρήστη να δημιουργεί συσκευές, να καταγράφει πλήκτρα και να ελέγχει το σύστημα χωρίς να περνά μέσω του Home Assistant και είναι χρήσιμο κυρίως για debugging και initial setup.

Συμπληρωματικά αρχεία. Στο repository περιλαμβάνονται επίσης shell scripts για εγκατάσταση (install.sh), απεγκατάσταση (uninstall.sh), ενημέρωση (update.sh) και αλλαγή του MQTT topic prefix (change_mqtt_topic.sh), καθώς και η μονάδα systemd (ir-bridge.service) για την αυτόματη εκκίνηση της υπηρεσίας κατά την εκκίνηση του Λειτουργικού Συστήματος, όπως περιγράφεται στην **ενότητα 3.3.ΣΤ**.

Η καρδιά της λειτουργικότητας της «γέφυρας», είναι ο μηχανισμός μάθησης (recording) και αναπαραγωγής (playback) υπέρυθρων σημάτων. Στην **ενότητα 2.3.Γ** τεκμηριώθηκε η γενική προσέγγιση raw timing που υιοθετείται· εδώ περιγράφεται η συγκεκριμένη υλοποίηση στον κώδικα της «γέφυρας», ο οποίος βασίζεται στις βιβλιοθήκες pigpio [28] και PiIR [29] που παρουσιάστηκαν στην προηγούμενη ενότητα.

Ροή καταγραφής. Όταν το integration του Home Assistant δημοσιεύσει στον broker ένα μήνυμα εντολής για καταγραφή νέου πλήκτρου, η ροή στη «γέφυρα» εξελίσσεται ως εξής. Πρώτον, το mqtt_bridge.py λαμβάνει το μήνυμα μέσω του raho-mqtt callback· το μήνυμα περιέχει το όνομα της συσκευής (π.χ. samsung_tv) και το όνομα του νέου πλήκτρου (π.χ. power). Δεύτερον, η «γέφυρα» δημοσιεύει αμέσως κατάσταση recording στο αντίστοιχο status topic (με retained=true), ώστε το Home Assistant να ενημερωθεί και να εμφανίσει την αλλαγή στον χρήστη. Τρίτον, καλείται το PiIR record API, το οποίο ενεργοποιεί τον υπέρυθρο δέκτη του ANAVI pHAT και αναμένει την εκπομπή σήματος από ένα φυσικό τηλεχειριστήριο εντός χρονικού ορίου 30 δευτερολέπτων. Τέταρτον, εάν εντοπιστεί σήμα εντός του ορίου, το PiIR επιστρέφει την ακολουθία raw timing (μια λίστα ακεραίων που αναπαριστούν διαδοχικά τα ON και OFF διαστήματα του σήματος σε ms). Η «γέφυρα» αποθηκεύει το νέο πλήκτρο μαζί με το σχετικό timing στο JSON αρχείο της συσκευής και δημοσιεύει κατάσταση done. Αν παρέλθει το timeout χωρίς να ληφθεί σήμα, δημοσιεύεται κατάσταση timeout και η διαδικασία ακυρώνεται χωρίς αλλαγή στο αρχείο.

Ροή αναπαραγωγής. Η αναπαραγωγή είναι σαφώς ταχύτερη και απλούστερη από την καταγραφή. Όταν το mqtt_bridge.py λαμβάνει μήνυμα τύπου press για συγκεκριμένο πλήκτρο μιας συσκευής, φορτώνει το αντίστοιχο JSON αρχείο της συσκευής (αν δεν είναι ήδη στη μνήμη), εντοπίζει το ζητούμενο πλήκτρο και ανακτά την αποθηκευμένη ακολουθία timing. Στη συνέχεια καλείται το PiIR send_raw API, το οποίο οδηγεί τα υπέρυθρα LED του pHAT με τη φέρουσα συχνότητα 38 kHz και αναπαράγει την ακολουθία timing με ακρίβεια μικροδευτερολέπτου, χάρη στον hardware-timed daemon της pigpio. Σε περίπτωση που για το συγκεκριμένο πλήκτρο έχουν διαμορφωθεί παράμετροι επανάληψης (πλήθος επαναλήψεων και καθυστέρηση μεταξύ τους), η αναπαραγωγή επαναλαμβάνεται τις απαιτούμενες φορές με την κατάλληλη αναμονή· λεπτομέρειες για τα repeat και τα εικονικά πλήκτρα δίνονται στην **ενότητα 3.3.Ε**.

Πεπερασμένη Μηχανή Καταστάσεων. Η «γέφυρα» διατηρεί ένα σαφές Finite State Machine για την κατάσταση του δέκτη, με πέντε τιμές: idle (αναμονή), recording (καταγραφή σε εξέλιξη), done (επιτυχής καταγραφή), error (απρόσμενο σφάλμα κατά τη διαδικασία) και timeout (λήξει χρονικού ορίου). Κάθε αλλαγή κατάστασης δημοσιεύεται ως retained MQTT μήνυμα, επιτρέποντας στο Home Assistant να αναπαριστά τη ζωντανή κατάσταση μέσω της αντίστοιχης sensor οντότητας. Με αυτόν τον τρόπο, ο χρήστης παρακολουθεί σε πραγματικό χρόνο τη διαδικασία μάθησης νέου πλήκτρου απευθείας από τη διεπαφή του Home Assistant, χωρίς να χρειάζεται πρόσβαση στο Raspberry Pi.

Περιορισμός σε συσκευές με stateful εντολές. Η μέθοδος raw timing αποθηκεύει την ακολουθία παλμών όπως εκπέμπεται από το τηλεχειριστήριο τη στιγμή της καταγραφής. Σε συσκευές όπου κάθε πάτημα κουμπιού μεταδίδει την πλήρη τρέχουσα κατάσταση της συσκευής — όπως συμβαίνει χαρακτηριστικά στα κλιματιστικά, όπου το ίδιο IR πλαίσιο περιέχει ταυτόχρονα θερμοκρασία, λειτουργία και ταχύτητα ανεμιστήρα — η αναπαραγωγή επιβάλλει τη συγκεκριμένη αποθηκευμένη κατάσταση και δεν διατηρεί τις τυχόν χειροκίνητες ρυθμίσεις που είχε εφαρμόσει, εν τω μεταξύ, ο χρήστης από το αρχικό τηλεχειριστήριο. Για τα κλιματιστικά, επομένως, η μέθοδος λειτουργεί αξιόπιστα κυρίως για την απενεργοποίηση. Αναλυτική συζήτηση του περιορισμού δίνεται στην **ενότητα 4.5**.

Δομή αποθήκευσης συσκευών και πλήκτρων

3.3.Δ

Η «γέφυρα» χρειάζεται έναν αξιόπιστο και ανθεκτικό σε επανεκκινήσεις μηχανισμό αποθήκευσης για τα σήματα IR που έχει μάθει. Στην παρούσα υλοποίηση η επιλογή είναι η αποθήκευση σε JSON αρχεία, ένα ανά συσκευή, στο τοπικό σύστημα αρχείων του Raspberry Pi.

Σκεπτικό σχεδίασης. Δεδομένου του πλήθους και του μεγέθους των δεδομένων (συνήθως μερικές δεκάδες έως μερικές εκατοντάδες σήματα συνολικά σε ένα σπίτι, με κάθε σήμα να καταλαμβάνει λίγες εκατοντάδες ακεραίους), η χρήση μιας πλήρους βάσης δεδομένων όπως SQLite ή PostgreSQL θα ήταν υπερβολή και θα εισήγαγε περιττή πολυπλοκότητα. Από την άλλη, ένα ενιαίο μεγάλο JSON αρχείο θα δυσκόλευε την παράλληλη επεξεργασία και θα ήταν δύσκολο να επεξεργαστεί χειροκίνητα. Η επιλογή ενός JSON αρχείου ανά συσκευή προσφέρει την κατάλληλη ισορροπία: απλότητα, αναγνωσιμότητα, ευκολία στη επεξεργασία και φυσικό διαχωρισμό των δεδομένων ανά οντότητα.

Τοποθεσία και ονοματολογία. Τα αρχεία αποθηκεύονται στον κατάλογο που ορίζεται από τη μεταβλητή περιβάλλοντος IR_DATA_DIR (ή στον κατάλογο εκτέλεσης του script αν η μεταβλητή δεν έχει οριστεί). Κάθε συσκευή αποθηκεύεται σε αρχείο με όνομα της μορφής {device_name}.json, όπου το όνομα της συσκευής μετατρέπεται σε πεζά γράμματα και τα κενά αντικαθίστανται με κάτω παύλες (π.χ. samsung_tv.json, living_room_ac.json).

Δομή του JSON. Κάθε αρχείο περιέχει τρία βασικά top-level πεδία. Το πρώτο είναι το keys, υποχρεωτικό · πρόκειται για ένα αντικείμενο που αντιστοιχίζει το όνομα κάθε πλήκτρου στο αντίστοιχο IR σήμα του (κωδικοποιημένο ως raw timing array, όπως αναλύθηκε στο **3.3.Γ**). Το δεύτερο είναι το key_options, προαιρετικό · επιτρέπει τη ρύθμιση παραμέτρων ανά πλήκτρο, ιδίως της επανάληψης: για συγκεκριμένα πλήκτρα μπορεί να οριστεί ο αριθμός των επαναλήψεων (repeat) και η καθυστέρηση σε χιλιοστά του δευτερολέπτου μεταξύ διαδοχικών εκπομπών (delay_ms). Αυτή η ρύθμιση είναι κρίσιμη για συσκευές που απαιτούν πολλαπλά πατήματα ώστε να αναγνωρίσουν ένα σήμα (π.χ. ορισμένοι ενισχυτές home cinema). Το τρίτο είναι το virtual_keys, επίσης προαιρετικό · πρόκειται για ένα αντικείμενο που ορίζει εικονικά πλήκτρα, τα οποία αποτελούν μακροεντολές πάνω από υπαρκτά πλήκτρα. Κάθε εικονικό πλήκτρο περιέχει τρία υπο-πεδία: key (το όνομα του πραγματικού πλήκτρου που θα εκτελεστεί), repeat (πλήθος επαναλήψεων) και delay_ms (καθυστέρηση μεταξύ τους). Λεπτομερής ανάλυση των εικονικών

πλήκτρων δίνεται στην **ενότητα 3.3.E**. Για παράδειγμα, ένα να τυπικό αρχείο `samsung_tv.json` θα έχει τη μορφή:

```
{
  "keys": {
    "power": [9000, 4500, 560, 560, 560, 1690, ...],
    "volume_up": [9000, 4500, 560, 560, ...]
  },
  "key_options": {
    "power": { "repeat": 2, "delay_ms": 300 }
  },
  "virtual_keys": {
    "vol_up_5": { "key": "volume_up", "repeat": 5, "delay_ms": 150 }
  }
}
```

Φόρτωση και ενημέρωση. Κατά την εκκίνηση, η «γέφυρα» καλεί τη συνάρτηση `load_all_devices()`, η οποία σαρώνει τον `IR_DATA_DIR` για όλα τα αρχεία τύπου JSON και τα φορτώνει στη μνήμη. Όταν ο χρήστης προσθέσει ή αφαιρέσει συσκευή ή πλήκτρο μέσω του Home Assistant ή του εργαλείου CLI της «γέφυρας», το αντίστοιχο αρχείο ενημερώνεται άμεσα και η «γέφυρα» δημοσιεύει μέσω MQTT την ενημερωμένη λίστα συσκευών και πλήκτρων προς το Home Assistant. Επιπλέον, παρέχεται η δυνατότητα χειροκίνητης σάρωσης, μέσω του κουμπιού “Reload Devices”, στη διεπαφή του Home Assistant, η οποία είναι χρήσιμη όταν ο χρήστης έχει επεξεργαστεί χειροκίνητα τα αρχεία JSON μέσω SSH.

Εικονικά πλήκτρα και μακροεντολές

3.3.E

Πέρα από τα κανονικά πλήκτρα που αντιστοιχούν ένα προς ένα σε φυσικά κουμπιά τηλεχειριστηρίων, η «γέφυρα» υποστηρίζει και τα λεγόμενα εικονικά πλήκτρα (virtual keys), τα οποία λειτουργούν ως μακροεντολές πάνω σε υπαρκτά πλήκτρα. Ένα εικονικό πλήκτρο, με ένα και μόνο πάτημα από τον χρήστη, εκτελεί N διαδοχικές αναπαραγωγές ενός κανονικού πλήκτρου με σταθερή καθυστέρηση μεταξύ τους.

Σκοπιμότητα. Η ανάγκη για αυτή τη λειτουργικότητα προέκυψε από πραγματικές περιπτώσεις χρήσης. Πολλές συσκευές απαιτούν πολλαπλά διαδοχικά πατήματα του ίδιου πλήκτρου, για να φτάσουν στην επιθυμητή κατάσταση: για παράδειγμα, η αύξηση της έντασης ήχου κατά πέντε επίπεδα προϋποθέτει πέντε πατήματα του “volume_up”, ή η ενεργοποίηση ενός κλιματιστικού με δύο συνεχόμενες εντολές “power”, σε ορισμένα μοντέλα, όπου το πρώτο πάτημα δεν έχει πάντα ανταπόκριση. Χωρίς εικονικά πλήκτρα, ο χρήστης θα έπρεπε να συντάξει έναν αυτοματισμό στο Home Assistant που να καλεί το ίδιο κουμπί πολλές φορές με χρονοκαθυστερήσεις — μια ασκόπως πολύπλοκη λύση για ένα συχνό σενάριο.

Δομή ενός εικονικού πλήκτρου. Όπως παρουσιάστηκε στην **ενότητα 3.3.Δ**, κάθε εικονικό πλήκτρο αποθηκεύεται στο πεδίο `virtual_keys` του JSON αρχείου της συσκευής και περιέχει τρία υπο-πεδία: το `key` που αναφέρει το όνομα του πραγματικού πλήκτρου, το `repeat` που ορίζει το πλήθος των διαδοχικών επαναλήψεων και το `delay_ms` που ορίζει την καθυστέρηση μεταξύ τους σε χιλιοστά του δευτερολέπτου (ms). Σημειώνεται ότι το εικονικό πλήκτρο δεν περιέχει δικό

του IR σήμα — βασίζεται πλήρως στο υπάρχον πλήκτρο, οπότε αν το αρχικό πλήκτρο τροποποιηθεί ή διαγραφεί, το εικονικό ακολουθεί την ίδια τύχη.

Παράδειγμα. Έστω ότι η συσκευή `samsung_tv` έχει το πλήκτρο `volume_up`. Ο χρήστης μπορεί να δημιουργήσει ένα εικονικό πλήκτρο με όνομα `vol_up_5` που εκτελεί 5 διαδοχικές εκπομπές του `volume_up` με 150 ms καθυστέρηση μεταξύ τους. Το αποτέλεσμα στο JSON είναι:

```
"vol_up_5": {  
  "key": "volume_up",  
  "repeat": 5,  
  "delay_ms": 150  
}
```

Στο Home Assistant εμφανίζεται αυτόματα ένα νέο `button.samsung_tv_vol_up_5` το οποίο, με ένα μόνο πάτημα, αυξάνει την ένταση κατά πέντε βήματα.

Σύγκριση με `key_options`. Στην **ενότητα 3.3.Δ** παρουσιάστηκε, επίσης, το πεδίο `key_options`, το οποίο δέχεται παρόμοιες ρυθμίσεις (`repeat`, `delay_ms`) αλλά ισχύει για το ίδιο το πλήκτρο. Η ουσιαστική διαφορά είναι ότι το `key_options` επηρεάζει κάθε πάτημα του κανονικού πλήκτρου, ενώ το εικονικό πλήκτρο δημιουργεί ένα ξεχωριστό κουμπί στη διεπαφή που εκτελεί το `multi-press`. Με άλλα λόγια, το `key_options` ταιριάζει όταν η επανάληψη πρέπει να είναι ενσωματωμένη στο σήμα (π.χ. ένα κλιματιστικό που πάντα χρειάζεται διπλό πάτημα), ενώ τα εικονικά πλήκτρα ταιριάζουν όταν θέλουμε διακριτές, παράλληλες παραλλαγές του ίδιου σήματος (π.χ. `vol_up_5` και `vol_up_10` ως ξεχωριστά `shortcuts`, με το αρχικό `vol_up` να παραμένει μονό πάτημα).

Δημιουργία και διαχείριση. Όλη η διαχείριση των εικονικών πλήκτρων γίνεται αποκλειστικά μέσα από τη διεπαφή του Home Assistant, χωρίς να απαιτείται η επεξεργασία των αρχείων JSON. Ο χρήστης συμπληρώνει τέσσερα πεδία (όνομα νέου εικονικού πλήκτρου, όνομα αρχικού πλήκτρου, πλήθος επαναλήψεων, καθυστέρηση σε ms) και πατά το κουμπί “Register Virtual Key” στη διεπαφή. Η ίδια διαδικασία διαγραφής εφαρμόζεται για τα εικονικά πλήκτρα όπως και για τα κανονικά: μέσω του κουμπιού “Delete Key”, αφού οριστεί το όνομά του στο αντίστοιχο πεδίο.

Υπηρεσία `systemd`

3.3.ΣΤ

Η «γέφυρα» πρέπει να λειτουργεί αξιόπιστα 24/7, ξεκινώντας αυτόματα μετά από επανεκκίνηση του Raspberry Pi και να επανέρχεται αυτόματα σε περίπτωση μη απρόοπτου τερματισμού. Για τον σκοπό αυτό υιοθετείται το `systemd` [31], το προεπιλεγμένο `init system` και `service manager` όλων των σύγχρονων διανομών του Linux, συμπεριλαμβανομένου του Raspberry Pi OS.

Το `systemd` οργανώνει τις υπηρεσίες του συστήματος σε αυτόνομες «μονάδες» (`units`), περιγραφόμενες σε απλά αρχεία κειμένου με κατάληξη “`.service`”. Κάθε `unit` ορίζει πώς εκκινείται η υπηρεσία, από ποιες άλλες υπηρεσίες εξαρτάται, υπό ποιες συνθήκες πρέπει να επανεκκινείται και πότε να ενεργοποιείται κατά την εκκίνηση του συστήματος.

Το αρχείο `ir-bridge.service`. Στο repository της «γέφυρας» περιλαμβάνεται το αρχείο `ir-bridge.service`, το οποίο ορίζει την εκτέλεση του `mqtt_bridge.py`. Τα σημαντικότερα directives είναι:

- "Description": ένα όνομα της υπηρεσίας (service), που εμφανίζεται στα logs και στην έξοδο του `systemctl status`.
- "Wants=network-online.target" και "After=network-online.target pigpiod.service": η υπηρεσία ξεκινά αφού το δίκτυο είναι πλήρως διαθέσιμο και αφού έχει εκκινηθεί ο daemon της pigpio, τα οποία αποτελούν προϋποθέσεις για τη σωστή λειτουργία της «γέφυρας».
- "Type=simple": η υπηρεσία θεωρείται ενεργή, μόλις ξεκινήσει η διεργασία που ορίζει το ExecStart, χωρίς να απαιτείται διπλό fork ή ειδικό signalling.
- "WorkingDirectory=/var/lib/ir-bridge": ο κατάλογος εργασίας στον οποίο εκτελείται η υπηρεσία· εδώ αποθηκεύονται και τα JSON αρχεία των συσκευών (3.3.Δ), ακολουθώντας τη σύμβαση του Filesystem Hierarchy Standard για variable state data.
- "EnvironmentFile=/etc/ir-bridge/env": οι ρυθμίσεις της υπηρεσίας (διεύθυνση MQTT broker, topic prefix κ.ά.) διαβάζονται από εξωτερικό αρχείο μεταβλητών περιβάλλοντος, διαχωρίζοντας τη διαμόρφωση από τον κώδικα και επιτρέποντας τροποποίηση χωρίς επανέκδοσή του.
- "Environment=PYTHONUNBUFFERED=1": αναγκάζει την Python να γράφει stdout και stderr χωρίς buffering, ώστε τα μηνύματα να εμφανίζονται άμεσα στο journal, αντί να χάνονται σε ενδιάμεσες ουρές.
- "ExecStart=/opt/ir-bridge/venv/bin/python /opt/ir-bridge/mqtt_bridge.py": ο πραγματικός κώδικας εκτελείται μέσα από ένα αυτόνομο Python virtualenv, στον κατάλογο `/opt/ir-bridge/`, ώστε να μην εξαρτάται από τη system-wide Python εγκατάσταση και να αποφεύγονται συγκρούσεις με άλλα πακέτα.
- "Restart=always", "RestartSec=5" και "StartLimitIntervalSec=0": σε οποιοδήποτε τερματισμό της υπηρεσίας (είτε ομαλό είτε λόγω σφάλματος), το systemd την επανεκκινεί μετά από 5 δευτερόλεπτα, χωρίς ανώτατο όριο επαναλήψεων σε δεδομένο χρονικό παράθυρο. Αυτή η ρύθμιση εξασφαλίζει ότι ένα παροδικό πρόβλημα στη σύνδεση MQTT ή ένα μη αναμενόμενο exception δεν αφήνει τη «γέφυρα» μόνιμα εκτός λειτουργίας.
- "StandardOutput=journal" και "StandardError=journal": όλη η έξοδος της υπηρεσίας ανακατευθύνεται στο systemd journal, διευκολύνοντας την κεντρική διαχείριση των logs.
- "WantedBy=multi-user.target": η υπηρεσία εκκινείται αυτόματα κατά την κανονική εκκίνηση του συστήματος (multi-user runlevel).

Εγκατάσταση και ενεργοποίηση. Η εγκατάσταση της υπηρεσίας αυτοματοποιείται μέσω του `install.sh` script που περιλαμβάνεται στο repository. Το script αντιγράφει το `ir-bridge.service` στον κατάλογο `/etc/systemd/system` του Pi, τα αρχεία κώδικα στο `/opt/ir-bridge/` και τις ρυθμίσεις στο `/etc/ir-bridge/env`. Έπειτα, εκτελεί `systemctl daemon-reload` για να αναγνωριστεί η νέα υπηρεσία από το systemd, `systemctl enable ir-bridge` για να τρέχει με κάθε εκκίνηση και

"systemctl start ir-bridge" για να ξεκινήσει αμέσως. Η πλήρης απεγκατάσταση γίνεται μέσω του `uninstall.sh`, ενώ το `update.sh` διευκολύνει τις μελλοντικές ενημερώσεις χωρίς να απαιτείται από τον χρήστη η γνώση των εντολών `systemctl`.

Διαχείριση και διάγνωση. Από τη στιγμή που η υπηρεσία είναι ενεργή, η διαχείρισή της γίνεται με τις τυποποιημένες εντολές `systemctl`: "`systemctl status ir-bridge`" για την τρέχουσα κατάσταση, "`systemctl restart ir-bridge`" για χειροκίνητη επανεκκίνηση και "`systemctl stop ir-bridge`" για τερματισμό. Για την παρακολούθηση των logs, η εντολή "`journalctl -u ir-bridge -f`" παρέχει live tail των μηνυμάτων της υπηρεσίας, εξαιρετικά χρήσιμη κατά τη φάση debugging και διάγνωσης προβλημάτων σύνδεσης ή αναγνώρισης συσκευών.

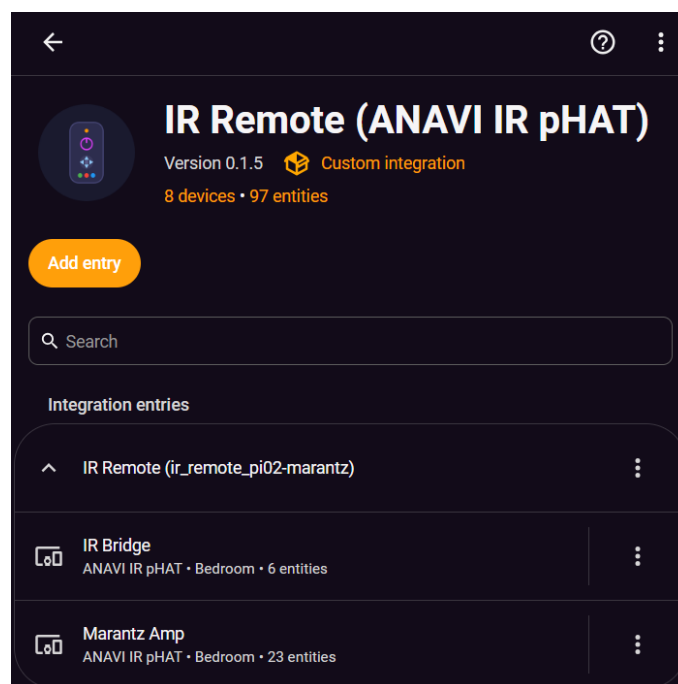
Υλοποίηση του Home Assistant Integration

3.4

Δομή του custom component

3.4.A

Το δεύτερο μέρος του συστήματος, είναι το custom component "IR Remote (ANAVI IR pHAT)" για το Home Assistant, το οποίο έρχεται σε επαφή με τον τελικό χρήστη, μέσα από το γραφικό περιβάλλον της πλατφόρμας και μεταφράζει τις ενέργειές του σε MQTT μηνύματα προς τη «γέφυρα». Το integration αυτό απαντεί ενεργή «γέφυρα» στο τοπικό δίκτυο για να έχει νόημα: από μόνο του δεν διαθέτει υλικό, δεν επικοινωνεί απευθείας με οποιαδήποτε συσκευή και τα οντολογικά αντικείμενα που εκθέτει (πλήκτρα, αισθητήρες, κ.ά.), προκύπτουν αποκλειστικά από τα δεδομένα που του στέλνει η «γέφυρα» μέσω του MQTT broker.



Εικόνα 3.2: Μέρος της κεντρικής σελίδας του integration στο Home Assistant, μετά την εγκατάσταση.

Συμμόρφωση με την επίσημη δομή. Το custom component ακολουθεί την τυποποιημένη δομή που ορίζει το Home Assistant για όλα τα integrations [3]. Η αυστηρή αυτή δομή είναι απαραίτητη, ώστε ο πυρήνας του Home Assistant να φορτώσει σωστά το integration, να εντοπίσει τους τύπους οντοτήτων που εκθέτει, και να διαχειριστεί τον κύκλο ζωής του (φόρτωση, εκφόρτωση, παραμετροποίηση). Όλα τα αρχεία βρίσκονται σε έναν κατάλογο, με όνομα ίδιο με το domain id του integration (ir_remote), μέσα στο /config/custom_components/ της εγκατάστασης του Home Assistant.

Αρχεία και ρόλοι. Η συνολική οργάνωση του integration έχει ως εξής:

- manifest.json: τα μεταδεδομένα του integration (όνομα, έκδοση, εξαρτήσεις, πληροφορίες HACS), όπως αναλύθηκαν στις **ενότητες 2.6.Γ** και θα ξανασυζητηθούν στη ροή έκδοσης του **3.6**.

- __init__.py: το σημείο εισόδου του integration. Υλοποιεί τα async_setup, async_setup_entry και async_unload_entry — τα υποχρεωτικά hooks που καλεί ο πυρήνας του Home Assistant κατά τη φόρτωση, την προσθήκη νέου config entry και την αφαίρεσή του αντίστοιχα.

- config_flow.py: η ροή διαμόρφωσης που εκτίθεται στο γραφικό περιβάλλον όταν ο χρήστης προσθέτει το integration. Αναλύεται διεξοδικά στην **ενότητα 3.4.B**.

- const.py: σταθερές μοιραζόμενες ανάμεσα στα modules — κατά κανόνα μόνο το DOMAIN id και κάποια κλειδιά διαμόρφωσης.

- button.py, sensor.py, number.py, text.py: ένα Python module ανά τύπο οντότητας. Κάθε ένα ορίζει το ασύγχρονο async_setup_entry του και τις κλάσεις οντοτήτων που εκθέτει στον χρήστη. Η αναλυτική παρουσίαση γίνεται στην **ενότητα 3.4.Γ**.

- services.yaml: δηλώνει τις υπηρεσίες (services/actions) που μπορούν να κληθούν από τα Developer Tools του Home Assistant ή από αυτοματισμούς. Συγκεκριμένα, ορίζει τα ir_remote.record_key, ir_remote.delete_key και ir_remote.rename_key.

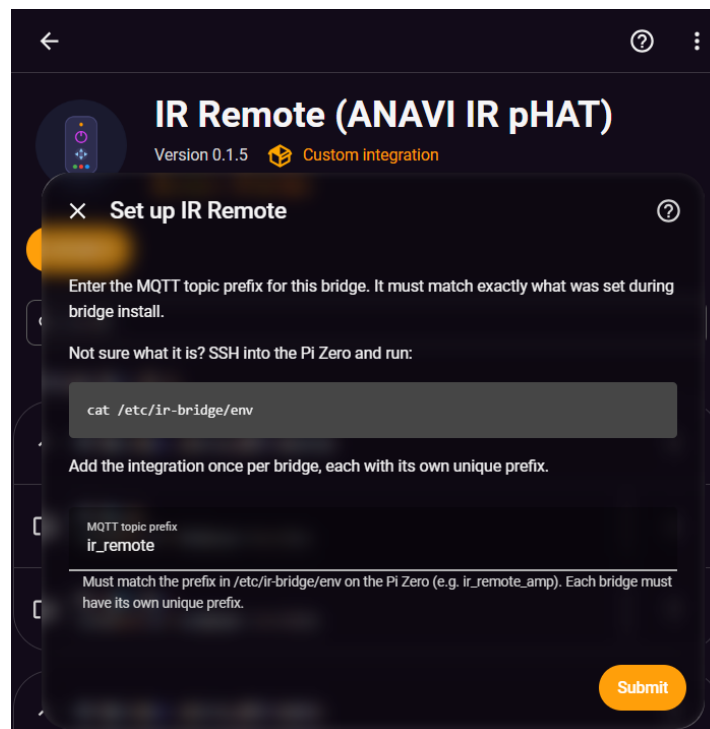
- translations/: αρχεία JSON με τις μεταφράσεις των ονομάτων των οντοτήτων και των μηνυμάτων του config flow.

- brand/: λογότυπο και εικονίδιο του integration, όπως απαιτούνται από το brand repository του Home Assistant, για την προβολή στη βιβλιοθήκη integrations.

Εξαρτήσεις. Το integration δεν χρειάζεται εξωτερικές Python βιβλιοθήκες (το πεδίο requirements στο manifest.json είναι κενό), καθώς όλη η MQTT επικοινωνία υλοποιείται μέσω της επίσημης ενσωμάτωσης MQTT του Home Assistant — που δηλώνεται ως υποχρεωτική εξάρτηση στο πεδίο dependencies του manifest. Αυτή η σχεδιαστική επιλογή εξασφαλίζει ότι το integration μένει ελαφρύ, χωρίς να επαναφέρει δικό του MQTT client και αξιοποιεί την υπάρχουσα διαμόρφωση του χρήστη για τον broker.

Στις σύγχρονες εκδόσεις, το Home Assistant ενθαρρύνει την UI-based διαμόρφωση των integrations, μέσω του μηχανισμού των config flows, αντί για χειροκίνητη επεξεργασία αρχείων YAML. Ο μηχανισμός αυτός εμφανίζει στον χρήστη μια φόρμα όταν προσθέτει το integration από το Settings → Devices & Services και αποθηκεύει τις απαντήσεις σε μια οντότητα που ονομάζεται config entry. Στο integration της παρούσας εργασίας, η ροή διαμόρφωσης υλοποιείται στο αρχείο config_flow.py και αποτελείται από ένα μόνο βήμα.

Υλοποίηση. Η μοναδική κλάση που υλοποιείται είναι η IRRemoteConfigFlow, που κληρονομεί από την βάση config_entries.ConfigFlow και δηλώνει το domain id ως "ir_remote". Το ενεργό βήμα είναι το async_step_user, το οποίο καλείται όταν ο χρήστης ξεκινήσει τη ροή προσθήκης από το UI. Η φόρμα που εμφανίζεται έχει ένα μόνο πεδίο εισόδου: το MQTT topic prefix, με προεπιλεγμένη τιμή το "ir_remote". Το πεδίο αυτό αντιστοιχεί στο topic prefix, με το οποίο επικοινωνεί η εκάστοτε «γέφυρα» (όπως αναλύεται στο 3.5.A) και είναι ο μόνος παράγοντας που πρέπει να γνωρίζει το integration για να επικοινωνήσει με τη σωστή «γέφυρα».



The screenshot shows the 'IR Remote (ANAVI IR pHAT)' integration setup screen. At the top, it says 'Version 0.1.5' and 'Custom integration'. Below this is a dialog box titled 'Set up IR Remote'. The dialog contains the following text: 'Enter the MQTT topic prefix for this bridge. It must match exactly what was set during bridge install.' followed by 'Not sure what it is? SSH into the Pi Zero and run:' and a code block containing 'cat /etc/ir-bridge/env'. Below this, it says 'Add the integration once per bridge, each with its own unique prefix.' There is an input field for 'MQTT topic prefix' with the value 'ir_remote' entered. Below the input field, it says 'Must match the prefix in /etc/ir-bridge/env on the Pi Zero (e.g. ir_remote_amp). Each bridge must have its own unique prefix.' At the bottom right of the dialog is a yellow 'Submit' button.

Εικόνα 3.3: Φόρμα διαμόρφωσης (config flow) του integration.

Έλεγχοι εγκυρότητας. Πριν την οριστική δημιουργία του config entry, η ροή εκτελεί δύο διαδοχικούς ελέγχους. Πρώτον, επιβεβαιώνει ότι το MQTT integration του Home Assistant είναι ήδη εγκατεστημένο και διαμορφωμένο· αν όχι, εμφανίζει στον χρήστη το μήνυμα σφάλματος "mqtt_not_configured", υποδεικνύοντας ότι πρέπει πρώτα να ρυθμιστεί ο MQTT broker. Δεύτερον, εκτελεί έναν λειτουργικό έλεγχο της «γέφυρας»: εγγράφεται στο topic "{prefix}/devices" και αναμένει για 5 δευτερόλεπτα ένα retained μήνυμα που δημοσιεύει η «γέφυρα» κατά την εκκίνησή της. Αν δεν ληφθεί μήνυμα εντός του χρονικού ορίου, εμφανίζεται το σφάλμα

"ri_offline", υποδηλώνοντας ότι είτε η «γέφυρα» δεν εκτελείται, είτε ο topic prefix έχει δηλωθεί λανθασμένα, είτε υπάρχει πρόβλημα δικτυακής συνδεσιμότητας με τον broker.

Η διπλή αυτή επικύρωση είναι σημαντική για την εμπειρία του χρήστη: αν παρακαμπτώταν, το integration θα δημιουργούσε τις οντότητές του με λανθασμένη επικοινωνία και θα εμφάνιζε σφάλματα κατά τη χρήση. Ο έλεγχος "ri_offline" κατά τη διαμόρφωση, εντοπίζει το πρόβλημα στο σημείο που ο χρήστης μπορεί ακόμη να το διορθώσει, χωρίς να έχει επενδύσει χρόνο σε ρυθμίσεις.

Μοναδικότητα και πολλαπλές «γέφυρες». Αν η επικύρωση επιτύχει, η ροή ορίζει ως `unique_id` του `config entry` το `topic prefix` και καλεί την `_abort_if_unique_id_configured()`, το οποίο αποτρέπει την προσθήκη δεύτερου `config entry` για την ίδια «γέφυρα». Αντίθετα, διαφορετικές «γέφυρες» — με διαφορετικό `topic prefix` — μπορούν να προστεθούν παράλληλα ως ξεχωριστά `config entries`, καθένα με τις δικές του οντότητες στο Home Assistant. Έτσι, η αρχιτεκτονική πολλαπλών «γεφυρών» που περιγράφηκε στην **ενότητα 3.2**, υποστηρίζεται απευθείας από τον μηχανισμό του `config flow`, χωρίς ειδικό κώδικα διαχείρισης.

Δημιουργία του `entry`. Με την επιτυχή καταχώρηση, η κλήση `async_create_entry` καταχωρεί το νέο `entry`, με τίτλο της μορφής "IR Remote (XXX)", όπου "XXX" είναι το πραγματικό `topic prefix` που έδωσε ο χρήστης και αποθηκεύει το `topic prefix` στα `data` του. Από αυτό το σημείο και μετά, ο πυρήνας του Home Assistant καλεί το `async_setup_entry` του `integration` (στο `__init__.py`), το οποίο ξεκινά τη φόρτωση των `entity platforms` `button`, `sensor`, `number` και `text` — η διαδικασία που αναλύεται στις επόμενες δύο ενότητες.

Οντότητες `button`, `sensor`, `number`, `text`

3.4.Γ

Το `integration` εκθέτει τη λειτουργικότητα του συστήματος στον χρήστη, μέσω τεσσάρων διαφορετικών τύπων οντοτήτων του Home Assistant: `button`, `sensor`, `number` και `text`. Καθένας από αυτούς τους τύπους υλοποιείται σε ξεχωριστό Python `module` και αντιστοιχεί σε διαφορετικό ρόλο στη διεπαφή — από κουμπιά ενεργειών, μέχρι πεδία εισαγωγής κειμένου και αισθητήρες κατάστασης. Συνολικά δημιουργούνται περίπου είκοσι κλάσεις οντοτήτων, αρθρωτά οργανωμένες σε δύο λογικά επίπεδα: αυτές που αφορούν τη «γέφυρα» στο σύνολό της και αυτές που αφορούν μία συγκεκριμένη συσκευή ή πλήκτρο.

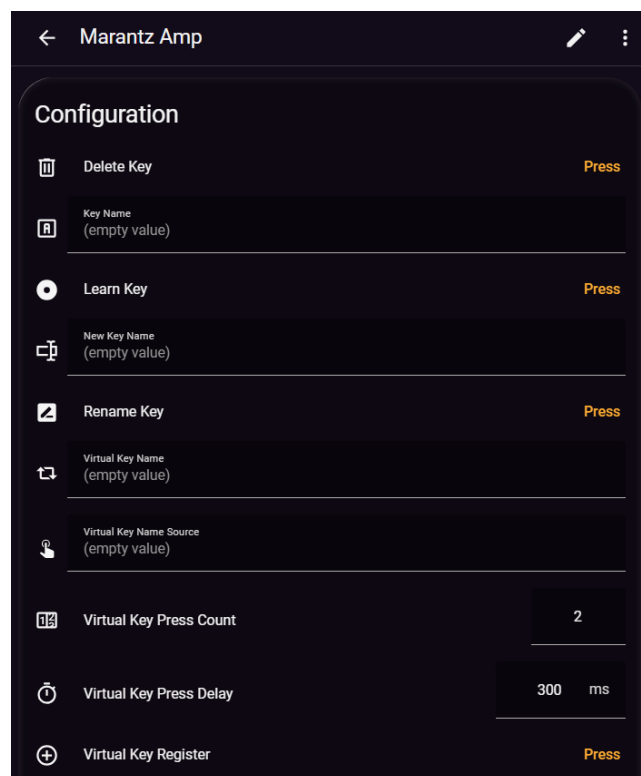
Οντότητες `button` (`button.py`). Το πιο πλούσιο σε λειτουργικότητα `module` είναι το `button.py`, με εννέα διακριτές κλάσεις. Στο επίπεδο της «γέφυρας» υπάρχουν τέσσερις: η `ReloadButton` που πυροδοτεί σάρωση, εκείνη τη στιγμή, των `device` αρχείων από τη «γέφυρα», η `CreateDeviceButton` για τη δημιουργία νέου `remote device`, η `DeleteDeviceButton` για τη διαγραφή υπάρχοντος και η `RenameDeviceButton` για τη μετονομασία του. Σε επίπεδο συσκευής υπάρχουν τέσσερις ακόμη: η `LearnButton` που ξεκινά τη διαδικασία καταγραφής νέου πλήκτρου (**3.3.Γ**), η `DeleteButton` για τη διαγραφή ενός πλήκτρου, η `RenameButton` για τη μετονομασία του και η `RegisterMultiPressButton` για τη δημιουργία νέου εικονικού πλήκτρου (**3.3.Ε**). Τέλος, μία κλάση `IRRemoteButton`

χρησιμοποιείται για κάθε καταγεγραμμένο πραγματικό ή εικονικό πλήκτρο IR, με μια οντότητα ανά πλήκτρο: όταν ο χρήστης την πατά (press), το integration δημοσιεύει στο αντίστοιχο MQTT topic εντολή αναπαραγωγής.

Οντότητα sensor (sensor.py). Σε αυτό το module υλοποιείται μία μόνο κλάση, η RecordingStatusSensor, η οποία αναπαριστά την τρέχουσα κατάσταση της μηχανής καταγραφής της «γέφυρας» που περιγράφηκε στο **3.3.Γ** (idle, recording, done, error, timeout). Δημιουργείται μία τέτοια οντότητα, ανά συσκευή και ενημερώνεται σε πραγματικό χρόνο, μέσω των retained μηνυμάτων κατάστασης, που δημοσιεύει η «γέφυρα».

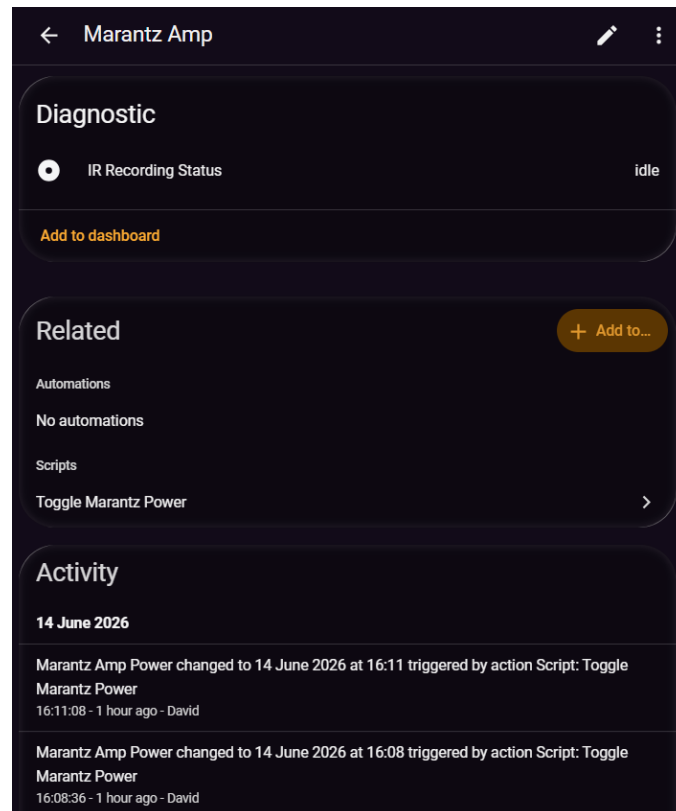
Οντότητες number (number.py). Δύο κλάσεις: η MultiPressCountNumber για το πλήθος επαναλήψεων του επόμενου εικονικού πλήκτρου που θα δημιουργηθεί και η MultiPressDelayNumber, για την καθυστέρηση μεταξύ επαναλήψεων σε χιλιοστά του δευτερολέπτου. Δημιουργούνται ανά συσκευή και επιτρέπουν στον χρήστη να ορίσει αριθμητικά τις παραμέτρους του εικονικού πλήκτρου πριν πατήσει το RegisterMultiPressButton.

Οντότητες text (text.py). Έξι κλάσεις πεδίων κειμένου, μοιρασμένες ανάμεσα σε bridge-level και per-device. Στο bridge level: η NewDeviceNameText για το όνομα της νέας συσκευής προς δημιουργία και η RenameTargetDeviceText για το όνομα της συσκευής προς μετονομασία. Στο per-device level: η KeyNameText για το όνομα ενός πλήκτρου (χρησιμοποιείται από τις LearnButton, DeleteButton, RenameButton), η RenameTargetText για το νέο όνομα κατά τη μετονομασία πλήκτρου, η MultiPressNameText για το όνομα του προς δημιουργία εικονικού πλήκτρου και η MultiPressSourceText για το όνομα του αρχικού πλήκτρου από το οποίο θα παραχθεί.



Εικόνα 3.4: Η σελίδα της συσκευής και οι δυνατότητές της.

Συμβάσεις ονοματολογίας. Όλες οι οντότητες ακολουθούν τις συμβάσεις του Home Assistant για ονόματα και entity ids. Το όνομα της συσκευής μετατρέπεται από snake_case (όπως αποθηκεύεται στη «γέφυρα», π.χ. samsung_tv) σε Title Case για την εμφάνιση στη διεπαφή (Samsung Tv), ενώ τα ονόματα οντοτήτων ακολουθούν το μοτίβο "{Device Title} {Entity Purpose}" — για παράδειγμα, "Samsung Tv IR Recording Status" ή "Samsung Tv Virtual Key Press Count". Έτσι, ο χρήστης βλέπει ομαδοποιημένες όλες τις οντότητες της κάθε συσκευής στη διεπαφή του Home Assistant.



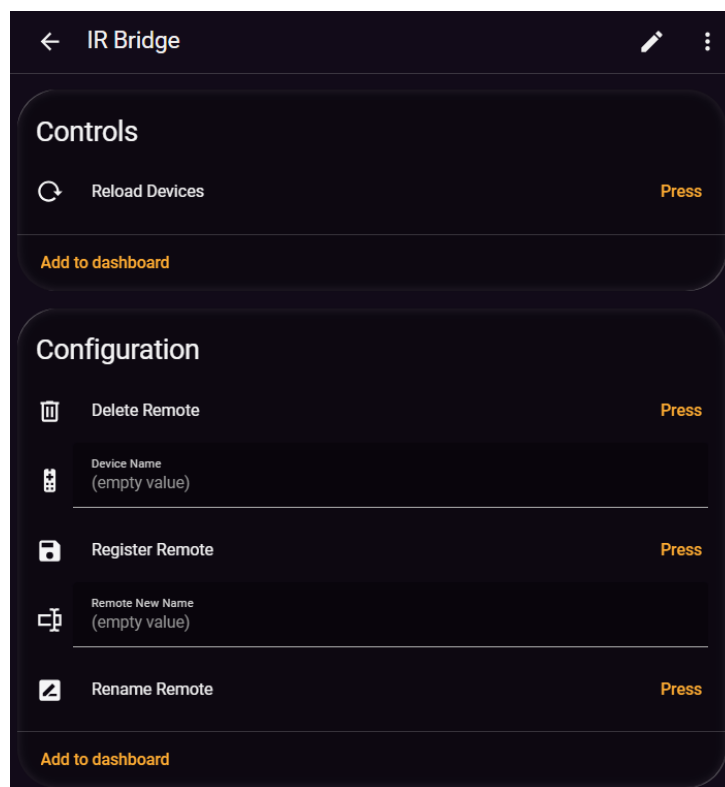
Εικόνα 3.5: Άλλες ενότητες της συσκευής

Δυναμική δημιουργία και ενημέρωση οντοτήτων

3.4.Δ

Ένα από τα κρίσιμα σχεδιαστικά χαρακτηριστικά του integration, είναι ότι ο τελικός χρήστης δεν χρειάζεται ποτέ να δηλώσει στατικά ποιες συσκευές ή πλήκτρα διαθέτει. Όταν η «γέφυρα» μάθει ένα νέο IR σήμα, η αντίστοιχη οντότητα button εμφανίζεται αυτόματα στο Home Assistant. Όταν διαγραφεί ένα πλήκτρο ή ολόκληρη συσκευή, η αντίστοιχη οντότητα εξαφανίζεται. Όλα αυτά γίνονται live, χωρίς επανεκκίνηση του Home Assistant. Στην ενότητα αυτή περιγράφεται ο μηχανισμός που το καθιστά αυτό δυνατό.

Βασική ιδέα. Κατά τη φόρτωσή του (μέσω `async_setup_entry` στο `init.py` και των ομώνυμων συναρτήσεων σε κάθε platform module), το integration δημιουργεί άμεσα τις σταθερές οντότητες σε επίπεδο «γέφυρας» (Reload Devices, Create/Delete/Rename Device, καθώς και τα αντίστοιχα text πεδία).



Εικόνα 3.6: Η σελίδα της «γέφυρας» (IR Bridge), από όπου ο χρήστης καταχωρεί (Register Remote), διαγράφει ή μετονομάζει συσκευές· το “Reload Devices” ανανεώνει τα αρχεία κάθε συσκευής.

Παράλληλα, εγγράφεται στο MQTT topic "{prefix}/devices", το οποίο η «γέφυρα» δημοσιεύει ως retained μήνυμα με τη λίστα όλων των συσκευών της και τα πλήκτρα τους σε μορφή JSON. Επειδή το μήνυμα είναι retained, η νέα εγγραφή λαμβάνει αμέσως την πιο πρόσφατη κατάσταση, χωρίς να χρειάζεται να περιμένει την επόμενη δημοσίευση.

Δομή του payload. Το payload του "{prefix}/devices" είναι ένα JSON object της μορφής {"samsung_tv": ["power", "volume_up", ...], "living_room_ac": [...], ...}, όπου τα κλειδιά είναι ονόματα συσκευών και οι τιμές είναι λίστες ονομάτων πλήκτρων (συμπεριλαμβανομένων των εικονικών). Έτσι, με ένα μόνο μήνυμα, το integration γνωρίζει την πλήρη κατάσταση του συστήματος.

Διαδικασία ενημέρωσης. Κάθε φορά που λαμβάνεται μήνυμα στο "{prefix}/devices", εκτελείται μια ασύγχρονη συνάρτηση callback (η handle_devices στο button.py, με αντίστοιχες στο sensor.py και το number.py). Η συνάρτηση αυτή ακολουθεί τρία βήματα. Πρώτον, υπολογίζει το σύνολο των unique_ids που πρέπει να υπάρχουν με βάση το payload — ένα για κάθε bridge-level οντότητα, τέσσερα ανά συσκευή (Learn, Delete, Rename, Register Virtual Key) και ένα ανά μαθημένο πλήκτρο. Δεύτερον, σαρώνει το entity registry του Home Assistant, για όλες τις υπάρχουσες οντότητες του config entry και αφαιρεί όσες δεν περιλαμβάνονται στο σύνολο των έγκυρων unique_ids — αυτές είναι ορφανές οντότητες που αντιστοιχούν σε συσκευές ή πλήκτρα που έχουν, εν τω μεταξύ, διαγραφεί στη «γέφυρα». Με τον ίδιο τρόπο καθαρίζεται και το device registry. Τρίτον, για κάθε συσκευή και πλήκτρο που υπάρχει στο payload, αλλά δεν έχει ακόμα αντίστοιχη οντότητα στο Home Assistant, δημιουργείται νέα και εγγράφεται μέσω της async_add_entities.

Παρακολούθηση κατάστασης σε ένα set. Για να αποφευχθεί η διπλή δημιουργία οντότητας που έχει ήδη προστεθεί, το callback διατηρεί ένα τοπικό σύνολο added (Python set) που καταγράφει ποια unique_ids έχουν ήδη παραχθεί κατά τη διάρκεια του session. Έτσι, ακόμη και αν η «γέφυρα» επαναδημοσιεύσει το ίδιο payload (π.χ. μετά από reload), το integration δεν διπλασιάζει τις οντότητες — απλώς αναγνωρίζει ότι όλες είναι ήδη παρούσες.

Ζωντανή ενημέρωση χωρίς επανεκκίνηση. Ο συνδυασμός των παραπάνω σημαίνει ότι κάθε νέο πλήκτρο που μαθαίνει η «γέφυρα» μέσω της διαδικασίας του 3.3.Γ εμφανίζεται αυτόματα στο dashboard του χρήστη μέσα σε δευτερόλεπτα — χωρίς να επανεκκινηθεί το Home Assistant, χωρίς να εγκατασταθεί από την αρχή το integration και χωρίς να γίνει οποιαδήποτε χειροκίνητη ενέργεια διαμόρφωσης. Η ίδια ροή ακολουθείται και για διαγραφές: εξαφανίζονται οι αντίστοιχες οντότητες χωρίς εκκρεμότητες στο entity registry.

Πρωτόκολλο επικοινωνίας MQTT

3.5

Δομή topic και πρόθεμα για πολλαπλές «γέφυρες»

3.5.A

Όλα τα MQTT topics του συστήματος ξεκινούν με ένα κοινό πρόθεμα (prefix), που ορίζεται κατά τη ροή διαμόρφωσης (3.4.B), τόσο στη «γέφυρα», όσο και στο integration. Το πρόθεμα εξυπηρετεί δύο σκοπούς: αποτρέπει συγκρούσεις με άλλες εφαρμογές που χρησιμοποιούν τον ίδιο broker και επιτρέπει την παράλληλη λειτουργία πολλαπλών «γεφυρών» στο ίδιο σπίτι, καθεμιάς με δικό της απομονωμένο namespace.

Προεπιλογή και προσαρμογή. Η προεπιλεγμένη τιμή του προθέματος είναι "ir_remote", ο ίδιος όρος που χρησιμοποιείται και ως domain id του integration. Αυτή η επιλογή ταιριάζει σε εγκαταστάσεις με μία μόνο «γέφυρα» και δεν απαιτεί επιπλέον ρυθμίσεις από τον χρήστη. Όταν υπάρχουν περισσότερες «γέφυρες» — π.χ. μία ανά δωμάτιο — κάθε «γέφυρα» διαμορφώνεται με δικό της μοναδικό πρόθεμα (όπως "ir_remote_living_room", "ir_remote_bedroom", "ir_remote_office") μέσω του αρχείου ρυθμίσεων /etc/ir-bridge/env και προστίθεται μία ξεχωριστή εγκατάσταση του integration στο Home Assistant για καθεμία.

Ιεραρχική δομή των topics. Κάτω από το πρόθεμα οργανώνονται ιεραρχικά οι παρακάτω κατηγορίες topics:

- "{prefix}/devices" — retained μήνυμα με τη λίστα όλων των γνωστών συσκευών και των πλήκτρων τους, σε JSON μορφή. Η «γέφυρα» το δημοσιεύει σε κάθε αλλαγή του store · το integration εγγράφεται για να ανακαλύπτει συσκευές και να δημιουργεί δυναμικά οντότητες (3.4.Δ).
- "{prefix}/record/start" — εντολή έναρξης καταγραφής νέου πλήκτρου, με payload τη συσκευή και το όνομα πλήκτρου.
- "{prefix}/record/status" — retained μήνυμα με την τρέχουσα κατάσταση του δέκτη (idle, recording, done, error, timeout), όπως αναλύθηκε στην **ενότητα 3.3.Γ**.

- "{prefix}/reload" — εντολή για επιτόπου σάρωση των device αρχείων από τη «γέφυρα».
- "{prefix}/device/create", "{prefix}/device/delete", "{prefix}/device/rename" — εντολές διαχείρισης των remote devices.
- "{prefix}/key/delete", "{prefix}/key/rename" — εντολές διαχείρισης μεμονωμένων πλήκτρων.
- "{prefix}/virtual_key/create", "{prefix}/virtual_key/delete" — εντολές διαχείρισης εικονικών πλήκτρων.
- "{prefix}/{device}/send" — εντολή αναπαραγωγής συγκεκριμένου πλήκτρου μιας συσκευής. Το {device} αντικαθίσταται με το όνομα της εκάστοτε συσκευής.
- "{prefix}/system/restart_needed" — ενημερωτικό μήνυμα από τη «γέφυρα» όταν εντοπίσει κατάσταση που απαιτεί επανεκκίνηση (π.χ. αλλαγή του topic prefix στις ρυθμίσεις της).

Λογική απομόνωση πολλαπλών «γεφυρών». Επειδή κάθε «γέφυρα» δημοσιεύει και εγγράφεται αποκλειστικά κάτω από το δικό της πρόθεμα, δεν υπάρχει παρεμβολή ή κίνδυνος ότι ένα μήνυμα που προορίζεται για μία «γέφυρα» θα ληφθεί κατά λάθος από άλλη. Αυτή η απομόνωση είναι φυσική συνέπεια του μοντέλου publish/subscribe και της ιεραρχικής δομής των topics (**2.4.B**) και δεν απαιτεί ειδικό κώδικα — αρκεί η σωστή δήλωση του προθέματος σε κάθε εγκατάσταση. Στην πλευρά του Home Assistant, η κάθε εγκατάσταση του integration κρατά το δικό της topic prefix στο config entry της και χρησιμοποιεί αυτό για όλες τις publish/subscribe ενέργειες, εξασφαλίζοντας ότι ο διαχωρισμός διατηρείται και στις δύο πλευρές της επικοινωνίας.

Μηνύματα εντολής και ενημέρωσης κατάστασης

3.5.B

Τα MQTT μηνύματα που ανταλλάσσουν η «γέφυρα» και το Home Assistant χωρίζονται σε δύο διακριτές κατηγορίες. Τα μηνύματα εντολής (Home Assistant → «γέφυρα») πυροδοτούν μια ενέργεια και δεν διατηρούνται στον broker μετά την παράδοσή τους. Τα μηνύματα ενημέρωσης κατάστασης («γέφυρα» → Home Assistant) αναπαριστούν την τρέχουσα κατάσταση του συστήματος και δημοσιεύονται ως retained, ώστε κάθε νέος subscriber να λαμβάνει αμέσως την πλέον πρόσφατη τιμή χωρίς να χρειάζεται να περιμένει την επόμενη δημοσίευση (όπως αναλύθηκε στην **ενότητα 2.4.Γ**).

Μηνύματα εντολής. Οι εντολές που στέλνει το Home Assistant προς τη «γέφυρα», μπορούν να ομαδοποιηθούν θεματικά:

- Διαχείριση remote devices: το "{prefix}/device/create" δημιουργεί νέο remote device με payload JSON της μορφής {"name": "samsung_tv"}, το "{prefix}/device/delete" το αφαιρεί, και το "{prefix}/device/rename" το μετονομάζει με payload {"old": "...", "new": "..."}.
- Διαχείριση καταγεγραμμένων πλήκτρων: το "{prefix}/record/start" πυροδοτεί τη ροή καταγραφής (**3.3.Γ**) με payload {"device": "samsung_tv", "key": "power"}, το

"{prefix}/key/delete" διαγράφει υπάρχον πλήκτρο, το "{prefix}/key/rename" το μετονομάζει με payload που περιλαμβάνει το παλιό και το νέο όνομα.

- Διαχείριση εικονικών πλήκτρων: το "{prefix}/virtual_key/create" δημιουργεί νέο εικονικό πλήκτρο (**3.3.E**) με payload που περιλαμβάνει όνομα, πηγή, repeat και delay_ms, ενώ το "{prefix}/virtual_key/delete" το αφαιρεί.

- Αναπαραγωγή IR σήματος: το "{prefix}/{device}/send" — όπου το {device} αντικαθίσταται με το όνομα της αντίστοιχης συσκευής — στέλνει εντολή αναπαραγωγής, με payload το όνομα του πλήκτρου (π.χ. "power").

- Συστημικές εντολές: το "{prefix}/reload" επαναφορτώνει τα device αρχεία από τον δίσκο, με ένα ελάχιστο payload (π.χ. "1").

Μηνύματα ενημέρωσης κατάστασης. Σε αντίθετη κατεύθυνση, η «γέφυρα» δημοσιεύει τρία βασικά μηνύματα κατάστασης. Πρώτο και κύριο, το "{prefix}/devices" είναι ένα retained JSON μήνυμα που αναπαριστά τη συνολική κατάσταση του store: ένα αντικείμενο όπου τα κλειδιά είναι ονόματα συσκευών και οι τιμές είναι λίστες ονομάτων πλήκτρων (μαθημένων και εικονικών). Δεύτερο, το "{prefix}/record/status" είναι ένα retained μήνυμα με απλό payload συμβολοσειράς που αντιπροσωπεύει την τρέχουσα κατάσταση της μηχανής καταγραφής (idle, recording, done, error, timeout). Τρίτο, το "{prefix}/system/restart_needed" είναι ένα event μήνυμα που δημοσιεύεται όταν η «γέφυρα» αλλάζει ρυθμίσεις που απαιτούν επανεκκίνηση (π.χ. μεταβολή του topic prefix) και ενεργοποιεί στο Home Assistant ένα warning issue, που εμφανίζεται στον χρήστη μέσω της καρτέλας Repairs.

Μορφή payload. Ως μορφή δεδομένων επιλέγεται κατά κανόνα το JSON για όσα μηνύματα μεταφέρουν δομημένα δεδομένα (πολλά πεδία ή λίστες). Για απλές εντολές με μηδενικό ή μονοδιάστατο περιεχόμενο, χρησιμοποιείται απλή συμβολοσειρά (π.χ. το payload "1" του reload ή το όνομα κλειδιού στο {device}/send). Η επιλογή αυτή ισορροπεί μεταξύ συνέπειας (παντού JSON) και μικρού μεγέθους (μηνύματα ελάχιστου μεγέθους όπου δεν χρειάζεται δομή).

Συνδυασμός QoS και retained. Όπως αναπτύχθηκε στην **ενότητα 2.4.Δ**, η αρχιτεκτονική του συστήματος αξιοποιεί το χαρακτηριστικό retained αποκλειστικά, για τα μηνύματα κατάστασης. Έτσι, ένα νέο instance του integration ή μετά από επανεκκίνηση του Home Assistant, λαμβάνει αυτόματα την πιο πρόσφατη λίστα συσκευών και την τρέχουσα κατάσταση καταγραφής — χωρίς να χρειάζεται να ζητήσει ενεργά ενημέρωση από τη «γέφυρα» μέσω explicit query. Τα μηνύματα εντολής αντίθετα, δεν διατηρούνται: μια εντολή έχει νόημα μόνο τη στιγμή της αποστολής της.

Αυτοματοποιημένη δημοσίευση εκδόσεων

3.6

CI/CD Pipeline στα GitHub Actions

3.6.A

Η διαδικασία δημοσίευσης νέων εκδόσεων του integration είναι πλήρως αυτοματοποιημένη μέσω ενός CI/CD pipeline, υλοποιημένου στα GitHub Actions

[32]. Το pipeline εξασφαλίζει ότι κάθε commit στο main branch ελέγχεται για συμβατότητα και ποιότητα προτού οδηγήσει σε νέα δημοσιοποιημένη έκδοση. Αποτελείται από τέσσερα διακριτά workflows, τα οποία ορίζονται σε αντιστοιχα YAML αρχεία μέσα στον κατάλογο `.github/workflows/` του repository [33].

Workflow `deploy.yml`. Το πρώτο workflow, το `deploy.yml`, εκτελείται σε self-hosted runner στην συσκευή του χρήστη — την ίδια που φιλοξενεί την Home Assistant Docker Container έκδοση.

Στην αρχική σχεδίαση, σε κάθε push στο main, το workflow εκτελούσε `git pull` στο τοπικό checkout του repository και κατόπιν επανεκκίνηση του Docker Container του Home Assistant. Σκοπός του ήταν να επιταχύνει τον κύκλο Development και Testing: ο Developer μπορούσε, έτσι, να κάνει push μια αλλαγή και να δει αμέσως το αποτέλεσμα στο Home Assistant, χωρίς να χρειάζεται κάθε φορά να συνδέεται μέσω SSH και να εκτελεί χειροκίνητα τα παραπάνω βήματα.

Στην παρούσα κατάσταση, το integration έχει εγγραφεί ως custom repository στο HACS του τρέχοντος Home Assistant και η διανομή των νέων εκδόσεων γίνεται μέσω αυτού. Έτσι, το `deploy.yml` απλοποιήθηκε και εκτελεί πλέον μόνο `git pull`, καθώς η φόρτωση του νέου κώδικα του integration στο Home Assistant πραγματοποιείται φυσικά από το HACS — είτε χειροκίνητα μέσω "Update information" στη διεπαφή του, είτε αυτόματα μέσω αυτοματισμού του Home Assistant που πυροδοτεί την ενημέρωση σε τακτά διαστήματα.

Workflow `hacs.yml`. Το δεύτερο workflow εκτελεί τον επίσημο HACS Action [25] σε `ubuntu-latest` runner του GitHub, που ελέγχει αν το integration συμμορφώνεται με τις προδιαγραφές του HACS — δομή φακέλων, ύπαρξη `manifest.json`, ονοματολογία, εικονίδια, κ.ά. Εκτελείται σε κάθε push και pull request, καθώς και ως νυχτερινό cron job (σε στις 0:00 UTC κάθε ημέρας), εξασφαλίζοντας ότι ακόμα και αλλαγές στις προδιαγραφές του HACS εντοπίζονται γρήγορα.

Workflow `hassfest.yml`. Το τρίτο workflow εκτελεί το Hassfest [24] — το επίσημο εργαλείο επικύρωσης `manifest.json` του Home Assistant — με την ίδια συχνότητα και τους ίδιους triggers όπως και το `hacs.yml`. Ο διπλός έλεγχος (HACS Action και Hassfest παράλληλα) είναι απαραίτητη προϋπόθεση για την ένταξη στο HACS default store.

Workflow `release.yml`. Το πιο σύνθετο workflow αυτοματοποιεί τη δημιουργία νέας έκδοσης. Πυροδοτείται μέσω `workflow_run`, όταν ολοκληρωθεί επιτυχώς το Hassfest workflow στον main, και εκτελεί τα ακόλουθα βήματα:

- (1) Επιβεβαιώνει ότι και το HACS Action έχει επίσης επιτυχημένη ολοκλήρωση για το ίδιο commit, κάνοντας polling στο GitHub API έως 10 λεπτά. Αν το HACS Action αποτύχει ή δεν ολοκληρωθεί, το release ακυρώνεται.

- (2) Υπολογίζει την επόμενη έκδοση συγκρίνοντας το τελευταίο `git tag` με την έκδοση που δηλώνεται στο `manifest.json`: επιλέγεται η μεγαλύτερη μεταξύ (α) `auto-incremented patch` (π.χ. `v0.1.3` → `v0.1.4`) και (β) όποιας έκδοσης δηλώνει το `manifest`. Έτσι, ο συγγραφέας μπορεί να επιβάλει αναβάθμιση `minor` ή `major version` γράφοντάς την στο `manifest`, αλλιώς το patch αυξάνεται αυτόματα.

- (3) Διαβάζει τις σημειώσεις έκδοσης από το σώμα (body) του commit message — αν δεν υπάρχει body, χρησιμοποιεί το subject του commit ως notes.

(4) Ενημερώνει το πεδίο `version` του `manifest.json` στη νέα τιμή, κάνει `commit` με μήνυμα `"chore: bump version to X [skip ci]"` (το `[skip ci]` αποτρέπει την επανεκκίνηση του pipeline) και `push`.

(5) Δημιουργεί το αντίστοιχο `git tag` και το `push` στο `remote`.

(6) Δημιουργεί GitHub Release με τίτλο το tag και body τις σημειώσεις από το βήμα (3), μέσω της action `softprops/action-gh-release`.

Self-hosted vs δημόσιοι runners. Σημειώνεται ότι το `deploy.yml` χρησιμοποιεί `self-hosted runner` γιατί χρειάζεται πρόσβαση στο φυσικό Pi, ενώ τα υπόλοιπα τρία workflows τρέχουν σε δημόσιους `ubuntu-latest runners` του GitHub, αξιοποιώντας τα δωρεάν `compute minutes` που παρέχονται για δημόσια repositories. Έτσι το pipeline συνδυάζει `cloud-based validation` με `local deployment` χωρίς επιπλέον υποδομή.

Έλεγχοι HACS και Hassfest

3.6.B

Δύο από τα τέσσερα workflows του pipeline — το `hassfest.yml` και το `hacs.yml` — εκτελούν αυτοματοποιημένους ελέγχους εγκυρότητας του integration, έναντι των επίσημων προδιαγραφών του Home Assistant και του HACS αντίστοιχα. Παρότι οι δύο έλεγχοι έχουν παρόμοιο στόχο (επικύρωση ποιότητας), διαφέρουν σαφώς στο αντικείμενο και στο εύρος τους και θεωρούνται συμπληρωματικοί.

Έλεγχοι Hassfest. Το Hassfest [24] είναι το επίσημο εργαλείο επικύρωσης που χρησιμοποιεί η ίδια η ομάδα του Home Assistant, για όλα τα integrations του πυρήνα. Όταν εφαρμόζεται σε `custom integration`, ελέγχει κυρίως τη συμμόρφωσή του με τη δομή και τα μεταδεδομένα που απαιτεί ο πυρήνας του Home Assistant για να φορτώσει σωστά το integration. Συγκεκριμένα, επιβεβαιώνει:

(α) ότι το `manifest.json` υπάρχει και περιέχει όλα τα υποχρεωτικά πεδία (`domain`, `name`, `codeowners`, `documentation`, `issue_tracker`, `iot_class`, `version`).

(β) ότι το `domain id` είναι μοναδικό και δεν συγκρούεται με υπάρχουσες επεκτάσεις του πυρήνα.

(γ) ότι το πεδίο `requirements` αναφέρει έγκυρες Python εξαρτήσεις με σωστά `version pinning`.

(δ) ότι το πεδίο `dependencies` (όπου αναφέρονται άλλα integrations του Home Assistant) είναι έγκυρο.

(ε) ότι τυχόν αρχεία JSON στο φάκελο `translations` έχουν συνεπή δομή και έγκυρα κλειδιά.

(στ) ότι το `services.yaml` ορίζει σωστά τις υπηρεσίες (`services/actions`) που εκθέτει το integration.

Με τον τρόπο αυτό, το Hassfest εξασφαλίζει ότι το integration μπορεί να φορτωθεί χωρίς σφάλμα σε οποιαδήποτε ενεργή εγκατάσταση Home Assistant.

Έλεγχοι HACS Action. Το HACS Action [25] συμπληρώνει το Hassfest εστιάζοντας στις προδιαγραφές του HACS οικοσυστήματος, οι οποίες είναι αυστηρότερες από τις ελάχιστες απαιτήσεις του Home Assistant. Ελέγχει:

(α) τη συνολική δομή του repository — αν τα αρχεία του custom_components/{domain}/ βρίσκονται στη σωστή τοποθεσία.

(β) την ύπαρξη και την ορθότητα του README.md αρχείου με ελάχιστες απαιτούμενες πληροφορίες προς τους χρήστες.

(γ) τη χρήση σωστών GitHub Releases ή tags για τη διαθεσιμότητα των εκδόσεων.

(δ) τη συμβατότητα του integration με τις τρέχουσες υποστηριζόμενες εκδόσεις του Home Assistant.

(ε) την ύπαρξη και την ορθότητα αρχείων branding (λογότυπο, εικονίδιο) στο επίσημο home-assistant/brands repository.

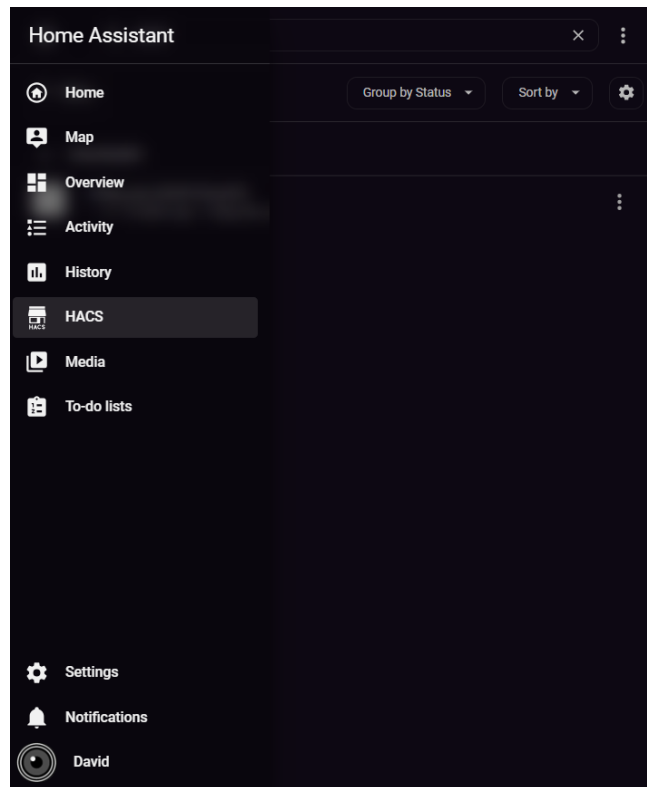
Διπλασιασμός για ασφάλεια. Η ταυτόχρονη εκτέλεση και των δύο workflows σε κάθε push, pull request και νυχτερινό cron, εξασφαλίζει ότι το integration παραμένει συμβατό και με τις δύο πλευρές: τον πυρήνα του Home Assistant και το οικοσύστημα της κοινότητας HACS. Στο release.yml workflow (**3.6.A**), η δημοσίευση νέας έκδοσης γίνεται μόνο αν και τα δύο workflows έχουν επιτυχημένη ολοκλήρωση για το ίδιο commit — διαφορετικά το release ακυρώνεται και ειδοποιείται ο συγγραφέας μέσω mail από το GitHub.

Εμπειρία ανάπτυξης. Στην πράξη, οι έλεγχοι αυτοί έχουν αποδειχθεί καθοριστικοί κατά την υποβολή στο HACS default store (**3.6.Γ**). Η αρχική υποβολή είχε χρειαστεί αρκετές διορθώσεις για να περάσει όλα τα προαναφερόμενα κριτήρια και χωρίς τους αυτοματοποιημένους ελέγχους τα σφάλματα θα εντοπιζόνταν μόνο κατά τη χειροκίνητη επιθεώρηση των ελεγκτών του HACS, καθυστερώντας πολύ τη διαδικασία. Με αυτό το pipeline, κάθε νέο pull request στο hacs/default repository παρουσιάζεται σε «πράσινη» κατάσταση από την πρώτη στιγμή.

[Δημοσίευση εκδόσεων στο Home Assistant Community Store](#)

[3.6.Γ](#)

Το τελευταίο βήμα της ροής δημοσίευσης είναι η διάθεση του integration στους τελικούς χρήστες μέσω του Home Assistant Community Store (HACS) [10]. Όπως αναλύθηκε στην **ενότητα 2.6.Γ**, το HACS υποστηρίζει δύο μηχανισμούς εγκατάστασης: μέσω του ελεγχόμενου default store και μέσω δήλωσης του GitHub repository ως custom repository στο HACS instance του κάθε χρήστη. Στην παρούσα κατάσταση της εργασίας, χρησιμοποιούνται και οι δύο διαδρομές, σε διαφορετικά στάδια ωριμότητας.

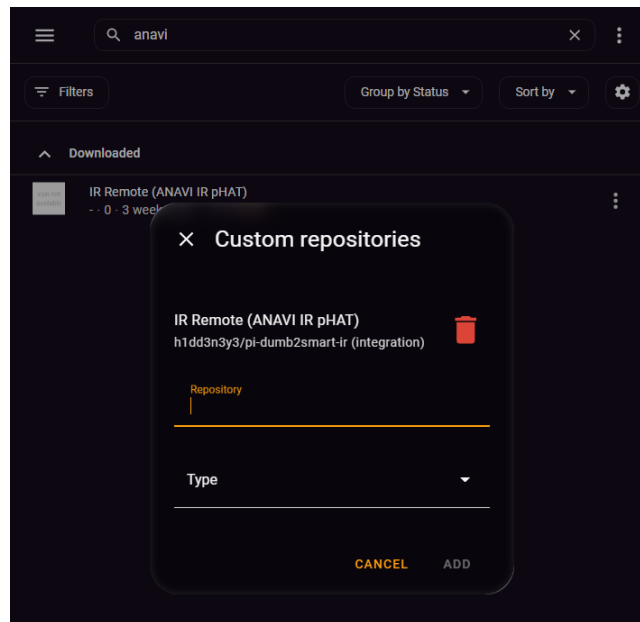


Εικόνα 3.7: Το HACS στο πλαίσιο μενού του HA, από το οποίο γίνεται η εγκατάσταση και ενημέρωση του integration.

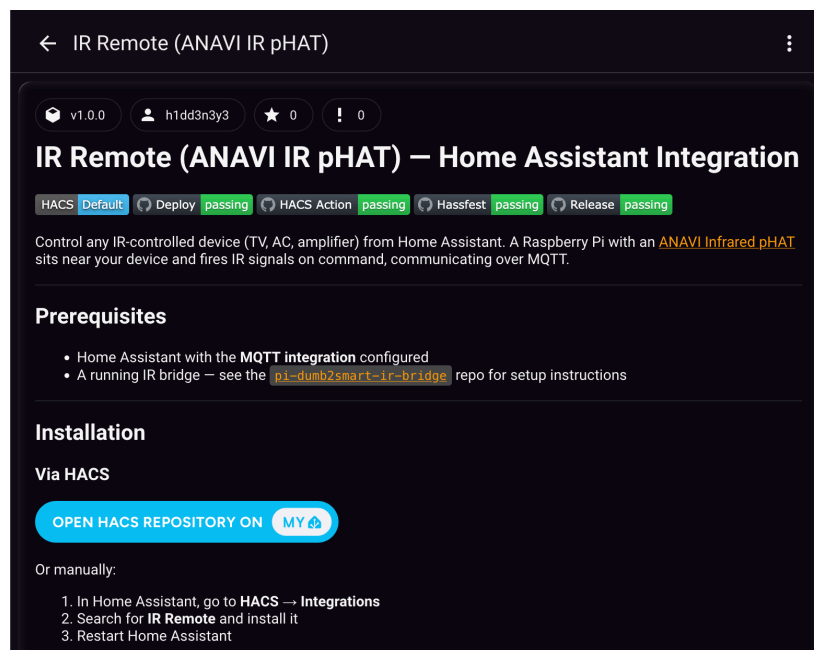
Υποβολή στο default store. Η ένταξη ενός integration στο default store του HACS είναι σχετικά αυστηρή διαδικασία: υποβάλλεται pull request στο δημόσιο repository hacs/default, προσθέτοντας μια εγγραφή για του repository στο επίσημο index. Ένα bot ελέγχει αυτόματα ότι το integration περνά τους ελέγχους που αναπτύχθηκαν στο **3.6.B** (HACS Action και Hassfest) και στη συνέχεια, ένας ελεγκτής της κοινότητας επιθεωρεί το repository χειροκίνητα για ζητήματα ποιότητας, τεκμηρίωσης και κανόνων συμπεριφοράς της κοινότητας.

Στην παρούσα εργασία, η υποβολή έγινε στις 4 Μαΐου 2026 με το pull request #7409 στο hacs/default repository [26]. Μετά από αναμονή δύο μηνών, το pull request εγκρίθηκε και έγινε merge στις 2 Ιουλίου 2026. Το integration "IR Remote (ANAVI IR pHAT)" είναι πλέον αναζητήσιμο απευθείας από οποιοδήποτε HACS instance σε ολόκληρο τον κόσμο, χωρίς ο χρήστης να χρειάζεται να γνωρίζει το URL του repository.

Custom repository ως προσωρινή λύση. Πριν την έγκριση του pull request, η εγκατάσταση γινόταν μέσω της λειτουργίας custom repositories του HACS. Ο χρήστης άνοιγε το HACS στο Home Assistant, επέλεγε "Custom repositories" από το μενού, έβαζε το URL του GitHub repository [33] με κατηγορία "Integration" και αμέσως το integration εμφανιζόταν προς λήψη, όπως οποιαδήποτε άλλη επέκταση του HACS. Από εκεί και πέρα, η εμπειρία ήταν ταυτόσημη με την τωρινή, όπως για οποιοδήποτε άλλο ενσωματωμένο integration στο hacs/default repository: αναζήτηση, εγκατάσταση, και αυτόματη ενημέρωση κάθε νέας έκδοσης.



Εικόνα 3.8a: Εισαγωγή custom repository στο HACS (πριν την έγκριση του Pull Request).



Εικόνα 3.8b: Το Integration μέσα από το HACS (μετά την έγκριση του Pull Request και την ενσωμάτωση).

Διαχείριση εκδόσεων από το HACS. Το HACS αναγνωρίζει τις διαθέσιμες εκδόσεις του integration μέσω των GitHub Releases που παράγει το release.yml workflow (3.6.A). Όταν δημοσιεύεται μια νέα έκδοση, το HACS του χρήστη εντοπίζει αυτόματα τη διαφορά μεταξύ της εγκατεστημένης και της νεότερης έκδοσης και εμφανίζει ειδοποίηση στα settings του Home Assistant. Ο χρήστης πατά "Update" και το HACS αναλαμβάνει τη λήψη της νέας έκδοσης, την αντικατάσταση των αρχείων στο /config/custom_components/ir_remote/ και την υπενθύμιση για επανεκκίνηση του Home Assistant (εφόσον απαιτείται), ώστε να φορτωθεί ο νέος κώδικας.

Διαφάνεια προς τον χρήστη. Σημαντικό χαρακτηριστικό αυτής της ροής είναι ότι ο τελικός χρήστης δεν χρειάζεται να γνωρίζει ή να αλληλεπιδρά καθόλου με το CI/CD pipeline του συγγραφέα. Από τη σκοπιά του, η εμπειρία είναι ίδια όπως με οποιοδήποτε άλλο επίσημο integration: εγκαθιστά μέσω HACS, ενημερώνεται όταν εμφανίζεται νέα έκδοση και εφαρμόζει την ενημέρωση με ένα κλικ. Η αυτοματοποίηση της παραγωγής και της δημοσίευσης συμβαίνει αυτόματα, εξασφαλίζοντας ότι κάθε έκδοση που φτάνει στους χρήστες έχει περάσει επιτυχώς τους ελέγχους HACS και Hassfest και ότι κανένα ημιτελές ή «σπασμένο» commit δεν διανέμεται κατά λάθος.

Οδηγίες εγκατάστασης και χρήσης

3.7

Στην ενότητα αυτή παρουσιάζεται ένας συνοπτικός οδηγός, για την πλήρη εγκατάσταση και τη βασική χρήση του συστήματος. Περισσότερες λεπτομέρειες βρίσκονται στα αντίστοιχα README των δύο repositories [27], [33].

Προαπαιτούμενα. Πριν την εγκατάσταση απαιτούνται: (α) ένα Raspberry Pi οποιουδήποτε μοντέλου με GPIO header 40 ακροδεκτών, μαζί με το ANAVI Infrared pHAT, (β) microSD κάρτα 8 GB ή μεγαλύτερη, (γ) υπάρχουσα εγκατάσταση Home Assistant σε οποιαδήποτε υποστηριζόμενη πλατφόρμα και (δ) διαμορφωμένος MQTT broker (συνήθως ο Mosquitto εγκατεστημένος ως add-on του Home Assistant), προσβάσιμος δικτυακά, τόσο από τη «γέφυρα», όσο και από το Home Assistant.

Εγκατάσταση «γέφυρας»:

(1) Εγκαθιστούμε το Raspberry Pi OS Lite στη microSD, μέσω του Raspberry Pi Imager, με συμπλήρωση, στοιχείων Wi-Fi και υπηρεσίας SSH, όπως αναφέρθηκε στην **ενότητα 3.3.A**.

(2) Συνδεόμαστε μέσω SSH στο Pi, «κατεβάζουμε» το bridge repository και εκτελούμε το install.sh script:

```
git clone https://github.com/h1dd3n3y3/pi-dumb2smart-ir-bridge.git
cd pi-dumb2smart-ir-bridge && sudo ./install.sh
```

(3) Το script αναλαμβάνει τη δημιουργία του Python virtualenv στο /opt/ir-bridge/ (με τις βιβλιοθήκες paho-mqtt και piir), την εγκατάσταση του pigpiod στο σύστημα (μέσω apt ή build από τον πηγαίο κώδικα, όπου δεν είναι διαθέσιμο), τη δημιουργία του καταλόγου δεδομένων στο /var/lib/ir-bridge/, τη ρύθμιση των υπηρεσιών systemd (pigpiod και ir-bridge) και την εκκίνησή τους.

(4) Κατά την εκτέλεση, το script ζητά από τον χρήστη τη διεύθυνση και τη θύρα του MQTT broker (προεπιλογή 1883), καθώς και ένα μοναδικό MQTT topic prefix για τη συγκεκριμένη «γέφυρα» (προεπιλογή ir_remote_<hostname>). Οι ρυθμίσεις αποθηκεύονται στο /etc/ir-bridge/env και μπορούν να τροποποιηθούν εκ των υστέρων με επεξεργασία του αρχείου. Σημειώνεται ότι η παρούσα διαμόρφωση, θέτει

ανώνυμη πρόσβαση στον MQTT broker εντός του τοπικού δικτύου και δεν χειρίζεται credentials — περιορισμός που αναλύεται περαιτέρω στην **ενότητα 4.5**.

Εγκατάσταση του Home Assistant integration:

- (1) Ανοίγουμε το HACS στο Home Assistant.
- (2) Αναζητούμε "IR Remote" και εγκαθιστούμε το integration "IR Remote (ANAVI IR pHAT)".
- (3) Επανεκκινούμε το Home Assistant.
- (4) Μετά την εκκίνηση, πηγαίνουμε στα Settings → Devices & Services → Add Integration, αναζητούμε "IR Remote (ANAVI IR pHAT)" και εισάγουμε το topic prefix (ίδιο με αυτό που δηλώθηκε στη «γέφυρα»).

Χρήση:

(1) Πηγαίνουμε στα Settings → Devices & Services, εντοπίζουμε το integration "IR Remote (ANAVI IR pHAT)" και πατάμε "Add entry". Στη φόρμα που εμφανίζεται, εισάγουμε το MQTT topic prefix της «γέφυρας» που θέλουμε να συνδέσουμε (ίδιο με αυτό που δηλώθηκε στο install.sh). Αν η «γέφυρα» είναι ενεργή και ο broker προσβάσιμος, δημιουργείται νέα integration entry με τίτλο της μορφής "IR Remote ({prefix})" και κάτω από αυτή εμφανίζεται αυτόματα ένα device entry με το όνομα "IR Bridge" — η αναπαράσταση της ίδιας της «γέφυρας» στο Home Assistant.

(2) Ανοίγουμε τη σελίδα του "IR Bridge", πληκτρολογούμε το όνομα μιας νέας τηλεχειριζόμενης συσκευής (π.χ. samsung_tv) στο πεδίο "Device Name" και πατάμε το κουμπί "Register Remote". Δημιουργείται, έτσι, νέο device entry για τη συσκευή, με τα δικά του κουμπιά διαχείρισης (Learn Key, Delete Key, Rename Key, Register Virtual Key) και πεδία κειμένου.

(3) Ανοίγουμε τη σελίδα της καινούριας συσκευής, πληκτρολογούμε το όνομα του προς καταγραφή πλήκτρου (π.χ. power) στο πεδίο "Key Name" και πατάμε το κουμπί "Learn Key". Εντός 30 δευτερολέπτων, στρέφουμε το φυσικό τηλεχειριστήριο προς τη «γέφυρα» και πατάμε το αντίστοιχο κουμπί (1-3 φορές).

(4) Μόλις η κατάσταση καταγραφής αλλάξει σε "done" (όπως αναπτύχθηκε στο 3.3.Γ), εμφανίζεται αυτόματα νέα οντότητα button για το πλήκτρο που μόλις καταγράφηκε. Με κάθε πάτημα αυτού του κουμπιού, η «γέφυρα» εκπέμπει το αντίστοιχο IR σήμα προς τη συσκευή-στόχο.

(5) Για τη δημιουργία εικονικών πλήκτρων (**3.3.Ε**): συμπληρώνουμε τα τέσσερα πεδία (όνομα νέου εικονικού πλήκτρου, όνομα πλήκτρου πηγής, πλήθος επαναλήψεων, καθυστέρηση σε ms) και πατάμε το κουμπί "Register Virtual Key".

(6) Για επιπλέον συσκευές στην ίδια «γέφυρα» (π.χ. ένας ενισχυτής δίπλα στην τηλεόραση), επιστρέφουμε στη σελίδα του "IR Bridge" του αντίστοιχου integration entry και επαναλαμβάνουμε τα βήματα (2) έως (4) — νέο "Device Name", "Register Remote" και στη συνέχεια, καταγραφή των πλήκτρων της. Κάθε νέα συσκευή

προστίθεται ως ξεχωριστό device entry, κάτω από το ίδιο integration entry, χωρίς να επηρεάζει τα υπόλοιπα.

(7) Για επιπλέον «γέφυρες» σε άλλα δωμάτια, επαναλαμβάνουμε το βήμα (1) από την αρχική σελίδα του integration πατώντας ξανά "Add entry", με διαφορετικό MQTT topic prefix κάθε φορά (**3.5.A**).

Ενημερώσεις. Νέες εκδόσεις του integration εμφανίζονται αυτόματα στο HACS όπως αναπτύχθηκε στην **ενότητα 3.6.Γ**: ο χρήστης πατά "Update" και επανεκκινεί το Home Assistant για να φορτωθεί η νέα έκδοση. Η «γέφυρα» ενημερώνεται μέσω SSH εκτελώντας το update.sh script του bridge repository (οπάνια χρειάζεται), το οποίο κάνει git pull, ενημερώνει τις εξαρτήσεις (dependencies) του virtualenv και επανεκκινεί την υπηρεσία systemd αυτόματα.

ΚΕΦΑΛΑΙΟ 4 – Πειράματα

Εισαγωγή

4.1

Στο προηγούμενο κεφάλαιο, περιγράφηκε αναλυτικά η υλοποίηση του συστήματος — η «γέφυρα» στο Raspberry Pi, το integration στο Home Assistant, το πρωτόκολλο επικοινωνίας MQTT και η αυτοματοποιημένη δημοσίευση εκδόσεων. Το κεφάλαιο αυτό αλλάζει σκοπιά: από την περιγραφή «πώς χτίστηκε το σύστημα», περνά στην εμπειρική αξιολόγηση, πώς συμπεριφέρεται σε ένα πραγματικό οικιακό περιβάλλον κάτω από συνθήκες μακροχρόνιας λειτουργίας. Σκοπός είναι να επιβεβαιωθεί ότι οι σχεδιαστικές επιλογές που τεκμηριώθηκαν στο Κεφάλαιο 3 παράγουν, στον πραγματικό κόσμο, ένα σύστημα που είναι λειτουργικό, με απόκριση και ανθεκτικό — και να εντοπιστούν τα όριά του.

Η δομή του κεφαλαίου, ακολουθεί τη σειρά από τη χρήση του συστήματος προς τις ακραίες συνθήκες λειτουργίας του. Στην **ενότητα 4.2**, παρουσιάζεται η εφαρμογή του συστήματος, σε τέσσερις πραγματικές συσκευές — μια τηλεόραση Samsung, δύο ενισχυτές (Technics, Marantz) και ένα κλιματιστικό — μαζί με την εξέλιξη της μεθόδου ελέγχου από SSH-based σε MQTT-based αρχιτεκτονική. Στην **ενότητα 4.3**, αξιολογείται η end-to-end καθυστέρηση από την ενέργεια του χρήστη, μέχρι την απόκριση της τελικής συσκευής, καθώς και η αξιοπιστία της επικοινωνίας σε κανονικές συνθήκες λειτουργίας. Στην **ενότητα 4.4**, εξετάζεται η ανθεκτικότητα του συστήματος σε προβλεπόμενα και απρόοπτα σφάλματα — επανεκκινήσεις «γέφυρας», διακοπές Wi-Fi, και επανεκκινήσεις του MQTT broker ή του Home Assistant — με αναφορά τόσο στους ενσωματωμένους μηχανισμούς ανάκαμψης, όσο και σε πρακτικές παρατηρήσεις από πραγματικές δοκιμές. Τέλος, στην **ενότητα 4.5**, παρουσιάζονται οι περιορισμοί που εντοπίστηκαν, χωρισμένοι σε εκείνους που πηγάζουν από τη φύση της υπέρυθρης επικοινωνίας και σε εκείνους που προέκυψαν

από σχεδιαστικές επιλογές της παρούσας υλοποίησης· οι αντίστοιχες κατευθύνσεις βελτίωσης θα αναπτυχθούν στο **Κεφάλαιο 5**.

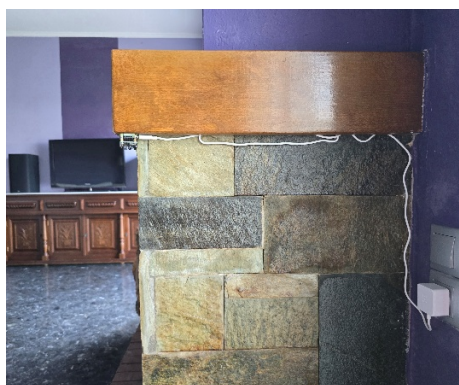
Εφαρμογή σε πραγματικές συσκευές

4.2

Η ανάπτυξη και αξιολόγηση του συστήματος έγινε σε ένα πραγματικό οικιακό περιβάλλον, με τέσσερις «γέφυρες» εγκατεστημένες σε διαφορετικά δωμάτια — κάθε μία με δικό της Raspberry Pi Zero 2 W, ANAVI Infrared pHAT και μοναδικό MQTT topic prefix. Όλες δηλώθηκαν στην ίδια εγκατάσταση Home Assistant μέσω του integration "IR Remote (ANAVI IR pHAT)" ως ξεχωριστά integration entries, σύμφωνα με τη σχεδίαση που περιγράφηκε στις **ενότητες 3.4** και **3.5**. Πάνω σε αυτή την υποδομή δοκιμάστηκαν τέσσερις διαφορετικές συσκευές: μια τηλεόραση Samsung, ένας ενισχυτής Technics στο ίδιο δωμάτιο με την τηλεόραση, ένας ενισχυτής Marantz σε άλλο δωμάτιο και ένα κλιματιστικό τοίχου.

Εξέλιξη της μεθόδου: από SSH σε MQTT. Στην αρχική, πιλοτική φάση, η καταγραφή πραγματοποιούνταν με το CLI script της βιβλιοθήκης PiIR, απευθείας πάνω στην κάθε «γέφυρα». Η αναπαραγωγή, από την άλλη, οργανώθηκε ως εξής: ο container του Home Assistant διαμορφώθηκε, ώστε να διαθέτει δικό του SSH key στις «γέφυρες» και κάθε ενέργεια του χρήστη στο Home Assistant dashboard, μεταφραζόταν σε εκτέλεση SSH εντολής προς το αντίστοιχο Raspberry Pi Zero 2 W. Η εντολή που εκτελούνταν απομακρυσμένα ήταν ένα απλό κάλεσμα του PiIR CLI της μορφής "piir play --gpio 17 --file <device>.json <key>", όπου δηλώνονται ρητά ο αριθμός του GPIO ακροδέκτη εκπομπής του ANAVI pHAT (17), το αρχείο JSON της συσκευής-στόχου και το όνομα του πλήκτρου ως τελική παράμετρος. Το PiIR φόρτωνε το αρχείο, ανακτούσε την αποθηκευμένη ακολουθία παλμών (raw timing) του πλήκτρου και την αναπαρήγαγε μέσω του pigpio και του ANAVI pHAT. Η προσέγγιση αυτή λειτούργησε ως απόδειξη ορθής λειτουργίας, αλλά παρουσίαζε δύο σοβαρά μειονεκτήματα: απαιτούσε από τον χρήστη να διαμορφώσει SSH keys και να έχει εξοικείωση με το τερματικό και εισήγαγε καθυστέρηση 1-2 δευτερολέπτων ανά εντολή, οφειλόμενη κυρίως στη δημιουργία νέου SSH session και στην εκκίνηση του Python interpreter για κάθε εκτέλεση. Η μετάβαση στο MQTT-based αρχιτεκτονικό σχήμα, που περιγράφηκε στο **Κεφάλαιο 3**, εξάλειψε την ανάγκη χειροκίνητης διαμόρφωσης SSH και περιόρισε την καθυστέρηση σε λιγότερο από 200 ms — μείωση τάξης μεγέθους που ποσοτικοποιείται στην **ενότητα 4.3**.

Τηλεόραση Samsung. Καταγράφηκαν τα ουσιαστά κουμπιά τηλεχειρισμού: power, volume up/down, βέλη πλοήγησης, OK, mute και source. Η βιβλιοθήκη PiIR αναγνώρισε χωρίς πρόβλημα τα IR πρωτόκολλα της τηλεόρασης Samsung (παραλλαγή του NEC, **2.3.B**) και η αναπαραγωγή μέσω Home Assistant ήταν αξιόπιστη, εφόσον η «γέφυρα» είχε άμεση οπτική επαφή με τον δέκτη της τηλεόρασης.



Εικόνα 4.1: Η «γέφυρα» της τηλεόρασης Samsung.

Ενισχυτής Technics. Ο ενισχυτής τοποθετείται δίπλα στην τηλεόραση Samsung και διαχειρίζεται από την ίδια «γέφυρα» μέσω δεύτερου device entry, λόγω απομόνωσής του σε έπιπλο. Καταγράφηκαν τα κουμπιά power, volume up/down και mute. Η ιδιαιτερότητα της περίπτωσης είναι ότι δεν υπήρχε διαθέσιμο το αυθεντικό τηλεχειριστήριο της συσκευής. Αντί αυτού, χρησιμοποιήθηκε το Flipper Zero [34] — φορητή συσκευή-πολυεργαλείο ανοιχτού κώδικα, με ενσωματωμένο IR transceiver — σε συνδυασμό με το Flipper-IRDB [35], κοινόχρηστο αποθετήριο στο GitHub με IR αρχεία πραγματικών τηλεχειριστηρίων. Εντοπίστηκε αρχείο για συγγενικό μοντέλο Technics (Technics_EUR646496.ir), φορτώθηκε στο Flipper Zero και χρησιμοποιήθηκε για να εκπέμψει τα σήματα προς τη «γέφυρα», η οποία τα κατέγραψε φυσιολογικά, όπως από κάθε άλλο φυσικό τηλεκοντρόλ. Όλες οι εντολές λειτούργησαν επιτυχώς από την πρώτη καταγραφή. Η περίπτωση δείχνει ότι η εγκατάσταση δεν εξαρτάται από τη διαθεσιμότητα του αυθεντικού τηλεχειριστηρίου — οποιαδήποτε αξιόπιστη πηγή IR σήματος αρκεί — και ότι κοινόχρηστα IR αποθετήρια μπορούν να εντάξουν στο σύστημα συσκευές με χαμένο ή κατεστραμμένο remote.



Εικόνα 4.2: Η «γέφυρα» του ενισχυτή Technics.

Ενισχυτής Marantz. Ο ενισχυτής βρίσκεται σε άλλο δωμάτιο και διαχειρίζεται από ξεχωριστή «γέφυρα». Καταγράφηκαν τα κουμπιά power, volume up/down, mute και sleep. Η συσκευή ανέδειξε δύο ζητήματα. Πρώτον, ένα τηλεχειριστήριο-ομοίωμα (replica) που χρησιμοποιήθηκε αρχικά, εκπέμπει για το πλήκτρο power διαφορετικό IR κωδικό από εκείνον που περιμένει ο αυθεντικός ενισχυτής. Η λύση ήταν η καταγραφή του power μέσω του ίδιου καθολικού τηλεχειριστηρίου που χρησιμοποιήθηκε για την τηλεόραση Samsung, αφού επιλέχθηκε ο κατάλληλος κωδικός για Marantz. Δεύτερον, ο ενισχυτής δεν αντιδρά σε ένα μεμονωμένο IR pulse του power: για να εναλλάξει την κατάσταση on/off, χρειάζεται δύο διαδοχικές εκπομπές της εντολής, με κενό επανάληψης ~150 ms. Το ζήτημα επιλύθηκε καθαρά μέσα από τον μηχανισμό εικονικών πλήκτρων (3.3.E): καταχωρήθηκε εικονικό πλήκτρο "power_toggle" με πηγή το "power" που καταγράφηκε αρχικά, πλήθος επαναλήψεων 2 και καθυστέρηση 150 ms. Πρόκειται για άμεση πρακτική επιβεβαίωση της σχεδιαστικής επιλογής των εικονικών πλήκτρων: ιδιαίτερες απαιτήσεις χρόνου καλύπτονται, χωρίς αλλαγές στον κώδικα της «γέφυρας» ή του integration.



Εικόνα 4.3: Η «γέφυρα» του ενισχυτή Marantz.

Κλιματιστικό. Σε αντίθεση με τις προηγούμενες συσκευές, το κλιματιστικό παρουσίασε ιδιαίτερη συμπεριφορά: κάθε πάτημα του φυσικού τηλεχειριστηρίου μεταδίδει σε ένα ενιαίο πλαίσιο IR την πλήρη τρέχουσα κατάσταση της συσκευής

(θερμοκρασία, λειτουργία cool/heat/dry/fan, ταχύτητα ανεμιστήρα, mode), όχι μόνο τη διαφορά. Συνέπεια αυτού, είναι ότι κάθε καταγεγραμμένο πλήκτρο αποθηκεύει την ολική κατάσταση εκείνης της στιγμής. Για την πειραματική επιβεβαίωση, καταγράφηκε ένα πάτημα «αύξηση θερμοκρασίας» από το αυθεντικό τηλεχειριστήριο ενώ το κλιματιστικό βρισκόταν σε λειτουργία ψύξης (cool) στους 24°C. Στη συνέχεια, η συσκευή ρυθμίστηκε χειροκίνητα στους 27°C και ενεργοποιήθηκε το καταγεγραμμένο πλήκτρο από το Home Assistant. Αντί η θερμοκρασία να αυξηθεί κατά μία βαθμίδα, το κλιματιστικό επανήλθε στην ακριβή κατάσταση της καταγραφής — 24°C cool. Η εντολή "power off" αντίθετα λειτούργησε σωστά ανεξάρτητα από την τρέχουσα κατάσταση, καθώς το IR πλαίσιο της απενεργοποίησης είναι πρακτικά ισοδύναμο σε κάθε στιγμιότυπο. Το σύστημα επομένως λειτουργεί άψογα για μονοσήμαντες εντολές αλλά είναι περιοριστικό για ενεργοποίηση και μεταβολή παραμέτρων εν λειτουργία. Ο περιορισμός αναλύεται στην **ενότητα 4.5** και προτείνεται ως μελλοντική επέκταση στην **5.2**.

Καθυστέρηση και αξιοπιστία επικοινωνίας

4.3

Δύο μη-λειτουργικά χαρακτηριστικά καθορίζουν την υποκειμενική ποιότητα ενός smart-home συστήματος: η end-to-end καθυστέρηση, από την ενέργεια του χρήστη μέχρι την απόκριση της τελικής συσκευής και η αξιοπιστία, με την οποία οι εντολές παραδίδονται κάτω από κανονικές συνθήκες λειτουργίας. Στην ενότητα αυτή παρουσιάζονται τα αντίστοιχα ευρήματα από την παραγωγική χρήση του συστήματος. Η ανθεκτικότητα σε σφάλματα (διακοπές δικτύου, επανεκκινήσεις) εξετάζεται ξεχωριστά στην **ενότητα 4.4**.

Καθυστέρηση. Όπως αναφέρθηκε στην **ενότητα 4.2**, η αρχική SSH-based υλοποίηση εισήγαγε καθυστέρηση 1-2 δευτερολέπτων ανά εντολή — εμφανώς αισθητή στον τελικό χρήστη. Το κόστος προερχόταν κυρίως από δύο πηγές: τη δημιουργία νέου SSH session για κάθε εντολή (TCP handshake, ανταλλαγή keys, αυθεντικοποίηση) και την εκκίνηση του Python interpreter για κάθε εκτέλεση του PiIR script. Η μετάβαση στο MQTT-based σχήμα εξάλειψε και τα δύο αυτά κόστη: η σύνδεση της «γέφυρας» στον broker είναι μόνιμη και αρχικοποιείται μία φορά, κατά την εκκίνηση της υπηρεσίας systemd, ενώ ο Python interpreter του mqtt_bridge.py παραμένει ενεργός καθ' όλη τη διάρκεια λειτουργίας, με τα αρχεία JSON ήδη φορτωμένα στη μνήμη μετά την πρώτη χρήση. Η συνολική end-to-end καθυστέρηση, από το πάτημα κουμπιού στο dashboard του Home Assistant, μέχρι την εκπομπή του IR σήματος από τον ANAVI pHAT, περιορίστηκε εμπειρικά σε λιγότερο από 200 ms — μείωση τάξης μεγέθους, η οποία στην πράξη γίνεται αντιληπτή από τον χρήστη ως "στιγμιαία" απόκριση, αντίστοιχη με εκείνη ενός κανονικού φυσικού τηλεχειριστηρίου.

Αξιοπιστία επικοινωνίας σε κανονική λειτουργία. Στη φάση κανονικής λειτουργίας — χωρίς διακοπές δικτύου ή reboots — η αξιοπιστία του MQTT καναλιού στηρίζεται σε δύο μηχανισμούς που αναπτύχθηκαν στις **ενότητες 2.4.Γ** και **3.5.Β**. Πρώτον, οι μόνιμες TCP συνδέσεις προς τον broker, σε συνδυασμό με τους keep-alive μηχανισμούς του πρωτοκόλλου MQTT, διασφαλίζουν ότι το κανάλι παραμένει ανοιχτό και έτοιμο να μεταφέρει νέα μηνύματα, χωρίς καθυστέρηση

επανασύνδεσης. Δεύτερον, τα μηνύματα κατάστασης ("{prefix}/devices", "{prefix}/record/status") δημοσιεύονται ως retained, οπότε ένα νέο instance του integration ή ένα Home Assistant που μόλις πραγματοποιήσει επανεκκίνηση, λαμβάνει αυτόματα τη φρέσκια εικόνα του store και του δέκτη, χωρίς explicit query. Στη μακρόχρονη παραγωγική χρήση των τεσσάρων «γέφυρών», σε διάστημα αρκετών μηνών, δεν παρατηρήθηκε απώλεια εντολής ή ασυμφωνία μεταξύ της κατάστασης που εμφανίζει το Home Assistant και αυτής που πραγματικά διατηρεί η εκάστοτε «γέφυρα», όσο τα δύο μέρη παραμένουν συνδεδεμένα στον broker.

Σημείωση για τα όρια του πρωτοκόλλου IR. Αξίζει να σημειωθεί ότι ένα μέρος του χρόνου που μετράται από το χρήστη ως «καθυστέρηση», οφείλεται όχι στο σύστημα, αλλά στο ίδιο το φυσικό όριο των IR πρωτοκόλλων: ένα τυπικό NEC πλαίσιο διαρκεί ~67 ms, συμπεριλαμβανομένου του AGC burst και κάποιες συσκευές περιμένουν 1-3 διαδοχικά τέτοια πλαίσια πριν θεωρήσουν την εντολή έγκυρη (βλ. και την περίπτωση Marantz στο 4.2). Η καθυστέρηση των <200 ms που αναφέρθηκε παραπάνω είναι το συνολικό end-to-end άθροισμα, που περιλαμβάνει τόσο το MQTT-side latency όσο και τη διάρκεια του ίδιου του IR σήματος.

Αντοχή σε σφάλματα (διακοπές δικτύου, επανεκκινήσεις) 4.4

Πέρα από την αξιοπιστία της επικοινωνίας σε κανονική λειτουργία (4.3), ένα σύστημα οικιακού αυτοματισμού οφείλει να ανέχεται διακοπές που είναι αναπόφευκτες σε μακρόχρονη χρήση: διακοπές ρεύματος, αναβαθμίσεις λογισμικού, στιγμιαίες απώλειες Wi-Fi και επανεκκινήσεις των κεντρικών υπηρεσιών. Στην ενότητα αυτή παρουσιάζονται οι μηχανισμοί ανάκαμψης του συστήματος και οι παρατηρήσεις από πραγματικές δοκιμές των τριών βασικών σεναρίων σφάλματος.

Επανεκκίνηση «γέφυρας». Το πιο συνηθισμένο σενάριο είναι η απρόοπτη απώλεια ρεύματος ή η σκόπιμη επανεκκίνηση κάποιας «γέφυρας». Όπως αναπτύχθηκε στο 3.3.ΣΤ, τόσο το pigpiod όσο και το mqtt_bridge.py, εκτελούνται ως υπηρεσίες systemd με Restart=always, ενώ τα αρχεία JSON των συσκευών διατηρούνται στο /var/lib/ir-bridge/ — μόνιμο σημείο του filesystem που επιβιώνει επανεκκινήσεων. Μετά από power cycle, η ακολουθία επαναφοράς εξελίσσεται αυτόματα: εκκίνηση του Raspberry Pi OS, σύνδεση στο Wi-Fi μέσω wpa_supplicant, εκκίνηση του pigpiod, εκκίνηση του mqtt_bridge.py, σύνδεση στον broker, και δημοσίευση των retained μηνυμάτων κατάστασης (devices, record/status) — τα οποία ενημερώνουν αυτόματα το Home Assistant για την τρέχουσα κατάσταση. Στη συνολική χρονική αλληλουχία, η «γέφυρα» επανέρχεται σε λειτουργικότητα εντός 30-60 δευτερολέπτων από την παροχή ρεύματος, χωρίς να απαιτείται καμία χειροκίνητη ενέργεια από τον χρήστη ούτε επανεκκίνηση του Home Assistant.

Διακοπή Wi-Fi. Όταν χάνεται προσωρινά η σύνδεση Wi-Fi, η TCP σύνδεση της «γέφυρας» προς τον MQTT broker διακόπτεται. Ο μηχανισμός loop_forever της raho-mqtt αναγνωρίζει την αποσύνδεση και προσπαθεί αυτόματα να επανασυνδεθεί με exponential backoff, μέχρι να αποκατασταθεί το δίκτυο. Όσο διαρκεί η διακοπή, τυχόν εντολές που στέλνει ο χρήστης από το Home Assistant δεν παραδίδονται στη «γέφυρα» (καθώς τα μηνύματα εντολής δεν είναι retained — βλ. 3.5.B)· μόλις όμως αποκατασταθεί η σύνδεση, η «γέφυρα» επιστρέφει σε πλήρη λειτουργία και τα

retained μηνύματα κατάστασης συγχρονίζονται αυτόματα. Στην πράξη, η συγκεκριμένη συμπεριφορά δοκιμάστηκε σε μία από τις «γέφυρες», που αρχικά τοποθετημένη σε δωμάτιο, μακριά από το access point, αποσυνδεόταν περιοδικά λόγω ασθενούς σήματος. Σε πρώτη φάση επιχειρήθηκε λύση σε επίπεδο λογισμικού — ένα Wi-Fi watchdog script, που έλεγε περιοδικά τη συνδεσιμότητα και επανεκκινούσε το wlan interface όταν εντόπιζε αποτυχία — αλλά αποδείχθηκε αναξιόπιστη: το restart του interface καθυστερούσε ή κάποιες φορές αποτύγχανε εντελώς και κυρίως δεν αντιμετώπιζε το πραγματικό αίτιο, που ήταν το αδύναμο RF σήμα. Η τελική λύση ήταν η φυσική μετακίνηση της «γέφυρας» πιο κοντά στο access point· από εκείνο το σημείο και μετά η σύνδεση παρέμεινε σταθερή χωρίς ad hoc μηχανισμούς. Το συμπέρασμα της εμπειρίας είναι ότι τα software workarounds δεν αντικαθιστούν τη σωστή φυσική εγκατάσταση — για μακρόχρονη αξιοπιστία 24/7, η «γέφυρα» πρέπει να τοποθετείται σε σημείο με ικανοποιητικό σήμα Wi-Fi.

Επανεκκίνηση του MQTT broker ή του Home Assistant. Στην περίπτωση που επανεκκινηθεί ο Mosquitto broker (π.χ. μετά από update του Home Assistant add-on) ή ολόκληρο το Home Assistant, η συμπεριφορά είναι ανάλογη. Οι MQTT clients — τόσο της «γέφυρας» όσο και του integration — εντοπίζουν την απώλεια σύνδεσης και επανασυνδέονται μόλις ο broker επιστρέψει. Επειδή ο Mosquitto διατηρεί τα retained μηνύματα σε persistent storage, ένα Home Assistant instance που μόλις επανεκκινήθηκε, λαμβάνει αμέσως μετά τη σύνδεση τη φρέσκια εικόνα του store και των στατικών χαρακτηριστικών της κάθε «γέφυρας», χωρίς να χρειάζεται explicit query — λειτουργικότητα κρίσιμη ώστε η αρχικοποίηση των integration entries να μη χρειάζεται ζωντανή «γέφυρα» (όπως αναλύθηκε στις **ενότητες 2.4.Γ** και **3.5.Β**). Σε καμία από τις δοκιμές δεν παρατηρήθηκε απώλεια ή διαφθορά αποθηκευμένων συσκευών ή πλήκτρων σαν αποτέλεσμα τέτοιων επανεκκινήσεων.

Πρακτική σύσταση: επαρκής τροφοδοσία. Πέρα από τους ίδιους τους μηχανισμούς ανάκαμψης, η πιο αποτελεσματική στρατηγική αντιμετώπισης σφαλμάτων είναι η πρόληψη. Ένας κλασικός παράγοντας απρόοπτων επανεκκινήσεων στα Raspberry Pi, είναι η ανεπαρκής τροφοδοσία — υπο-τάσεις που οδηγούν σε CPU throttling, αυτόματα reboots και σε βάθος χρόνου, σε φθορά της microSD κάρτας λόγω επανειλημμένων απότομων απενεργοποιήσεων. Για το Raspberry Pi Zero 2 W που χρησιμοποιείται στις «γέφυρες», αρκεί ένα τροφοδοτικό 5V/1A καλής ποιότητας, χωρίς ιδιαίτερες απαιτήσεις. Σε εγκαταστάσεις όπου ο server που φιλοξενεί το Home Assistant τρέχει επίσης σε Raspberry Pi, τα νεότερα μοντέλα έχουν ουσιαστικά μεγαλύτερες απαιτήσεις (Pi 4: USB-C 5V/3A, Pi 5: USB-C 5V/5A). Η επιλογή κατάλληλου τροφοδοτικού, είναι πρακτικά η απλούστερη και πιο αποτελεσματική παρέμβαση για μακρόχρονη σταθερή λειτουργία.

Περιορισμοί και προβλήματα

4.5

Παρά τη γενικά επιτυχημένη λειτουργία που τεκμηριώθηκε στις προηγούμενες ενότητες, το σύστημα έχει εγγενείς περιορισμούς που πηγάζουν είτε από την ίδια τη φύση της υπέρυθρης επικοινωνίας, είτε από σχεδιαστικές επιλογές που έγιναν για χάρη απλότητας στην πρώτη υλοποίηση. Στην ενότητα αυτή παρουσιάζονται τέσσερις βασικοί περιορισμοί που εντοπίστηκαν κατά τη χρήση, μαζί με συνοπτική

αναφορά στις πιθανές κατευθύνσεις αντιμετώπισής τους — οι οποίες αναπτύσσονται ως μελλοντικές βελτιώσεις στην **ενότητα 5.2**.

Stateful IR συσκευές (κλιματιστικά). Όπως αναπτύχθηκε στις **ενότητες 3.3.Γ** και **4.2**, η αρχιτεκτονική «καταγραφής και αναπαραγωγής αυτούσιου σήματος» αποθηκεύει κάθε πλήκτρο ως ακολουθία raw timing — μια προσέγγιση που λειτουργεί άψογα για συσκευές όπου κάθε πάτημα αντιστοιχεί σε διακριτή, μονοσήμαντη εντολή. Στα κλιματιστικά όμως, το αυθεντικό τηλεχειριστήριο μεταδίδει σε κάθε πάτημα την πλήρη τρέχουσα κατάσταση της συσκευής (θερμοκρασία, λειτουργία cool/heat/dry/fan, ταχύτητα ανεμιστήρα, mode) σε ένα ενιαίο IR πλαίσιο. Συνέπεια αυτού είναι ότι ένα καταγεγραμμένο πλήκτρο αποθηκεύει την ολική κατάσταση εκείνης της στιγμής και η αναπαραγωγή επιβάλλει αυτή τη συγκεκριμένη κατάσταση, ανεξάρτητα από τις ρυθμίσεις που έχει εφαρμόσει εν τω μεταξύ ο χρήστης. Πρακτικά, το σύστημα λειτουργεί αξιόπιστα για μονοσήμαντες εντολές όπως η απενεργοποίηση (όπου το IR πλαίσιο της απενεργοποίησης είναι πρακτικά ισοδύναμο σε κάθε στιγμιότυπο), αλλά γίνεται περιοριστικό για ενεργοποίηση και μεταβολή παραμέτρων εν λειτουργία. Η κατεύθυνση αντιμετώπισης που προτείνεται στο **5.2**, είναι η προσθήκη ενός νέου τύπου οντότητας climate στο integration, το οποίο θα τηρεί εσωτερικά το tracked state και θα επιλέγει το κατάλληλο pre-recorded πλήκτρο πλήρους κατάστασης για κάθε αλλαγή του χρήστη — με το αντίστοιχο compromise ότι ο χρήστης θα πρέπει να αποφεύγει το φυσικό τηλεχειριστήριο, ώστε να μην αποσυγχρονίζεται η πραγματική κατάσταση από το tracked state.

Ανώνυμο MQTT και έλλειψη TLS. Στην παρούσα υλοποίηση, το install.sh script (**3.7**) διαμορφώνει τη «γέφυρα», ώστε να συνδέεται στον MQTT broker ανώνυμα, χωρίς authentication και χωρίς TLS κρυπτογράφηση. Η επιλογή αυτή απλοποιεί σημαντικά την εγκατάσταση από τον τελικό χρήστη και λειτουργεί επιτυχώς στο τυπικό σενάριο όπου ο Mosquitto εγκαθίσταται ως add-on του Home Assistant, σε ένα έμπιστο οικιακό LAN, με anonymous access ενεργοποιημένο. Ωστόσο, εισάγει συγκεκριμένους περιορισμούς ασφάλειας και ευελιξίας ανάπτυξης: το σύστημα δεν μπορεί να λειτουργήσει με authenticated brokers (π.χ. εταιρικά ή κοινόχρηστα δίκτυα όπου ο Mosquitto διαθέτει password file), δεν προστατεύει την κίνηση από υποκλοπή σε μη εμπιστευτικά δίκτυα και δεν αποτρέπει την πιθανότητα ένα κακόβουλο πρόγραμμα στο ίδιο τοπικό δίκτυο να εκπέμψει αυθαίρετα IR σήματα μέσω της «γέφυρας». Η αντιμετώπιση στο **5.2** προτείνει την επέκταση τόσο του install.sh όσο και του config_flow.py του integration, ώστε να δέχονται προαιρετικά πεδία username/password και να υποστηρίζουν TLS πιστοποιητικά για κρυπτογραφημένη επικοινωνία με broker, σε λιγότερο εμπιστευτικά περιβάλλοντα.

Μονόδρομη υπέρυθρη επικοινωνία. Η ίδια η τεχνολογία υπέρυθρου τηλεχειρισμού (**2.3**) είναι κατασκευαστικά μονόδρομη: ο πομπός εκπέμπει σήμα προς τον δέκτη, χωρίς ο δεύτερος να επιστρέφει επιβεβαίωση ή πληροφορία κατάστασης. Συνεπώς, η «γέφυρα» δεν έχει τρόπο να γνωρίζει αν η εντολή που εξέπεμψε ελήφθη πράγματι από τη συσκευή-στόχο, ούτε ποια είναι η πραγματική τρέχουσα κατάσταση της συσκευής (αναμμένη/οθνηστή, ποια πηγή, τι ένταση κ.ο.κ.). Αν για παράδειγμα ένας χρήστης πατήσει "power" στο dashboard ενώ η τηλεόραση είναι ήδη αναμμένη, το σύστημα δεν θα ξεχωρίσει αν η εντολή είχε αποτέλεσμα (έσβησε την τηλεόραση) ή όχι (την άναψε, αν ήταν οθνηστή). Η αρχιτεκτονική περιορίζεται έτσι, σε ένα fire-and-forget μοντέλο, όπου η αναπαράσταση κατάστασης

στο Home Assistant βασίζεται μόνο στις εντολές που έχει στείλει ο ίδιος ο HA, χωρίς δυνατότητα επαλήθευσης από τη συσκευή. Πρόκειται για φυσικό όριο της τεχνολογίας IR — όχι αδυναμία της συγκεκριμένης υλοποίησης — και δεν αντιμετωπίζεται χωρίς συμπληρωματικές τεχνολογίες όπως αισθητήρες κατανάλωσης ρεύματος, smart plugs ή άλλες συσκευές, για επιβεβαίωση κατάστασης.

Απαίτηση οπτικής επαφής και εμβέλεια. Ο υπέρυθρος πομπός του ANAVI Infrared pHAT έχει συγκεκριμένο γωνιακό πεδίο εκπομπής και πεπερασμένη εμβέλεια — τυπικά λίγα μέτρα σε ευθεία γραμμή. Επιπλέον, οι IR δέκτες των τυπικών οικιακών συσκευών απαιτούν άμεση οπτική επαφή με τον πομπό, καθώς το υπέρυθρο σήμα δεν διαπερνά τοίχους, πόρτες ή ντουλάπια. Στην πράξη αυτό σημαίνει ότι κάθε «γέφυρα» πρέπει να τοποθετείται σε σημείο με ξεκάθαρη οπτική επαφή με τη συσκευή-στόχο και ότι μία «γέφυρα» δεν επαρκεί για συσκευές σε διαφορετικά δωμάτια — όπως φάνηκε άλλωστε στην παρούσα εργασία, όπου χρησιμοποιήθηκαν τέσσερις διαφορετικές «γέφυρες», μία ανά συσκευή. Η αρχιτεκτονική του topic prefix (**3.5.A**) σχεδιάστηκε ακριβώς για να επιτρέπει αυτή την κλιμάκωση σε πολλαπλά σημεία, αλλά παραμένει εγγενής περιορισμός ότι κάθε νέα συσκευή σε νέο σημείο απαιτεί επιπλέον υλικό (Raspberry Pi + ANAVI pHAT + τροφοδοσία ρεύματος).

ΚΕΦΑΛΑΙΟ 5 – Συμπεράσματα

Ανακεφαλαίωση και αξιολόγηση αποτελεσμάτων

5.1

Σκοπός της παρούσας εργασίας, ήταν η ενσωμάτωση τυπικών ηλεκτρονικών συσκευών — τηλεοράσεων, ηχοσυστημάτων, κλιματιστικών και αντίστοιχων οικιακών συσκευών που ελέγχονται αποκλειστικά μέσω υπέρυθρου τηλεχειρισμού — στο οικοσύστημα του Home Assistant, ώστε να μπορούν να συμμετέχουν ισότιμα στις αυτοματοποιήσεις, τα dashboards και τα φωνητικά interfaces ενός σύγχρονου έξυπνου σπιτιού. Για την επίτευξή του, σχεδιάστηκε και υλοποιήθηκε ένα σύστημα δύο μερών: μια φυσική «γέφυρα» που εκτελείται σε Raspberry Pi Zero 2 W με το ANAVI Infrared pHAT και αναλαμβάνει την καταγραφή και αναπαραγωγή IR σημάτων, και ένα custom integration για το Home Assistant, που διανέμεται μέσω του HACS και εκθέτει τις τηλεχειριζόμενες συσκευές ως κανονικά entities (button, sensor, number, text). Τα δύο μέρη επικοινωνούν μέσω MQTT, ενώ μια αυτοματοποιημένη ροή στα GitHub Actions διασφαλίζει τη συνεπή και ποιοτικά ελεγμένη δημοσίευση κάθε νέας έκδοσης.

Η εμπειρική αξιολόγηση που παρουσιάστηκε στο **Κεφάλαιο 4**, επιβεβαιώνει ότι το σύστημα ανταποκρίνεται με συνέπεια στους αρχικούς στόχους. Δοκιμάστηκε επιτυχώς σε τέσσερις πραγματικές συσκευές (μια τηλεόραση Samsung, δύο ενισχυτές Technics και Marantz, ένα κλιματιστικό), σε πραγματικό οικιακό περιβάλλον, με τέσσερις «γέφυρες» εγκατεστημένες σε διαφορετικά σημεία. Η μετάβαση από το αρχική SSH-based πιλοτική προσέγγιση στο MQTT-based

αρχιτεκτονικό σχήμα της τελικής υλοποίησης μείωσε την end-to-end καθυστέρηση κατά μία τάξη μεγέθους (από 1-2 δευτερόλεπτα σε λιγότερο από 200 ms), σε επίπεδα που γίνονται αντιληπτά από τον χρήστη ως «στιγμιαία» απόκριση, πλέον. Η ανθεκτικότητα του συστήματος σε σφάλματα — επανεκκινήσεις «γέφυρας», διακοπές Wi-Fi και επανεκκινήσεις του MQTT broker ή του Home Assistant — αποδείχθηκε επαρκής, χωρίς να απαιτείται οποιαδήποτε χειροκίνητη παρέμβαση. Επιπλέον, σχεδιαστικές επιλογές, όπως ο μηχανισμός εικονικών πλήκτρων, αποδείχθηκαν πολύτιμες στην πράξη, καλύπτοντας απαιτήσεις χρόνου που εμφανίστηκαν σε πραγματική συσκευή (π.χ. η ανάγκη διπλής εκπομπής της εντολής power στον ενισχυτή Marantz με διάστημα 150 ms), χωρίς να χρειαστεί τροποποίηση του βασικού κώδικα.

Παράλληλα, η αξιολόγηση ανέδειξε με ειλικρίνεια τα όρια της παρούσας υλοποίησης. Ο σημαντικότερος λειτουργικός περιορισμός αφορά τις stateful IR συσκευές, τυπικά τα κλιματιστικά, όπου η μέθοδος καταγραφής αυτούσιου σήματος αποθηκεύει την πλήρη κατάσταση κάθε πατήματος και δεν μπορεί να αναπαράγει πιστά εντολές τύπου «αύξησε κατά μία βαθμίδα». Δύο επιπλέον περιορισμοί προέρχονται από την αρχιτεκτονική επιλογή της απλής εγκατάστασης (ανώνυμη πρόσβαση MQTT, χωρίς TLS) και από τη φύση του ίδιου του IR (μονόδρομη επικοινωνία χωρίς feedback από τις συσκευές). Καθένας από αυτούς αντιστοιχεί σε σαφή κατεύθυνση βελτίωσης που αναπτύσσεται στην επόμενη ενότητα. Συνολικά, η εργασία αποδεικνύει ότι είναι εφικτή η ενσωμάτωση συσκευών χωρίς εργοστασιακή υποστήριξη IoT στο σύγχρονο smart-home, με ανοιχτό λογισμικό, ευρέως διαθέσιμο υλικό χαμηλού κόστους και αυτοματοποιημένες ροές διανομής που εξασφαλίζουν τη μακρόχρονη βιωσιμότητα του κώδικα — επιβεβαιώνοντας τη βασική θέση που τέθηκε στην εισαγωγική **ενότητα 1.2**.

Μελλοντικές βελτιώσεις

5.2

Από τους περιορισμούς που εντοπίστηκαν στην **ενότητα 4.5** και από τη γενικότερη εμπειρία ανάπτυξης και χρήσης του συστήματος, προκύπτουν αρκετές κατευθύνσεις βελτίωσης. Στην ενότητα αυτή παρουσιάζονται τέσσερις από αυτές, οι οποίες έχουν σαφή πρακτική σημασία και είναι υλοποιήσιμες, χωρίς ριζική αλλαγή της υπάρχουσας αρχιτεκτονικής — δηλαδή μπορούν να ενσωματωθούν σταδιακά ως νέες εκδόσεις του «IR Remote (ANAVI IR pHAT)» integration και της «γέφυρας», διατηρώντας πλήρη συμβατότητα με τις υπάρχουσες εγκαταστάσεις.

Climate entity για stateful IR συσκευές. Η σημαντικότερη βελτίωση αφορά την αντιμετώπιση του περιορισμού των κλιματιστικών που τεκμηριώθηκε στις **ενότητες 4.2** και **4.5**. Σε κάθε περίπτωση, στο integration θα προστεθεί νέου τύπου οντότητα climate, η οποία θα εκθέτει στο Home Assistant standard controls για ρύθμιση θερμοκρασίας, λειτουργίας (π.χ. κρύο ή ζεστό) και ταχύτητα ανεμιστήρα. Ως προς τον μηχανισμό παραγωγής του αντίστοιχου IR σήματος για κάθε ενέργεια του χρήστη, διακρίνονται δύο εναλλακτικές προσεγγίσεις, με διαφορετική πολυπλοκότητα και ισχύ:

- (1) Πρώτη λύση: pre-recorded state table. Ο χρήστης καταγράφει εφάπαξ ένα σύνολο «πλήρων καταστάσεων», ως ξεχωριστά πλήκτρα στη «γέφυρα» (π.χ.

cool_20_turbo, cool_22_auto, heat_24_low, off), καλύπτοντας τους συνηθισμένους συνδυασμούς που επιθυμεί να χρησιμοποιεί. Η οντότητα climate διατηρεί εσωτερικά το tracked state και αντιστοιχεί κάθε αλλαγή στο HA UI στο κατάλληλο pre-recorded πλήκτρο πλήρους κατάστασης. Η προσέγγιση είναι ελαφριά, ανεξάρτητη από τη μάρκα του κλιματιστικού και επαναχρησιμοποιεί την υπάρχουσα αρχιτεκτονική raw timing, που λειτουργεί άψογα για όλες τις άλλες συσκευές. Ο συμβιβασμός είναι ότι ο χρήστης πρέπει να αποφεύγει το φυσικό τηλεχειριστήριο (ώστε το tracked state να μην αποσυγχρονίζεται) και ότι υποστηρίζονται μόνο οι συνδυασμοί που έχει καταγράψει εκ των προτέρων · συνεχείς αυξομειώσεις θερμοκρασίας ή οπάνιοι συνδυασμοί δεν καλύπτονται χωρίς πρόσθετη καταγραφή.

- (2) Δεύτερη λύση: βιβλιοθήκη με γνώση των IR πρωτοκόλλων. Εναλλακτικά, το λογισμικό της «γέφυρας» θα μπορούσε να ενσωματώσει μια βιβλιοθήκη που γνωρίζει εκ των προτέρων τη δομή των bits, τα πεδία θερμοκρασίας/mode/fan και τους αλγόριθμους checksum για κάθε υποστηριζόμενο μοντέλο κλιματιστικού — υπάρχοντα projects, όπως η IRremoteESP8266, αποτελούν αντίστοιχα παραδείγματα. Με αυτή την προσέγγιση, η οντότητα climate παράγει στιγμιαία το πλήρες IR πλαίσιο για οποιονδήποτε συνδυασμό temperature/mode/fan, χωρίς να χρειάζεται προ-καταγραφή σταθερών συνδυασμών. Επιπλέον, αν η ίδια βιβλιοθήκη χρησιμοποιηθεί και για αποκωδικοποίηση εισερχόμενων σημάτων, η «γέφυρα» μπορεί να ενημερώνει το tracked state όταν ο χρήστης χρησιμοποιεί το φυσικό τηλεχειριστήριο, εξαλείφοντας τον περιορισμό της πρώτης λύσης. Το κόστος, σε αντιστάθμιση, είναι σαφώς υψηλότερη πολυπλοκότητα: απαιτείται υποστήριξη συγκεκριμένων brand/model πρωτοκόλλων και συνεχής συντήρηση του καταλόγου, ενώ συσκευές εκτός της λίστας υποστήριξης δε μπορούν να ελεγχθούν.

Οι δύο λύσεις είναι συμπληρωματικές και θα μπορούσαν ακόμη και να συνυπάρξουν στο ίδιο integration: η πρώτη ως default fallback για κάθε κλιματιστικό και η δεύτερη ως «προηγμένη» επιλογή για τα μοντέλα όπου η βιβλιοθήκη παρέχει πλήρη υποστήριξη πρωτοκόλλου.

Προαιρετική αυθεντικοποίηση και κρυπτογράφηση MQTT. Για την κάλυψη του περιορισμού ασφάλειας που αναφέρθηκε στο 4.5, προτείνεται η επέκταση του install.sh και του config_flow.py, ώστε να υποστηρίζουν προαιρετικά πεδία username/password και πιστοποιητικά TLS για τη σύνδεση στον MQTT broker. Η επέκταση πρέπει να παραμένει πλήρως συμβατή, μεταξύ υπάρχουσας υλοποίησης, με την τρέχουσα ανώνυμη πρόσβαση: αν τα νέα πεδία αφεθούν κενά, η συμπεριφορά πρέπει να είναι ταυτόσημη με τη σημερινή, ώστε υπάρχοντες χρήστες με ανώνυμο Mosquitto add-on να μη χρειαστεί να αλλάξουν τίποτα. Η ίδια λογική θα ισχύει στο integration: τα username/password/TLS fields θα εμφανίζονται στο config flow ως optional, με κατάλληλες προεπιλογές ασφαλείας. Με την προσθήκη αυτή, το σύστημα γίνεται κατάλληλο και για εγκαταστάσεις με authenticated brokers ή για δίκτυα όπου απαιτείται κρυπτογραφημένη επικοινωνία, χωρίς να επιβαρύνει το απλό οικιακό σενάριο.

Ενσωμάτωση εξωτερικού feedback για την κατάσταση συσκευών. Όπως αναπτύχθηκε στο 4.5, η μονόδρομη φύση της IR επικοινωνίας δεν επιτρέπει στο σύστημα να γνωρίζει την πραγματική κατάσταση των τηλεχειριζόμενων συσκευών.

Παρότι αυτό είναι φυσικό όριο της τεχνολογίας IR και δεν αντιμετωπίζεται με αλλαγή του πρωτοκόλλου, μπορεί να μετριαστεί με την προσθήκη συμπληρωματικών αισθητήρων που παρέχουν έμμεση πληροφορία κατάστασης:

- (1) Για παράδειγμα, μια έξυπνη πρίζα (smart plug) με δυνατότητα μέτρησης κατανάλωσης ρεύματος, τοποθετημένη μεταξύ της παροχής ρεύματος και της τηλεόρασης, επιτρέπει στο Home Assistant να συμπεράνει αν η τηλεόραση είναι όντως αναμμένη ή σβηστή με βάση το επίπεδο κατανάλωσης.
- (2) Μια απλούστερη και χαμηλότερου κόστους εναλλακτική, εφαρμόσιμη για συσκευές με οθόνη όπως οι τηλεοράσεις, είναι η προσθήκη ενός αισθητήρα φωτεινότητας απευθείας πάνω στην «γέφυρα», με τρόπο ώστε να «βλέπει» την κάτω δεξιά γωνία της οθόνης (ή το status LED της τηλεόρασης): η σαφής διαφορά φωτεινότητας μεταξύ αναμμένης και σβηστής κατάστασης αρκεί για να συμπεράνει αξιόπιστα η «γέφυρα» την πραγματική κατάσταση και να τη δημοσιεύσει ως MQTT μήνυμα κατάστασης. Αντίστοιχα για τα κλιματιστικά, ένας αισθητήρας θερμοκρασίας στο δωμάτιο επιτρέπει επαλήθευση της λειτουργίας τους. Το integration θα μπορούσε να εκθέτει optional configuration fields για τη σύνδεση κάθε remote device με αντίστοιχη entity feedback (π.χ. έναν power sensor), ώστε να αναπαριστά αξιόπιστα την πραγματική κατάσταση αντί να βασίζεται μόνο στις εντολές που έχει στείλει.

Οντότητες τύπου `media_player`. Στην τρέχουσα υλοποίηση, κάθε καταγεγραμμένο πλήκτρο εκτίθεται στο Home Assistant ως ξεχωριστό button entity (3.4.Γ) — μια αναπαράσταση απλή και γενική, αλλά όχι ιδανική για συσκευές πολυμέσων όπως τηλεοράσεις και ηχοσυστήματα. Το Home Assistant προσφέρει εξειδικευμένη οντότητα τύπου `media_player`, η οποία παρέχει ενσωματωμένα controls (volume slider, play/pause/stop, mute toggle, source selector) με βελτιωμένη παρουσίαση στα dashboards και υποστήριξη φωνητικού ελέγχου. Σε μελλοντική έκδοση, το integration θα μπορούσε να προσφέρει στον χρήστη την επιλογή να δηλώσει μια συσκευή ως «τύπου `media_player`» και να αντιστοιχίσει τα καταγεγραμμένα πλήκτρα (power, volume up/down, mute, source) στα αντίστοιχα standard media controls. Η αντιστοιχισμός μπορεί να είναι είτε χειροκίνητη μέσω του config flow, βάσει συμβατικών ονομάτων πλήκτρων (π.χ. αν υπάρχουν πλήκτρα με ονόματα "power", "vol_up", "vol_down", "mute", να προσφέρεται αυτόματα η αναβάθμιση σε `media_player`). Τα ίδια raw timing δεδομένα και η ίδια «γέφυρα» εξακολουθούν να χρησιμοποιούνται· αλλάζει μόνο το επίπεδο αναπαράστασης στο Home Assistant.

BIBΛΙΟΓΡΑΦΙΑ

- [1] Rye, Dave (October 1999). "My Life at X10". AV and Automation Industry eMagazine. AV and Automation Industry eMagazine. Retrieved October 8, 2014.
- [2] L. Atzori, A. Iera and G. Morabito, "The Internet of Things: A survey," Computer Networks, vol. 54, no. 15, pp. 2787–2805, 2010.
- [3] Home Assistant. [Online]. Available: <https://www.home-assistant.io>

- [4] openHAB. [Online]. Available: <https://www.openhab.org>
- [5] SmartThings. [Online]. Available: <https://www.smarthings.com>
- [6] Logitech, "Harmony Remote Manufacturing, Service and Support Update," 2021. [Online]. Available: <https://support.myharmony.com/en-us/harmony-remote-manufacturing-update>
- [7] Broadlink, "RM Pro Universal Remote." [Online]. Available: <https://www.ibroadlink.com>
- [8] ESPHome, "Remote Receiver / Remote Transmitter components." [Online]. Available: https://esphome.io/components/remote_transmitter.html
- [9] Tasmota, "IR Remote." [Online]. Available: <https://tasmota.github.io/docs/Tasmota-IR/>
- [10] HACS — Home Assistant Community Store. [Online]. Available: <https://hacs.xyz>
- [11] M. Alaa, A. A. Zaidan, B. B. Zaidan, M. Talal and M. L. M. Kiah, "A review of smart home applications based on Internet of Things," Journal of Network and Computer Applications, vol. 97, pp. 48–65, 2017.
- [12] F. Bonomi, R. Milito, J. Zhu and S. Addepalli, "Fog computing and its role in the Internet of Things," in Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing (MCC '12), pp. 13–16, 2012.
- [13] Apple, "Apple Home." [Online]. Available: <https://www.apple.com/home-app>
- [14] Google, "Google Home." [Online]. Available: <https://home.google.com/welcome>
- [15] Domoticz. [Online]. Available: <https://www.domoticz.com>
- [16] B. L. Risteska Stojkoska and K. V. Trivodaliev, "A review of Internet of Things for smart home: Challenges and solutions," Journal of Cleaner Production, vol. 140, pp. 1454–1464, 2017.
- [17] Amazon, "Alexa Smart Home." [Online]. Available: <https://amazon.com/alexa-smart-home>
- [18] SB-Projects, "IR Remote Control." [Online]. Available: <https://www.sbprojects.net/knowledge/ir/>

- [19] Vishay Semiconductors, "Data Formats for IR Remote Control," Application Note, Document No. 80071, Rev. 2.3, Jun. 26, 2024. [Online]. Available: <https://www.vishay.com/docs/80071/dataform.pdf>.
- [20] LIRC — Linux Infrared Remote Control. [Online]. Available: <https://www.lirc.org>
- [21] A. Banks and R. Gupta, Eds., "MQTT Version 3.1.1," OASIS Standard, October 2014. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>
- [22] Raspberry Pi Foundation. [Online]. Available: <https://www.raspberrypi.com>
- [23] ANAVI Technology, "ANAVI Infrared pHAT (open hardware repository)." [Online]. Available: <https://github.com/AnaviTechnology/anavi-infrared>
- [24] Home Assistant, "Hassfest." [Online]. Available: <https://developers.home-assistant.io/blog/2020/04/16/hassfest>
- [25] HACS, "HACS Action — GitHub Action for HACS integration validation." [Online]. Available: <https://github.com/hacs/action>
- [26] "Pull Request #7409: Add IR “Remote (ANAVI IR pHAT)” integration," HACS default repository. [Online]. Available: <https://github.com/hacs/default/pull/7409>
- [27] D. Parathyras, "pi-dumb2smart-ir-bridge — IR bridge service for Raspberry Pi." [Online]. Available: <https://github.com/h1dd3n3y3/pi-dumb2smart-ir-bridge>
- [28] joan2937, "pigpio — C library for the Raspberry Pi which allows control of the GPIOs." [Online]. Available: <https://github.com/joan2937/pigpio>
- [29] ts1, "PiIR — IR remote control for Raspberry Pi." [Online]. Available: <https://github.com/ts1/PiIR>
- [30] Eclipse Foundation, "Eclipse Paho MQTT Python client." [Online]. Available: <https://www.eclipse.org/paho/index.php?page=clients/python/index.php>
- [31] systemd, “System and Service Manager”. [Online]. Available: <https://systemd.io>
- [32] GitHub, "GitHub Actions documentation." [Online]. Available: <https://docs.github.com/en/actions>

- [33] D. Parathyras, "pi-dumb2smart-ir — Home Assistant integration." [Online]. Available: <https://github.com/h1dd3n3y3/pi-dumb2smart-ir>
- [34] Flipper Devices Inc., "Flipper Zero — Portable Multi-tool Device for Geeks." [Online]. Available: <https://flipperzero.one>
- [35] Lucaslhm, "Flipper-IRDB — community-maintained IR signal database for Flipper Zero." [Online]. Available: https://github.com/Lucaslhm/Flipper-IRDB/blob/main/Audio_and_Video_Receivers/Technics/Technics_EUR6464_96.ir
- [36] Eclipse Foundation, "Eclipse Mosquitto — An open source MQTT broker." [Online]. Available: <https://mosquitto.org>
- [37] crankyoldgit, "IRremoteESP8266 — Send and receive infrared signals with multiple protocols (ESP8266/ESP32)." [Online]. Available: <https://github.com/crankyoldgit/IRremoteESP8266>
- [38] Docker Inc., "Docker — Accelerated container application development." [Online]. Available: <https://www.docker.com>